

# CPU 메커니즘과 스케줄링

## 필기 보존 · 원문과 한국어 보충 번역

제공된 네 PDF의 원본 57쪽을 모두 보존하고, 두 영문 장에 한국어 번역을 덧붙인 통합본입니다. 영문 각 페이지 뒤에 해당 페이지의 번역을 배치했습니다.

한국어 자료와 영어 자료는 서로 다른 장입니다. 원문의 장 번호는 판본에 따라 달라지므로, 아래 주제 순서와 PDF 책갈피를 기준으로 읽으면 됩니다.

### 01 제한적 직접 실행 원리

한국어 원본 16쪽 · 사용자/커널 모드, 시스템 콜, 인터럽트, 문맥 교환

### 02 스케줄링 개요

한국어 원본 14쪽 · FIFO, SJF, STCF, RR, 입출력과 스케줄링

### 03 비례 배분 스케줄링

영문 원본 14쪽 + 한국어 번역 · lottery, stride, CFS

### 04 멀티프로세서 스케줄링

영문 원본 13쪽 + 한국어 번역 · 캐시 일관성, SQMS, MQMS, 부하 균형

## 읽는 방법

본문, 그림, 코드, 참고문헌, 연습문제를 원본과 함께 볼 수 있습니다. 필기는 원본 페이지에서 그대로 확인할 수 있으며 번역 페이지에는 옮겨 적지 않았습니다. 번역은 첨부된 영문판을 기준으로 새로 작성했습니다.

## 번역 범위와 표기

본문의 시대적 설명과 수치는 첨부 원문 기준입니다. 원문 그림은 그대로 보존하고, 번역에서 그림 번호·축·주요 표기를 안내합니다. 코드의 식별자와 명령행 옵션, 참고문헌의 저자·제목은 원문 표기를 유지합니다.

## 제한적 직접 실행 원리

CPU를 가상화하기 위해서 운영체제는 여러 작업들이 동시에 실행되는 것처럼 보이도록 물리적인 CPU를 공유한다. 기본적인 아이디어는 간단하다. 한 프로세스를 잠시 동안 실행하고 다른 프로세스를 또 잠깐 실행하고, 이런 식으로 계속해서 잠깐씩 실행시키면 된다. 이러한 방식처럼 **CPU 시간을 나누어 씀으로써** 가상화를 구현할 수 있다.

그러나 이러한 가상화 기법을 구현하기 위해서는 몇 가지 문제를 해결해야 한다. 첫 번째는 **성능 저하**이다. 시스템에 과중한 오버헤드를 주지 않으면서 가상화를 구현할 수 있을까? 두 번째는 **제어 문제**이다. CPU에 대한 통제를 유지하면서 프로세스를 효율적으로 실행시킬 수 있는 방법은 무엇인가? 운영체제의 입장에서는 자원 관리의 책임자로서 특히 제어 문제가 중요하다. 제어권을 상실하면 한 프로세스가 영원히 실행을 계속할 수 있고 컴퓨터를 장악하거나 접근해서는 안 되는 정보에 접근하게 된다. 제어권을 유지하면서 성능 저하가 없도록 하는 것이 운영체제를 구축하는 데 핵심적인 도전 과제이다.

### 핵심 질문: 제어를 유지하면서 효과적으로 CPU를 가상화하는 방법

운영체제는 효율적인 방식으로 CPU를 가상화하지만 시스템에 대한 제어를 잃지는 않는다. 제어를 잃지 않기 위해서 하드웨어와 운영체제의 지원이 필수적이다. 종종 운영체제는 작업을 효과적으로 수행하기 위하여 하드웨어가 제공하는 기능을 신중하게 사용한다.

### 9.1 기본 원리: 제한적 직접 실행

운영체제 개발자들은 프로그램을 빠르게 실행하기 위하여 **제한적 직접 실행(Limited Direct Execution)**이라는 기법을 개발하였다. 이 아이디어의 “직접 실행”에 해당하는 부분은 간단하다. 프로그램을 CPU 상에서 그냥 직접 실행시키는 것이다. 따라서 운영체제가 프로그램을 실행하기 시작할 때 프로세스 목록에 해당 프로세스 항목을 만들고 메모리를 할당하며 프로그램 코드를 디스크에서 탑재하고 진입점(예, `main()` 루틴 혹은 유사한 무엇)을 찾아 그 지점으로 분기하여 사용자 코드를 실행하기 시작한다. 그림

| 운영체제                              | 프로그램                                            |
|-----------------------------------|-------------------------------------------------|
| 프로세스 목록의 항목을 생성                   |                                                 |
| 프로그램 메모리 할당                       |                                                 |
| 메모리에 프로그램 탑재                      |                                                 |
| <code>argc/argv</code> 를 위한 스택 셋업 |                                                 |
| 레지스터 내용 삭제                        |                                                 |
| <code>call main()</code> 실행       | <code>main()</code> 실행                          |
|                                   | <code>main</code> 에서 <code>return</code> 명령어 실행 |
| 프로세스 메모리 반환                       |                                                 |
| 프로세스 목록에서 항목 제거                   |                                                 |

### 〈그림 9.1〉 직접 실행 프로토콜(제한 없음)

9.1은 이 기본적인 직접 실행 프로토콜(아무 제한이 없는)을 보이고 있다. 그림 9.1은 프로그램의 `main()`으로 분기하고 커널로 되돌아 가기 위해 일반적인 호출과 리턴을 사용하였다.

간단하다, 그렇지? 그러나 이 접근법은 CPU를 가상화함에 있어 몇 가지 문제를 일으킨다. 첫 번째는 간단한 문제이다. 프로그램을 직접 실행시킨다면 프로그램이, 운영체제가 원치않는 일을 하지 않는다는 것을 어떻게 보장할 수 있는가? 두 번째 문제는 프로세스 실행 시, 운영체제는 어떻게 프로그램의 실행을 중단하고 다른 프로세스로 전환시킬 수 있는가, 즉, CPU를 가상화하는 데 필요한 시분할(time sharing) 기법을 어떻게 구현할 수 있는냐는 것이다.

아래에서 이와 같은 질문에 답하면서 CPU를 가상화하는 데 필요한 사항을 더 잘 이해하게 될 것이다. 이러한 기법을 발전시키는 과정 중에 “제한적”이라는 이름이 어디서 비롯되었는지 보게 될 것이다. 프로그램 실행에 제한을 두지 않으면 운영체제는 어떠한 것도 제어할 수 없으며 따라서 단순한 라이브러리일 뿐이다. 장치 운영체제로 발전하기 위해서는 매우 슬픈 상황이다.

## 9.2 문제점 1: 제한된 연산

직접 실행의 장점은 빠르게 실행된다는 것이다. 기본적으로 프로그램이 하드웨어 CPU에서 실행되기 때문이다. 그러나 CPU에서 직접 실행시키면 새로운 문제가 발생한다. 만일 프로세스가 특수한 종류의 연산을 수행하길 원한다면 어떻게 될 것인가? 이러한 연산에는 디스크 입출력 요청이나 CPU 또는 메모리와 같은 시스템 자원에 대한 추가할당 요청 등이 포함된다.

프로세스가 원하는 대로 할 수 있게 방치하는 방안이 있다. 그러나 이 방안은 바람직한 시스템을 구축하는 데에는 방해 요인이다. 파일에 대한 접근을 허용하기 전에 접근 권한을 검사하는 파일 시스템을 구현하는 것을 예를 들어 보자. 프로세스가 디스크에

**핵심 질문: 제한 연산을 수행하는 방법**

프로세스는 입출력 연산을 비롯한 다른 제한된 연산을 수행해야 한다. 그러나, 프로세스는 시스템에 대한 권한이 없기 때문에 제한된 연산을 수행할 수 없다. 이 일을 위해 운영체제와 하드웨어가 할 일은 무엇인가?

**팁: 보호된 제어 양도**

하드웨어는 두 가지 실행 모드를 제공하여 운영체제를 돕는다. **사용자 모드(user mode)**에서 응용 프로그램은 하드웨어 자원에 대한 접근 권한이 일부 제한되어 있다. 운영체제는 컴퓨터의 모든 자원에 대한 접근 권한을 **커널 모드(kernel mode)**에서 가진다. 이를 위하여 커널 모드로 진입하기 위한 **trap** 명령어와 사용자 모드로 돌아가기 위한 **return-from-trap** 명령어가 제공된다. 또한, 운영체제가 하드웨어에게 트랩 테이블(trap table)의 메모리 주소를 알려주기 위한 명령어도 함께 제공된다.

대하여 입출력하는 것을 제한하지 않으면 프로세스는 전체 디스크를 읽고 쓸 수 있기 때문에 접근 권한을 검사하는 기능이 아무런 의미가 없다.

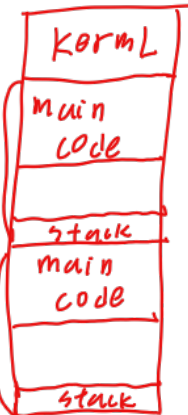
이 때문에 사용자 모드(user mode)라고 알려진 새로운 모드가 도입되었다. 사용자 모드에서 실행되는 코드는 할 수 있는 일이 제한된다. 예를 들어, 프로세스가 사용자 모드에서 실행 중이면 입출력 요청을 할 수 없도록 설정한다. 이때 입출력 요청을 하면 프로세서가 예외를 발생시키고, 운영체제는 해당 프로세스를 제거한다.

커널 모드(kernel mode)는 사용자 모드와 대비되는 모드로서 운영체제의 중요한 코드들이 실행된다. 이 모드에서 실행되는 코드는 모든 특수한 명령어를 포함하여 원하는 모든 작업을 수행할 수 있다.

그러나 우리에게 아직 해결해야 할 한 가지 문제가 남아 있다. 사용자 프로세스가 디스크를 읽기와 같은 특권 명령어를 실행해야 할 때는 어떻게 해야 하는가? 이런 제한 작업의 실행을 허용하기 위하여 거의 모든 현대 하드웨어는 사용자 프로세스에게 **시스템 콜**을 제공한다. Atlas [Kil+62; Lav78]와 같은 커널은 시스템 콜을 통하여 자신의 주요 기능을 사용자 프로그램에게 제공한다. 이러한 기능에는 파일 시스템 접근, 프로세스 생성 및 제거, 다른 프로세스와의 통신 및 메모리 할당 등이 포함된다. 대부분의 운영체제는 수백 개의 시스템 콜을 제공한다(자세한 사항에 관해서는 POSIX 표준을 참조하십시오 [P10]). 초기 UNIX 시스템은 약 20개 정도의 시스템 콜을 제공하였다.

시스템 콜을 실행하기 위해 프로그램은 trap 특수 명령어를 실행해야 한다. 이 명령어는 커널 안으로 분기하는 동시에 특권 수준을 커널 모드로 상향 조정한다. 커널 모드로 진입하면 운영체제는 모든 명령어를 실행할 수 있고 이를 통하여 프로세스가 요청한 작업을 처리할 수 있다. 완료되면 운영체제는 **return-from-trap** 특수 명령어를 호출한다. 예상하는 것처럼 이 명령어는 특권 수준을 사용자 모드로 다시 하향 조정하면서 호출한 사용자 프로그램으로 리턴한다.

하드웨어는 **trap** 명령어를 수행할 때 주의가 필요하다. 호출한 프로세스의 필요한



유저 모드에서  
PA는 PB로  
연결X

### 여담: 왜 시스템 콜은 프로시저 콜과 비슷하게 보일까

`open()` 또는 `read()`와 같은 시스템 콜을 호출하는 형태는 C 프로그램의 전형적인 프로시저 호출의 형태와 똑같다. 즉, 만약 프로시저 호출과 똑같다면 운영체제는 시스템 콜을 어떻게 구분할 수 있으며 그에 맞는 작업을 할 수 있는가? 간단한 이유: 진짜 프로시저 호출이지만 유명한 `trap` 명령어가 그 안에 감추어져 있다. 더 정확하게 설명하면, 예를 들어, `open()`을 호출할 때, C 라이브러리의 프로시저 호출을 수행하고 있는 것이다. `open()`(다른 시스템 콜들도 마찬가지이다)이 호출되면, 라이브러리는 커널과 약속된 호출 규약을 사용하여 `open`에게 전달할 인자와 시스템 콜 번호를 지정된 장소(스택 또는 특정 레지스터)에 저장한다. 그런 다음 `trap` 명령어를 실행한다. 그 다음에 반환 값을 정리하고 시스템 콜을 호출한 프로그램에게 제어를 넘긴다. 이를 위해서 C 라이브러리에서 시스템 콜을 호출하는 부분의 코드는 어셈블리어로 작성된다. 인자의 반환 값을 올바르게 처리하고, 하드웨어마다 다른 `trap` 명령어를 실행하기 위해 꼭 필요한 일이다. 자 이제 운영체제로 진입하기 위해서 어셈블리 코드 작업을 직접할 필요가 없다는 것을 알았다. 어셈블리 코드는 이미 만들어져 있다.

레지스터들을 저장해야 한다. 운영체제가 `return-from-trap` 명령어 실행 시 사용자 프로세스로 제대로 리턴할 수 있도록 하기 위함이다. 예를 들면, x86에서는 프로그램 카운터, 플래그와 다른 몇 개의 레지스터를 각 프로세스의 커널 스택(kernel stack)에 저장한다. `return-from-trap` 명령어가 이 값들을 스택에서 팝(pop)하여 사용자 모드 프로그램의 실행을 다시 시작한다(Intel systems manual [Int11]을 참조하시오). 다른 하드웨어 시스템은 다른 규약을 사용하긴 하지만 기본적인 개념은 모두 비슷하다.

현재까지의 논의에서 다루지 않은 중요한 사항이 있다. 그것은 `trap`이 운영체제 코드의 어디를 실행할지 어떻게 아느냐는 것이다. 호출한 프로세서는 분기할 주소를 명시할 수 없다. 주소를 명시한다는 것은 커널 내부의 원하는 지점을 접근할 수 있다는 것이기 때문에 위험하다. 커널이 임의의 코드를 실행하기 위해서는 접근 권한 검사가 끝난 후 분기해야 한다 [Sha07]. 이러한 문제 때문에 커널은 `trap` 발생 시 어떤 코드를 실행할지 신중하게 통제해야 한다.

커널은 부팅 시에 트랩 테이블(trap table)을 만들고 이를 이용하여 시스템을 통제한다. 컴퓨터가 부트될 때는 커널 모드에서 동작하기 때문에 하드웨어를 원하는 대로 제어할 수 있다. 운영체제가 하는 초기 작업 중 하나는 하드웨어에게 예외 사건이 일어났을 때 어떤 코드를 실행해야 하는지 알려주는 것이다. 예를 들어, 하드 디스크 인터럽트가 발생하면, 키보드 인터럽트가 발생하면, 또는 프로그램이 시스템 콜을 호출하면 무슨 코드를 실행해야 하는가? 운영체제는 특정 명령어를 사용하여 하드웨어에게 트랩 핸들러(trap handler)의 위치를 알려준다. 하드웨어는 이 정보를 전달받으면 해당 위치를 기억하고 있다. 따라서 시스템 콜과 같은 예외적인 사건이 발생했을 때 하드웨어는 무엇을 해야 할지(즉, 어느 코드로 분기하여 실행할지) 알 수 있다.

마지막 하드웨어에게 트랩 테이블의 위치를 알려주는 것은 매우 강력한 기능이다. 당연히 이 역시 특권 명령어이다. 사용자 모드에서 이 명령어를 실행하려고 하면 실행할

| 운영체제 @부트<br>(커널 모드)                                                                                       | 하드웨어                                                         |                                                          |
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------|
| 트랩 테이블을 초기화한다                                                                                             | syscall 핸들러의 주소를 기억한다                                        |                                                          |
| 운영체제 @실행<br>(커널 모드)                                                                                       | 하드웨어                                                         | 프로그램<br>(사용자 모드)                                         |
| 프로세스 목록에 항목을 추가한다<br>프로그램을 위한 메모리를 할당한다<br>프로그램을 메모리에 탑재한다<br>argv를 사용자 스택에 저장한다<br>레지스터와 PC를 커널 스택에 저장한다 |                                                              |                                                          |
| <b>return-from-trap</b>                                                                                   | 커널 스택으로부터 레지스터를 복원한다<br>다<br>사용자 모드로 이동한다<br>main으로 분기한다     | <b>main()</b> 을 실행한다<br>...<br>시스템 콜을 호출한다<br>운영체제로 트랩한다 |
| 트랩을 처리한다<br>syscall의 임무를 수행한다<br><b>return-from-trap</b>                                                  | 레지스터를 커널 스택에 저장한다<br>커널 모드로 이동한다<br>트랩 핸들러로 분기한다             |                                                          |
|                                                                                                           | 커널 스택으로부터 레지스터를 복원한다<br>다<br>사용자 모드로 이동한다<br>트랩 이후의 PC로 분기한다 | main에서 리턴한다<br><b>trap(exit())</b> 를 통하여                 |
| 프로세스의 메모리를 반환한다<br>프로세스 목록에서 제거한다                                                                         |                                                              |                                                          |

### 〈그림 9.2〉 제한적 직접 실행 프로토콜

수 없고, 어떤 결과가 올지 알고 있을 것이다(힌트, 잘 가거라. 프로그램). 숙고해 볼 주제: 만약 당신이 자신의 트랩 테이블을 설치할 수 있다면, 시스템에 어떤 황당한 일이 일어날까? 컴퓨터를 장악할 수 있는가?

그림 9.2는 이 메커니즘을 요약해서 나타내고 있다. 연대표는 시간의 흐름에 따라 아래 방향으로 진행된다. 프로세스는 커널 스택을 각자 가지고 있다. 커널 모드로 진입하거나 진출할 때 하드웨어에 의해 프로그램 카운터와 범용 레지스터 등의 레지스터가 저장되고 복원되는 용도로 사용된다.

LDE 프로토콜은 두 단계로 진행된다. 전반부에서(부팅 시) 커널은 트랩 테이블을 초기화하고 CPU는 나중에 사용하기 위하여 테이블의 위치를 기억한다. 커널은 이러한 작업을 커널 모드에서만 사용할 수 있는 명령어를 이용하여 수행한다(이러한 명령어는 굵게 강조하였다).

후반부에서(프로세스를 실행할 때) `return-from-trap`을 이용하여 사용자 프로세스를 시작할 때 몇 가지 작업을 수행한다. 새로운 프로세스를 위한 노드를 할당하여 프로세스 리스트에 삽입하고, 메모리를 할당하는 등의 작업이 포함된다. `return-from-trap` 명령어는 CPU를 사용자 모드로 전환하고 프로세스 실행을 시작한다. 프로세스가 시스템 콜을 호출하면 운영체제로 다시 트랩된다. 운영체제는 시스템 콜을 처리하고 `return-from-trap` 명령어를 사용하여 다시 제어를 프로세스에게 넘긴다. 프로세스는 이후 자신의 할 일을 다하면 `main()`에서 리턴한다. 이때 일반적으로 스텝 코드로 리턴하고 스텝 코드가 프로그램을 종료시킨다. 종료시킬 때 `exit()` 시스템을 호출하고 다시 운영체제로 트랩된다. 이 시점에 운영체제는 정리 작업을 하게 되어 모든 일이 완료된다.

### 9.3 문제점 2: 프로세스 간 전환

직접 실행의 두 번째 문제점은 프로세스 간 전환을 할 수 있어야 한다는 점이다. 프로세스 간 전환은 당연히 간단해야 한다. 맞지? 운영체제는 실행 중인 프로세스를 계속 실행할 것인지, 멈추고 다른 프로세스를 실행할 것인지를 결정해야 한다. 이게 뭐가 문제지? 실제로는 까다로운 문제이다. CPU에서 프로세스가 실행 중이라는 것은 운영체제는 실행 중이지 않다는 것을 의미한다. 운영체제가 실행하고 있지 않다면 어떻게 이런 일들을 할 수 있을까? (힌트: 할 수 없다). 거의 철학적으로 들릴지 모르겠지만 이건 진짜 심각한 문제다. CPU에서 실행하고 있지 않다면 운영체제는 어떠한 조치도 취할 수 없다. 이제 핵심 문제에 도달했다.

**핵심 질문: CPU를 어떻게 다시 획득할 수 있는가**  
운영체제는 어떻게 CPU를 다시 획득하여 프로세스를 전환할 수 있는가?

#### 협조 방식: 시스템 콜 기다리기

협조(cooperative) 방식으로 알려진 방법은 과거의 몇몇 시스템에서 채택되었던 방식이다. 예를 들어, Macintosh 운영체제의 초기 버전 [Mac11], 또는 오래 전 Xerox Alto 시스템 [Alt79]이 채택했던 방식이다. 이 방식에서는 운영체제가 프로세스들이 합리적으로 행동할 것이라고 신뢰한다. 너무 오랫동안 실행할 가능성이 있는 프로세스는 운영체제가 다른 작업을 실행할 결정을 할 수 있도록 주기적으로 CPU를 포기할 것이라고 가정한다.

그러면 이렇게 질문할 수 있을 것이다: 이 이상적인 환경에서 우호적인 프로세스는 어떻게 CPU를 포기할 수 있을까? 알려진 것처럼 대부분의 프로세스는 자주 시스템 콜을 호출하여 CPU의 제어권을 운영체제에게 넘겨준다. 예를 들어, 파일을 열고 읽는 작업을 한다거나 다른 컴퓨터에게 메시지를 송신하거나 또는 새 프로세스를 생성하는 등의 시스템 콜을 호출한다. 이런 유형의 운영체제는 yield 시스템 콜을 제공하는데,

너무 많을 때

이 시스템 콜은 운영체제에게 제어를 넘겨 운영체제가 다른 프로세스를 실행할 수 있게 한다.

#### 팁: 응용 프로그램의 오작동 처리하기

운영체제는 종종 오작동 프로세스를 처리해야 한다. 오작동하는 프로세스란 악의적으로 설계되었거나 실수로 인한 버그 때문에 해서는 안 될 무언가를 하려는 프로세스를 말한다. 현대 시스템에서 운영체제가 그러한 부정 행위를 처리하는 방법은 단순히 행위자를 종료시키는 것이다. 윈 스트라이크에 아웃! 조금 가혹하다고 할지는 모르겠지만, 만약 메모리에 불법적으로 접근하거나 불법적인 명령어를 실행할 때 운영체제가 취할 수 있는 일은 무엇이 있을까?

응용 프로그램이 비정상적인 행위를 하게 되면 운영체제에게 제어가 넘어간다. 예를 들어 응용 프로그램이 어떤 수를 0으로 나누는 연산을 실행하거나 접근할 수 없는 메모리에 접근하려고 하면 운영체제로의 트랩이 일어난다. 그러면 운영체제는 다시 CPU를 획득하여 해당 행위를 하는 프로세스를 종료할 수 있다.

협조 방식의 스케줄링 시스템에서 운영체제는 시스템 콜이 호출되기를 기다리거나 불법적인 연산이 일어나기를 기다려서 CPU의 제어권을 다시 획득한다. 이렇게 생각할 수도 있다: 이상적이라기보다 수동적인 방법 아닐까? 예를 들어 악의적이든 버그로 인한 것이든 프로세스가 무한 루프에 빠져 시스템 콜을 호출할 수 없을 때에는 어떤 일이 벌어지는가? 운영체제가 할 수 있는 일은 무엇인가?

#### 비협조 방식: 운영체제가 전권을 행사

프로세스가 시스템 콜을 호출하기를 거부하거나 실수로 호출하지 않아 운영체제에게 제어를 넘기지 않을 경우 하드웨어의 추가적인 도움없이 운영체제가 할 수 있는 일은 거의 없다. 사실, 협조적 방법에서 프로세스가 무한 루프에 빠졌을 경우 할 수 있는 일은 컴퓨터 시스템의 모든 문제를 해결해 왔던 아주 오래된 방법을 동원하는 수밖에 없다. 바로 컴퓨터를 다시 부팅하는 것이다. CPU의 제어권을 획득하기 위한 과정에서 해결해야 할 사소한 문제를 만나게 된다.

#### 핵심 질문: 협조 없이 제어를 얻는 방법

프로세스가 비협조적인 상황에서도 CPU의 제어를 획득하는 방법은 무엇인가? 악의적인 프로세스가 컴퓨터를 장악하지 않도록 보장하기 위하여 운영체제는 무엇을 할 수 있을까?

그 해결책은 간단한 것으로 수십 년 전에 컴퓨터 시스템을 만들었던 많은 사람들에 의해 발명되었다. 해결책은 타이머 인터럽트(timer interrupt)를 이용하는 것이다 [McC+63]. 타이머 장치는 수 밀리 초마다 인터럽트를 발생시키도록 프로그램

마지막으로 interrupt가 되게

가능하다. 인터럽트가 발생하면 현재 수행 중인 프로세스는 중단되고 미리 구성된 운영체제의 **인터럽트 핸들러(interrupt handler)**가 실행된다. 이 시점에 운영체제는 CPU 제어권을 다시 얻게 되고 자신이 원하는 일을 할 수 있다. 현재의 프로세스를 중단하고 다른 프로세스를 실행시키는 작업 등이 해당된다.

#### 팁 : 제어를 다시 획득하기 위해 타이머 인터럽트 사용하기

**타이머 인터럽트** 기능을 추가하면 운영체제는 프로세스가 비협조적으로 행동하는 상황에서도 CPU 상에서 실행될 수 있는 능력을 가지게 된다. 타이머 인터럽트 하드웨어 기능은 운영체제가 컴퓨터의 제어를 유지하는 핵심적인 기능이다.

운영체제는 하드웨어에게 타이머 인터럽트가 발생했을 때 실행해야 할 코드<sup>1</sup>를 알려 주어야 한다. 부팅될 때 운영체제가 이런 준비를 한다. 부팅 과정 진행 중에 운영체제는 타이머를 시작한다. 타이머가 시작되면 운영체제는 자신에게 제어가 돌아올 것이라는 것을 알고 부담 없이 사용자 프로그램을 실행할 수 있다. 또한, 타이머는 특정 명령어를 수행하여 끝 수도 있다. 이에 대한 자세한 사항은 병행성을 설명할 때 논의할 것이다.

인터럽트 발생 시 하드웨어에게도 약간의 역할이 있다. 인터럽트가 발생했을 때 실행 중이던 프로그램의 상태를 저장하여 나중에 **return-from-trap** 명령어가 프로그램을 다시 시작할 수 있도록 해야 한다. 이러한 일련의 동작은 시스템 콜이 호출되었을 때 하드웨어가 하는 동작과 매우 유사하다. 다양한 레지스터가 커널 스택에 저장되고, **return-from-trap** 명령어를 통하여 복원된다.

### 문맥의 저장과 복원

시스템 콜을 통하여 협조적으로 하던, 또는 타이머 인터럽트를 통하여 약간은 강제로 하던, 운영체제가 제어권을 다시 획득하면 중요한 결정을 내려야 한다. 즉, 현재 실행 중인 프로세스를 계속 실행할 것인지 아니면 다른 프로세스로 전환할 것인지를 결정해야 한다. 이 결정은 운영체제의 **스케줄러(scheduler)**라는 부분에 의해 내려진다. 우리는 이후의 장에서 스케줄링 정책에 관해 상세하게 논의할 것이다.

다른 프로세스로 전환하기로 결정되면 운영체제는 **문맥 교환(context switch)**이라고 알려진 코드를 실행한다. 문맥 교환은 개념적으로는 간단하다. 운영체제가 해야 하는 작업은 현재 실행 중인 프로세스의 레지스터 값을 커널 스택 같은 곳에 저장하고 곧 실행될 프로세스의 커널 스택으로부터 레지스터 값을 복원하는 것이 전부다. 그렇게 함으로써 운영체제는 **return-from-trap** 명령어가 마지막으로 실행될 때 현재 실행 중이던 프로세스로 리턴하는 것이 아니라 다른 프로세스로 리턴하여 실행을 다시 시작할 수 있다.

프로세스 전환을 위하여 운영체제는 저수준 어셈블리 코드를 사용하여 현재 실행 중인 프로세스의 범용 레지스터, PC뿐 아니라 현재 커널 스택 포인터를 저장한다.

1) 특정 작업을 대처하는 코드

그리고 곧 실행될 프로세스의 범용 레지스터, PC를 복원하고 커널 스택을 이 프로세스의 커널 스택으로 전환한다. 이로써 운영체제는 인터럽트된 프로세스 문맥에서 전환 코드를 호출하고, 실행될 프로세스 문맥으로 리턴할 수 있다. 운영체제가 마지막으로 **return-from-trap** 명령어를 실행하면 곧 실행될 프로세스가 현재 실행 중인 프로세스가 된다. 그래서 문맥 교환이 마무리 된다.

이 모든 과정의 연대표가 그림 9.3에 나와 있다. 이 예에서 프로세스 A가 실행 중이고 타이머 인터럽트에 의해 중단된다. 하드웨어는 A의 레지스터를 커널 스택에 저장하고 커널 모드로 진입한다. 타이머 인터럽트 핸들러에서 운영체제는 프로세스 B로 전환하기로 결정한다. 이 시점에 **switch()** 루틴을 호출한다. 이 루틴이 A의 레지스터의 현재 값을 A의 프로세스 구조체에 저장하고 B의 프로세스 구조체에서 B의 레지스터를 복원한다. 그런 후에 A의 커널 스택이 아니라 B의 커널 스택을 사용하도록 스택 포인터를 바꾸어서 **문맥 교환**을 수행한다. 마지막으로 운영체제는 **return-from-trap**을 수행하여 B의

| 운영체제 @부트<br>(커널 모드)                                                                                                                                | 하드웨어                                                         |                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|------------------|
| 트랩 테이블을 초기화 한다                                                                                                                                     |                                                              |                  |
|                                                                                                                                                    | syscall 핸들러, 타이머 핸들러의 주소를 기억한다                               |                  |
| 인터럽트 타이머를 시작시킨다                                                                                                                                    | 타이머를 시작시킨다<br>X msec 지난 후 CPU를 인터럽트한다                        |                  |
| 운영체제 @실행<br>(커널 모드)                                                                                                                                | 하드웨어                                                         | 프로그램<br>(사용자 모드) |
|                                                                                                                                                    |                                                              | 프로세스 A           |
|                                                                                                                                                    |                                                              | ...              |
|                                                                                                                                                    | 타이머 인터럽트<br>A의 레지스터를 A의 커널 스택에 저장<br>커널 모드로 이동<br>트랩 핸들러로 분기 |                  |
| 트랩을 처리한다<br><b>switch()</b> 루틴 호출<br>A의 레지스터를 A의 proc 구조체에 저장<br>B의 proc 구조체로부터 B의 레지스터를 복원<br>B의 커널 스택으로 전환<br><b>return-from-trap</b> (B 프로세스 로) |                                                              |                  |
|                                                                                                                                                    | B의 커널 스택을 B의 레지스터로 저장<br>사용자 모드로 이동<br>B의 PC로 분기             |                  |
|                                                                                                                                                    |                                                              | 프로세스 B           |
|                                                                                                                                                    |                                                              | ...              |

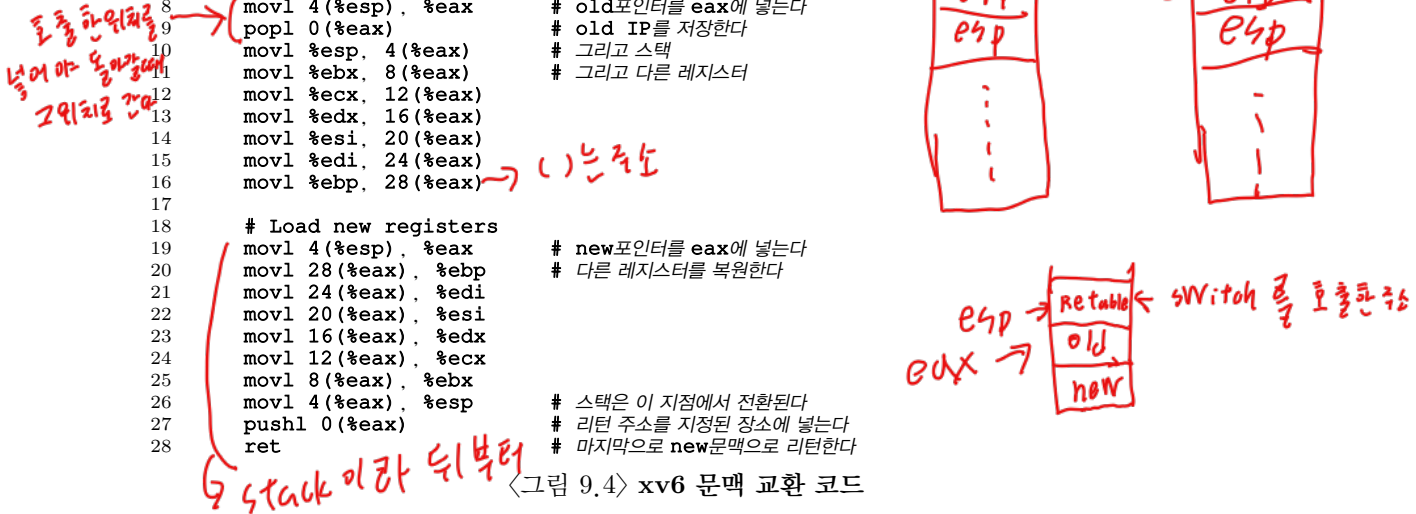
〈그림 9.3〉 제한적 직접 실행 프로토콜(타이머 인터럽트)

```

1 # void switch(struct context *old, struct context *new);
2 #
3 # Save current register context in old
4 # and then load register context from new.
5 .globl switch
6 switch:
7     # Save old registers
8     movl 4(%esp), %eax      # old 포인터를 eax에 넣는다
9     popl 0(%eax)           # old IP를 저장한다
10    movl %esp, 4(%eax)      # 그리고 스택
11    movl %ebx, 8(%eax)      # 그리고 다른 레지스터
12    movl %ecx, 12(%eax)
13    movl %edx, 16(%eax)
14    movl %esi, 20(%eax)
15    movl %edi, 24(%eax)
16    movl %ebp, 28(%eax)
17
18    # Load new registers
19    movl 4(%esp), %eax      # new 포인터를 eax에 넣는다
20    movl 28(%eax), %ebp     # 다른 레지스터를 복원한다
21    movl 24(%eax), %edi
22    movl 20(%eax), %esi
23    movl 16(%eax), %edx
24    movl 12(%eax), %ecx
25    movl 8(%eax), %ebx
26    movl 4(%eax), %esp
27    pushl 0(%eax)
28    ret

```

← 뒤에서 복귀 push



레지스터를 복원하고 실행을 시작한다.

이 과정이 실행되는 동안 두 번의 레지스터의 저장/복원이 일어난다는 것에 주의하자. 첫 번째는 타이머 인터럽트가 발생했을 때 일어난다. 이 경우 실행 중인 프로세스의 사용자 레지스터가 하드웨어에 의해 암묵적으로 저장되고 저장 장소로 해당 프로세스의 커널 스택이 사용된다. 두 번째는 운영체제가 A에서 B로 전환하기로 결정했을 때 일어난다. 이 경우 커널 레지스터는 운영체제에 의하여 해당 프로세스의 프로세스 구조체에 저장된다. 이것은 운영체제가 A가 아닌 B로부터 커널로 트랩된 것처럼 만든다.

이러한 문맥 교환이 어떻게 일어나는지 잘 이해할 수 있도록 xv6의 문맥 교환 코드를 그림 9.4에 보이고 있다. 우선 이해할 수 있는지 살펴보기 바란다. 물론 xv6와 x86에 관한 얼마간의 지식을 가지고 있어야 한다. **context** 구조체 **old**와 **new**는 구 프로세스와 새 프로세스의 프로세스 구조체 안에 위치해 있다.

## 9.4 병행성이 걱정

집중력과 사고력이 좋은 몇몇 친구는 지금 이런 생각을 하고 있을지 모른다. “음, 만약 시스템 콜을 처리하는 도중에 타이머 인터럽트가 발생하면 어떤 일이 발생하는가?” 또는 “하나의 인터럽트를 처리하고 있을 때 다른 인터럽트가 발생하면 어떤 일이 생기는가? 커널에서 처리하기 더 어려워지는 것은 아닌가?” 좋은 질문. 우리에게엔 아직 희망이 있다!

대답은 “그렇다” 이다. 운영체제는 인터럽트 또는 트랩을 처리하는 도중에 다른 인터럽트가 발생할 때 어떤 일이 생기는가에 대해 신중하게 고려할 필요가 있다. 사실

### 여담: 문맥 교환하는 데 걸리는 시간은 얼마인가

떠올릴 수 있는 자연스러운 질문은 문맥 교환과 같은 작업의 처리 소요 시간이다. 혹은 시스템 콜의 처리 소요 시간은? 호기심 많은 당신을 위해 lmbench [MS96]라는 도구가 있다. 이 도구는 언급한 그러한 작업의 시간을 정확하게 측정할 뿐 아니라 연관된 다른 성능 수치도 측정한다.

결과는 시간이 지나면서 대략 프로세서의 성능개선 추이와 비슷하게 점점 좋아졌다. 예를 들어, 1996년에 200 MHz P6 CPU에서 실행되는 Linux 1.3.37의 경우 시스템 콜은 4마이크로초 그리고 문맥 교환에는 대략 6마이크로초가 소요되었다 [MS96]. 현대 시스템에서는 자릿수가 달라질 정도로 성능이 좋아져서 2 또는 3 GHz 프로세서의 경우 1마이크로초 미만이 소요된다.

운영체제의 모든 동작이 CPU 성능에 따라 좋아지는 것이 아니라는 것에 주의하자. Ousterhout가 발견한 것처럼 운영체제의 많은 연산은 주로 메모리를 접근하는 연산이며 메모리의 대역폭은 프로세서 속도가 발전하는 것만큼 극적으로 향상되지 않았기 때문이다 [Ous90]. 사용한 워크로드의 특성에 따라 최신의 좋은 프로세서를 구매하더라도 운영체제의 속도가 기대만큼 증가하지 않을 수 있다.

이 내용이 바로 이 책의 두 번째 부분에서 논의할 주제인 **병행성**에 관한 것으로서 자세한 논의는 그때까지 미룰 것이다.

흥미를 돋우기 위하여 운영체제가 이 다루기 힘든 상황을 어떻게 처리하는지 간략히 알아보자. 운영체제가 할 수 있는 간단한 해법은 인터럽트를 처리하는 동안 인터럽트를 불능화시키는 것이다. 이럴 경우 하나의 인터럽트가 처리되고 있는 동안 다른 어떤 인터럽트도 CPU에게 전달되지 않는다. 물론, 운영체제는 이 작업을 신중하게 해야 한다. 인터럽트를 너무 오랫동안 불능화시키면 인터럽트를 놓치게 되고 기술적으로도 좋지 않다.

운영체제는, 또한 내부 자료 구조에 동시에 접근하는 것을 방지하기 위해 많은 정교한 락(lock) 기법을 개발해 왔다. 이 잠금 기법은 커널 안에서 동시에 다수의 활동이 진행될 수 있게 허용한다. 이 책의 병행성에 관한 부분에서 살펴보겠지만 이런 잠금 기법은 복잡해질 수 있으며 그로 인해 흥미로우면서도 발견하기 힘든 버그를 만들어 낸다.

## 9.5 요약

우리는 CPU 가상화를 구현하기 위한 핵심적인 저수준 기법에 관해 설명하였다. 이런 기법들을 묶어서 **제한적 직접 실행**이라고 부른다. 기본적인 아이디어는 간단하다. CPU에서 실행하고 싶은 프로그램을 실행시킨다. 그러나 운영체제가 CPU를 사용하지 못하더라도 프로세스의 작업을 제한할 수 있도록 하드웨어를 셋업해야 한다.

이러한 일반적인 방법은 실생활에서도 사용된다. 예를 들어, 아이가 있거나 적어도 아이들에 관해 들어 보았다면 방에 **아기 보호 장치**를 설치하는 일에 친숙할 것이다. 위험한 물건이 들어 있는 캐비닛을 잠그고 전기 소켓의 두껍은 덮개를 덮는다거나 하는 일이

**팁: 리부팅은 유용하다**

협조적 선점 모드에서 무한루프와 같은 행동의 유일한 해결책은 컴퓨터를 **리부팅**하는 것이다. 단순 무식한 방법이지만, 연구가들은 리부팅이나 소프트웨어를 다시 시작하는 방법이 강건한 시스템 구축에 있어 매우 유용하다는 것을 보였다 [Can+04].

구체적으로 리부트는 소프트웨어를 이미 알려진 상태 그리고 검증된 상태로 되돌리기 때문에 유용하다. 리부트는 오래된 자원 또는 제어를 벗어난 자원을 시스템에 반환한다. 이러한 자원은 반환하지 않으면 처리하기 곤란하다. 마지막으로 리부트는 자동화하기 쉽다. 이러한 이유들 때문에 시스템 관리 소프트웨어를 위한 대규모 클러스터 인터넷 서비스에서는 컴퓨터 일부를 리셋시키기 위하여 주기적으로 리부팅하는 것이 흔한 일이다. 이렇게 함으로써 앞서 언급한 모든 이점을 얻을 수 있다.

따라서 리부팅한다고 해서 그것이 단순무식한 방법은 아니다. 오히려 컴퓨터 시스템의 동작을 향상시키기 위하여 충분히 검증된 방법을 사용하고 있는 것이다. 잘했어!

포함된다. 이렇게 방이 준비되면 대부분의 위험한 요소들을 막아 놓았기 때문에 아기가 자유롭게 안전하게 돌아다니게 할 수 있다.

비슷한 방식으로 운영체제는 CPU에 안전 장치를 준비해 놓을 수 있다. 우선 부팅할 때 트랩 핸들러 함수를 셋업하고 인터럽트 타이머를 시작시키고 그런 후에 제한된 모드에서만 프로세스가 실행되도록 한다. 이로써 운영체제는 프로세스를 효율적으로 실행할 수 있고, 특별한 연산을 수행할 때, 즉 프로세스가 CPU를 독점하거나, 다른 프로세스로 전환해야 할 때만 개입한다.

이리하여 우리는 CPU 가상화를 위한 기본 개념을 학습하였다. 중요한 질문이 그대로 남았다. 특정 시점에 어떤 프로세스를 실행시켜야 하는가? 바로 스케줄러가 답해야 하는 질문이고 우리가 공부할 다음 주제이다.

## 참고 문헌

- [Alt79]      **“Alto User’s Handbook”**  
*Xerox Palo Alto Research Center, September 1979*  
 URL: <http://history-computer.com/Library/AltoUsersHandbook.pdf>  
 시대를 앞선 환상적인 시스템. Steve Jobs가 방문하고 기록하고 Lisa, 궁극적으로 Mac을 구현하였기 때문에 유명해졌다.
- [Can+04]    **“Microreboot---A Technique for Cheap Recovery”**  
 George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox  
*OSDI '04, San Francisco, CA, December 2004*  
 강인한 시스템을 구축하는 데 리부트를 어디까지 활용할 수 있는지를 지적인 훌륭한 논문.
- [Int11]      **“Intel 64 and IA-32 Architectures Software Developer’s Manual”**  
*Volume 3A and 3B: System Programming Guide, Intel Corporation, January 2011*
- [Kil+62]     **“One-Level Storage System”**  
 T. Kilburn, D.B.G. Edwards, M.J. Lanigan, and F.H. Sumner  
*IRE Transactions on Electronic Computers, April 1962*  
 Atlas는 현대 시스템에서 우리가 볼 수 있는 많은 것들을 개척하였다. 그러나 이 논문이 읽을 만한 가장 좋은 논문은 아니다. 한편의 논문을 읽어야 한다면 아래에 있는 역사적 관점의 논문을 읽는 것이 좋을 것이다 [Lav78].
- [Lav78]      **“The Manchester Mark I and Atlas: A Historical Perspective”**  
 S.H. Lavington  
*Communications of the ACM, 21:1, January 1978*  
 초창기 컴퓨터 개발과 Atlas의 개척적인 노력에 관한 역사.
- [Mac11]      **“Mac OS 9”**  
*January 2011*  
 URL: [http://en.wikipedia.org/wiki/Mac\\_OS\\_9](http://en.wikipedia.org/wiki/Mac_OS_9)
- [McC+63]    **“A Time-Sharing Debugging System for a Small Computer”**  
 J. McCarthy, S. Boilen, E. Fredkin, and J.C.R. Licklider  
*AFIPS '63 (Spring), May, 1963, New York, USA*  
 타이머 인터럽트 사용에 관해 언급한 시분할에 관한 초기 논문. 그 논의를 인용하면 “채널 17 클럭 루틴의 기본 임무는 코어에서 현재 사용자를 제거할지 결정하는 것이고 제거하기로 결정한 경우 대신에 어느 사용자 프로그램이 스왑-인 될지 결정하는 것이다.”
- [MS96]       **“lmbench: Portable tools for performance analysis”**  
 Larry McVoy and Carl Staelin  
*USENIX Annual Technical Conference, January 1996*  
 운영체제와 그의 성능에 관해 여러 측면을 어떻게 측정하느냐에 관한 흥미로운 논문. lmbench를 다운로드하여 한 번 측정해 보라.
- [Ous90]      **“Why Aren’t Operating Systems Getting Faster as Fast as Hardware?”**  
 J. Ousterhout  
*USENIX Summer Conference, June 1990*  
 운영체제 성능의 본질에 관한 고전적인 논문.

[P10]           **“The Single Unix Specification, Version 3”**

*The Open Group, May 2010*

URL: <http://www.unix.org/version3/>

읽기에 어렵고 힘든 문서. 가능하다면 읽지 말도록.

[Sha07]           **“The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)”**

Hovav Shacham

*CCS '07, October 2007*

연구 중에 가끔 만나게 되는 경이로우면서 놀라운 아이디어 중 하나. 저자는 임의의 위치로 분기할 수 있다면 주어진 커다란 코드 베이스에서 원하는 코드 시퀀스를 만들어 낼 수 있다는 것을 보인다. 자세한 사항은 논문을 읽도록. 안타깝게도 이 기법은 악의적인 공격에 대한 방어를 훨씬 어렵게 만든다.

## 숙제 (측정)

### 여담: 측정 숙제

측정 숙제는 운영체제 또는 하드웨어 성능의 특정 측면을 측정하기 위해 실제 컴퓨터에서 실행시킬 코드를 작성하는 작은 연습이다. 이 숙제의 의도에는 운영체제를 직접 경험해 보자는 의미가 있다.

이 숙제에서 시스템 콜과 문맥 교환의 비용을 측정할 것이다. 시스템 콜의 비용을 측정하는 것은 상대적으로 쉽다. 예를 들어, 간단한 시스템 콜을(0 바이트 읽기 등) 반복적으로 호출하여 통틀어 걸린 시간을 측정한다. 이 시간을 반복 횟수로 나누면 시스템 콜의 비용을 추정할 수 있다.

한 가지 고려해야 할 사항은 타이머의 정확도와 정밀도이다. 사용할 수 있는 전형적인 타이머는 `gettimeofday()` 이다. 자세한 사항에 대해서는 `man` 페이지를 참조하도록. `man` 페이지에서 볼 수 있는 정보는 `gettimeofday()` 이 1970년 이후 경과한 시간을 마이크로초 단위로 반환한다는 것이다. 그러나 이 사실이 타이머가 마이크로초 수준의 정밀도를 가진다는 것을 의미하지는 않는다. 타이머가 얼마나 정밀한지를 알고 싶다면 `gettimeofday()` 를 연속해서 호출하여 측정하라. 이 결과로부터 만족스러운 측정 결과를 얻기 위해서는 널 시스템 콜 테스트를 몇 번 반복해야 하는지 알 수 있을 것이다. `gettimeofday()` 가 원하는 만큼 정밀도를 보이지 않는다면 x86 CPU에서 제공되는 `rdtsc` 명령어를 사용하는 것을 검토해 보아야 한다.

문맥 교환의 비용을 측정하는 것은 더 곤란하다. `lmbench` 벤치마크는 두 개의 프로세스를 하나의 CPU에서 실행시키고 둘 사이에 UNIX 파이프를 설정하여 문맥 교환 비용을 측정한다. 파이프는 UNIX 시스템에서 프로세스끼리 통신할 수 있는 여러 방법 중의 하나이다. 첫 번째 프로세스는 첫 번째 파이프에 데이터를 쓰고, 두 번째 파이프로부터 데이터가 전달되기를 기다린다. 첫 번째 프로세스가 두 번째 파이프로부터 읽을 무언가를 기다린다는 사실을 알게 되면 운영체제는 첫 번째 프로세스를 봉쇄 상태로 만들고 다른 프로세스로 전환한다. 이 프로세스는 첫 번째 파이프로부터 데이터를 읽고 두 번째 파이프에 데이터를 쓴다. 두 번째 프로세스가 첫 번째 파이프로부터 다시 데이터를 읽으려고 하면 봉쇄 상태로 들어간다. 이런 식으로 주고 받는 통신 사이클이 계속된다. 이러한 통신 비용을 반복적으로 측정하여 `lmbench`는 문맥 교환의 비용에 대해 훌륭한 추정치를 구할 수 있다. 이 숙제에서 파이프를 사용하거나 UNIX 소켓과 같은 다른 통신 기법을 사용하여 유사한 상황을 만들어 볼 수 있다.

문맥 교환의 비용을 측정하는 데 있어 어려움은 하나 이상의 CPU를 가진 시스템에서 발생한다. 그런 시스템에서 해야만 하는 일은 문맥 교환되는 프로세스들이 같은 프로세서 상에 위치하는 것을 보장하는 것이다. 다행히도 대부분의 운영체제는 프로세스를 특정 프로세서에 묶어두는 시스템 콜을 가진다. 예를 들어, Linux에서는 `sched_setaffinity()` 시스템 콜이 기대하는 일을 한다. 두 프로세스를 같은 프로세

서에 존재하도록 보장하면 같은 프로세서에 존재하는 한 프로세스를 중단시키고 다른 프로세스를 복원하는 운영체제의 비용을 확실하게 측정할 수 있다.

## 스케줄링 : 개요

이제 프로세스를 실행시키기 위한 문맥 교환 등의 저수준 기법에 대해서는 분명하게 이해하고 있어야 한다. 그렇지 않다면 한 두 장 이전으로 돌아가서 이러한 기법들이 어떻게 동작하는지에 대한 설명을 다시 읽기 바란다. 운영체제 스케줄러의 고수준 정책에 관해서는 이해가 필요하다. 이제부터 다양한 **스케줄링 정책(scheduling policy)**을 소개하고 그에 관한 이해를 높일 것이다. 이러한 정책은 **원칙(discipline)**이라고도 불리며 많은 똑똑하고 부지런한 사람들이 오랫동안 개발한 정책들이다.

사실 스케줄링의 기원은 컴퓨터 시스템 개발 이전으로 거슬러 올라간다. 초기의 방법들은 생산 관리 분야에서 개발되었으며 컴퓨터에 적용되었다. 이 사실은 그렇게 놀라운 일은 아니다. 생산 공정과 기타 많은 인간의 행동 역시 스케줄링이 필요하고, 효율성에 대한 요구 등, 공통된 해결 사항이 존재하기 때문이다. 우리의 문제는 다음과 같다.

### 핵심 질문 : 스케줄링 정책은 어떻게 개발하는가

스케줄링 정책을 생각하기 위한 기본적인 프레임워크를 어떻게 만들어야 하는가? 핵심 가정은 무엇인가? 어떤 평가 기준이 중요한가? 컴퓨터 시스템의 초창기에 사용되었던 기본 접근법은 무엇인가?

## 10.1 워크로드에 대한 가정

가장 먼저 프로세스에 대하여 몇 가지 가정을 할 것이다. 일련의 프로세스들이 실행하는 상황을 **워크로드(workload)**라고 부르기로 한다. 워크로드를 결정하는 것은 정책 개발에 매우 중요한 부분이다. 워크로드에 관해 더 잘 알수록 그에 맞게 정책을 정교하게 손질할 수 있다.

우리가 지금 설명에서 사용할 워크로드에 대한 가정은 비현실적이다. 하지만, 당장 논의하는 데 아무 문제 없다. 앞으로 논의가 진행됨에 따라 가정을 완화시킬 것이고 최종적으로 제대로 동작하는 스케줄링 정책을 만들게 될 것이다.

우리는 시스템에서 실행 중인 프로세스 혹은 작업(job)에 대해 다음과 같은 가정을 한다.

1. 모든 작업은 같은 시간 동안 실행된다.
2. 모든 작업은 동시에 도착한다.
3. 각 작업은 시작되면 완료될 때까지 실행된다.
4. 모든 작업은 CPU만 사용한다(즉, 입출력을 수행하지 않는다).
5. 각 작업의 실행 시간은 사전에 알려져 있다.

이러한 가정들이 비현실적이라고 이미 언급하였다. Animal Farm [Orw45]에서 어떤 동물은 다른 동물보다 더 평등하다고 한 것처럼 몇몇 가정은 다른 가정에 비해 더 비현실적이다. 특히, 각 작업의 실행 시간이 미리 알려져 있다는 가정은 더욱 그렇다. 이 가정은 스케줄러가 모든 것을 다 알 수 있다는 것을 의미한다. 좋은 일이지는 않지만 가까운 미래에 나올 것 같지는 않다.

## 10.2 스케줄링 평가 항목

워크로드에 대한 가정 이외에 스케줄링 정책의 비교를 위해 **스케줄링 평가 항목(scheduling metric)**을 결정해야 한다. 스케줄링의 경우 다양한 평가 기준이 존재한다.

우선, 문제를 간단하게 하기 위해 **반환 시간(turnaround time)**이라는 하나의 평가 기준을 사용한다. 작업 반환 시간은 작업이 완료된 시각에서 작업이 시스템에 도착한 시각을 뺀 시간으로 정의된다. 반환 시간  $T_{turnaround}$ 는

$$T_{turnaround} = T_{completion} - T_{arrival} \quad (10.1)$$

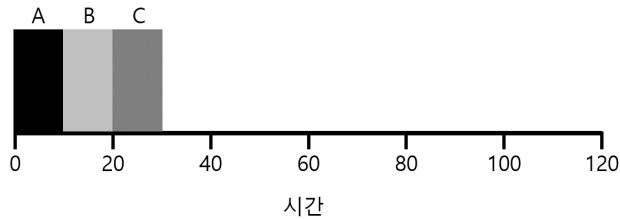
모든 작업은 동시에 도착한다고 가정했으므로  $T_{arrival}=0$ 으로 생각할 수 있다.  $T_{turnaround} = T_{completion}$ 이다. 이 가정은 완화해 나갈 것이다.

반환 시간은 **성능** 측면에서의 평가 기준이라는 것을 주목해야 한다. 다른 평가 기준으로 **공정성(fairness)**이다. 예를 들면 **Jain's Fairness Index** [Jai91]에 따라 측정된다. 성능과 공정성은 스케줄링에서는 서로 상충되는 목표이다. 예를 들어, 스케줄러는 성능을 극대화하기 위해 몇몇 작업의 실행을 중지하며, 결과적으로 공정성이 악화된다. 인생은 대부분 완벽하지 않다.

## 10.3 선입선출

가장 기초적인 알고리즘은 **선입선출(First In First Out, FIFO)** 또는 **선도착선처리(First Come First Served, FCFS)** 스케줄링이라고 알려져 있다. FIFO는 많은 장점을 가진다. 단순하고 구현하기 쉽다. 우리의 가정 하에서는 매우 잘 동작한다.

간단한 예를 스케줄 해 보자. 시스템에 3개의 작업 A, B, C가 거의 동시에 도착했다고 가정하자 ( $T_{arrival} = 0$ ). 간발의 차이로 A, B, C 순서대로 도착했다고 가정하자. 또한, 각 작업은 10초 동안 실행된다고 가정하자. 이 작업들의 평균 반환 시간은 얼마인가?



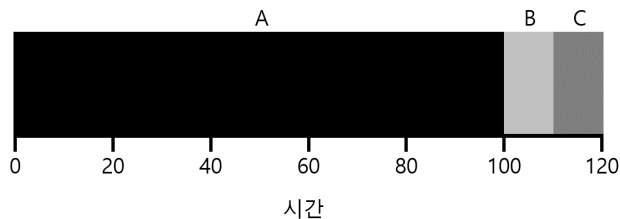
〈그림 10.1〉 FIFO 간단한 예

그림 10.1에서 A는 10, B는 20, C는 30에 종료했다는 것을 알 수 있다. 세 작업의 평균 반환 시간은  $\frac{(10+20+30)}{3} = 20$ 이다. 반환 시간의 계산은 쉽다.

자 이제 가정 중 한 가지를 완화하자. 구체적으로 1번 가정을 완화하여 작업을 실행 시간이 모두 같지 않다고 가정하자. FIFO는 어떻게 동작하는가? FIFO는 어떤 워크로드에서 좋지 않은 성능을 보일까?

(다음으로 넘어가기 전에 생각해 보자... 생각에 생각... 알았지?!)

FIFO 스케줄링이 어떤 문제를 야기하는지 같이 생각해 보자. 구체적으로 다시 A, B, C 세 개의 작업을 가정하자. 그러나 이번에는 A는 100초, B와 C는 10초 동안 실행된다.



〈그림 10.2〉 FIFO가 그렇게 좋은 스케줄링이 아닌 이유

그림 10.2에서 볼 수 있듯이, 작업 A가 B와 C 보다 먼저 100초 동안 실행된다. 따라서 시스템의 평균 반환 시간은 110초로 늘어난다.  $\frac{(100+110+120)}{3} = 110$ .

이 문제점은 **convoy effect** [Bla+79]라고 칭해지며 짧은 시간 동안 자원을 사용할 프로세스들이 자원을 오랫동안 사용하는 프로세스의 종료를 기다리는 현상을 말한다. 이 스케줄링 시나리오는 슈퍼마켓에서 줄 서서 계산을 기다릴 때, 앞사람이 카트 세 개에 물건을 가득 싣고 있는 경우와 느낌이 비슷하다. 한참 걸리겠지?<sup>1</sup>

그러면 어떻게 해야 할까? 작업 실행 시간이 다른 경우 더 좋은 알고리즘은 무엇인가? 먼저 생각해 보아라. 그런 후 다음 내용을 읽기 바란다.

1) 이 경우 빨리 다른 줄에 가서 서거나 길고 깊게 심호흡을 하라고 권하고 싶다. 오케이, 들이쉬고 내쉬고.

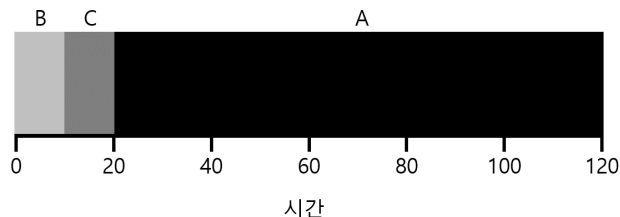
**팁: SJF의 원칙**

최단 작업 우선 원칙은 평균 반환 시간이 (혹은 우리의 경우, 작업당) 강조되는 모든 시스템에 적용될 수 있는 일반적인 스케줄링 원리이다. 당신이 기다린 경험이 있는 모든 줄을 생각해 보라. 문제의 장소가 고객 만족을 중요시한다면 SJF의 사용을 고려했을 가능성이 높다. 예를 들어, 식료품 가게들은 적은 개수의 물건을 구매한 사람들이 다가올 겨울을 준비하는 가족 뒤에 서서 마냥 기다리지 않도록 “소량 구매 계산대”를 갖추고 있다.

**10.4 최단 작업 우선**

이 문제는 간단히 해결할 수 있다. 사실 이 아이디어는 오퍼레이션 리서치 분야에서 차용하여 [Cob54; JV56] 컴퓨터 시스템의 작업 스케줄링에 적용한 것이다. 이 새로운 스케줄링 개념용 **최단 작업 우선(Shortest Job First, SJF)**으로 이름 자체가 정책을 설명하고 있기 때문에 기억하기 쉽다. 이 원칙은 가장 짧은 실행 시간을 가진 작업을 먼저 실행시킨다.

위의 예를 이번에는 SJF로 스케줄 해 보자. 그림 10.3이 A, B, C를 실행시킨 결과를 보이고 있다. SJF가 평균 반환 시간 기준으로 더 나은 성능을 보이는지 보인다. B, C, A 순서로 실행시킴으로써 SJF는 평균 반환 시간을 110초에서 50초 ( $\frac{(10+20+120)}{3} = 50$ )로 2배 이상 향상시켰다.



〈그림 10.3〉 SJF 간단한 예

모든 작업이 동시에 도착한다면 SJF가 **최적(optimal)**의 스케줄링 알고리즘임을 증명할 수 있다. 지금은 이론이나 오퍼레이션 리서치 수업이 아니라 시스템 수업을 듣고 있기 때문에 이에 대한 증명은 하지 않을 것이다.

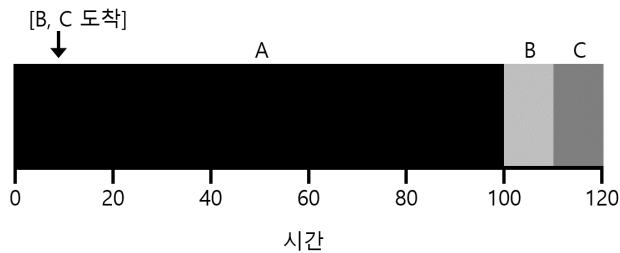
SJF라는 좋은 스케줄링 기법을 개발하였지만 가정이 아직 비현실적이다. 이제 가정을 완화해 보자. 특히, 가정 2를 완화하여 이제 모든 작업은 동시에 도착하는 것이 아니라 임의의 시간에 도착할 수 있다고 가정한다. 이러면 어떤 문제가 발생하는가?

(또 생각할 시간... 생각하고 있는 거지?... 자, 할 수 있어!)

자 문제가 무엇인지 설명하기 위하여 예를 들어 보자. 이번에는 A는  $t = 0$ 에 도착하고 100초의 실행 시간을 가지고 있고, 반면에 B와 C는  $t = 10$ 에 도착하고 각각 10초의 실행

별 문제 아니다, 걱정하지말자.

시간을 가진다고 가정하자. 순수한 SJF로 스케줄 했을 경우의 스케줄이 그림 10.4에 나와 있다.



〈그림 10.4〉 B와 C가 늦게 도착한 경우의 SJF 스케줄

그림에서 알 수 있듯이 B와 C가 A 바로 뒤에 도착한다고 하더라도 A가 끝날 때까지 기다릴 수밖에 없어서 이전의 convoy 문제가 다시 발생한다. 이 경우 평균 반환 시간은  $103.33\left(\frac{100+(110-10)+(120-10)}{3}\right)$ 이다. 스케줄러는 어떻게 스케줄링 해야 할까?

#### 여담: 선점형 스케줄러

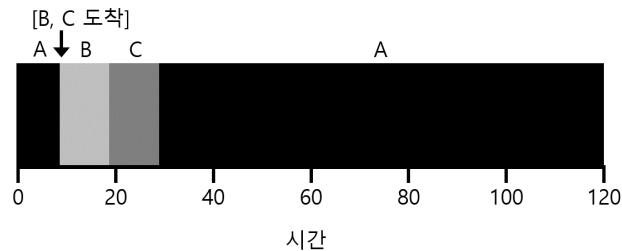
예전 일괄처리 시절에 많은 비선점(non-preemptive) 스케줄러가 개발되었다. 이 시스템은 각 작업이 종료될 때까지 계속 실행한다. 이론적으로 모든 현대 스케줄러는 선점형이고, 다른 프로세스를 실행시키기 위하여 필요하면 현재 프로세스의 실행을 중단한다. 이 사실은 스케줄러가 앞에서 배웠던 기법을 채용하고 있다는 것을 의미한다. 구체적으로 스케줄러는 문맥 교환을 수행할 수 있고, 실행 중인 프로세스를 잠시 중단시킬 수 있으며 다른 프로세스를 재개 또는 시작시킬 수 있다.

## 10.5 최소 잔여시간 우선

이 문제를 해결하기 위해서는 가정 3 (작업은 끝날 때까지 계속 실행된다)을 완화해야 한다. 이제 그 작업을 해 보자. 타이머 인터럽트와 문맥 교환을 고려하면 B와 C가 도착했을 때 스케줄러는 다른 일을 할 수 있다. 작업 A를 중지하고 지금 도착한 B나 C를 실행하기로 결정할 수도 있다. 물론, 선점된 A는 나중에 다시 실행된다. SJF는 비선점형 스케줄러이기 때문에 이와 같은 동작을 하지 못한다.

다행히도 이렇게 동작할 수 있는 스케줄러는 이미 존재한다. SJF에 선점 기능을 추가한 최단 잔여시간 우선(Shortest Time-to-Completion First, STCF) 또는 선점형 최단 작업 우선 (PSJF)으로 알려진 스케줄러이다 [CK68]. 언제든지 새로운 작업이 시스템에 들어오면, 이 스케줄러는 남아 있는 작업과 새로운 작업의 잔여 실행 시간을 계산하고 그 중 가장 적은 잔여 실행 시간을 가진 작업을 스케줄한다. 우리의 예에서는

STCF는 A를 선점하고 B와 C를 끝날 때까지 실행시킨다. 두 작업이 모두 끝난 후에 A가 스케줄되어 남은 실행 시간 만큼 실행된다. 그림 10.5의 스케줄 예를 보라.



〈그림 10.5〉 STCF의 간단한 예

그 결과 평균 반환 시간이 단축되어 50초 ( $\frac{((120-0)+(20-10)+(30-10))}{3}$ )가 된다. 먼저와 같이, 새로운 가정 하에서 STCF가 최적의 스케줄러이다. 작업들이 동시에 도착할 경우, SFJ가 최적의 결과를 낸다는 것을 고려하면, STCF가 최적의 스케줄링이 되는 이유를 쉽게 알 수 있을것이다.

## 10.6 새로운 평가 기준: 응답 시간

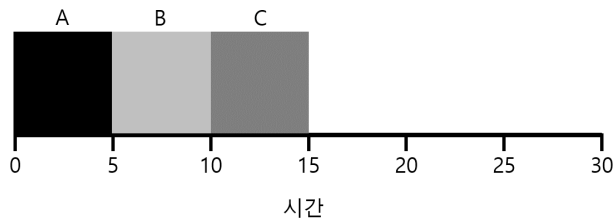
작업의 길이를 미리 알고 있고, 작업이 오직 CPU만 사용하며, 평가 기준이 반환 시간 하나라면, STCF는 매우 훌륭한 정책이다. 초기 일괄처리 컴퓨터 시스템에서는 이러한 스케줄링 알고리즘이 의미가 있었다. 그러나 시분할 컴퓨터의 등장이 모든 것을 바꾸었다. 이제 사용자는 터미널에서 작업하게 되어 시스템에게 상호작용을 원활히 하기 위한 성능을 요구하게 되었다. 응답 시간(response time)이라는 새로운 평가 기준이 태어나게 된다.

응답 시간은 작업이 도착할 때부터 처음 스케줄 될 때까지의 시간으로 정의된다. 다음과 같다.

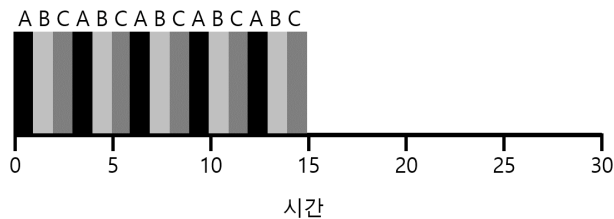
$$T_{response} = T_{firstrun} - T_{arrival} \quad (10.2)$$

예를 들어, 위와 같은 스케줄의 (A는 시간 0, B와 C는 시간 10에 도착) 경우, 각 작업의 응답 시간은 A는 0, B는 0, C는 10, 평균 3.33이 된다.

이미 생각하고 있는지 모르지만, STCF를 비롯하여 비슷한 류의 정책들은 응답 시간이 짧다고 할 수 없다. 예를 들어, 3개의 작업이 동시에 도착한 경우, 세 번째 작업은 딱 한 번 스케줄 되기 위해 먼저 실행된 두 작업이 완전히 끝날 때까지 기다린다. 반환 시간 기준으로는 매우 훌륭한 반면, 응답 시간과 상호작용 측면에서는 매우 나쁜 방법이다. 터미널 앞에 앉아 입력한 후, 단지 다른 작업이 당신의 작업보다 먼저 스케줄 되었다는 이유로 시스템의 응답이 올 때까지 10초 기다리는 것을 상상해 보자. 별로 즐거운 일은 아니다.



〈그림 10.6〉 다시 SJF 스케줄링 (응답 시간은 좋지 않음)



〈그림 10.7〉 라운드 로빈 (응답 시간이 좋음)

우리는 다른 문제를 해결해야 한다. 응답 시간에 민감한 스케줄러를 어떻게 만들 수 있는가?

## 10.7 라운드 로빈

응답 시간 문제를 해결하기 위하여 라운드 로빈(Round-Robin, RR) 스케줄링 [Kle64]이라 불리는 스케줄링 알고리즘을 도입한다. 기본 발상은 간단하다. RR은 작업이 끝날 때까지 기다리지 않는다. 대신 일정 시간 동안 실행한 후 실행 큐의 다음 작업으로 전환한다. 이때 작업이 실행되는 일정 시간을 **타임 슬라이스(time slice)** 또는 **스케줄링 쿼텀(scheduling quantum)**이라 부른다. 작업이 완료될 때까지 이런 식으로 계속 진행된다. 이러한 이유로 RR은 때때로 **타임 슬라이싱**이라고 불린다. 타임 슬라이스의 길이는 타이머 인터럽트 주기의 배수여야 한다. 타이머가 10 msec 마다 인터럽트를 발생시킨다면 타임 슬라이스는 10, 20 등 10 msec의 배수가 될 수 있다.

RR을 더 자세하게 이해하기 위해 하나의 예를 살펴보자. 3개의 작업 A, B, C가 시스템에 동시에 도착하고, 각각 5초간 실행된다고 가정한다. SJF 스케줄러는 다른 작업을 실행하기 전에 각 작업을 종료할 때까지 실행한다(그림 10.6). 대조적으로, 1초의 타임 슬라이스를 가지는 RR은 작업을 빠르게 순환하게 된다(그림 10.7). RR의 평균 응답 시간은  $\frac{(0+1+2)}{3} = 1$  이고, SJF의 평균 응답 시간은  $\frac{(0+5+10)}{3} = 5$ 이다.

쉽게 알 수 있듯이, 타임 슬라이스의 길이는 RR에게 매우 중요하다. 타임 슬라이스가 짧을수록, 응답 시간 기준으로 RR의 성능은 더 좋아진다. 그러나 타임 슬라이스를 너무 짧게 지정하면 문제가 생긴다. 짧게 지정하면 문맥 교환 비용이 전체 성능에 큰 영향을 미치게 된다. 시스템 설계자는 시스템이 최적의 상태로 동작할 수 있도록 타임 슬라이스의

### 팁: 비용의 상쇄

일반적인 **상쇄(amortization)** 기술은 어떤 연산에 고정 비용이 존재하는 시스템에서 흔히 사용된다. 그 비용이 적은 횟수로 발생하게 하여, 즉 해당 연산을 적게 실행함으로써 시스템의 전체 비용이 감소된다. 예를 들어, 타임 슬라이스는 10 msec로 설정되고 문맥 교환 비용이 1 msec라면 대강 10%의 시간이 문맥 교환에 사용되고 따라서 낭비된다. 이 비용을 상쇄하고 싶은 경우, 타임 슬라이스를 예로 들어 100 msec로 늘릴 수 있다. 이러면 문맥 교환에 1% 미만의 시간이 소모되고 타임 슬라이싱의 비용이 상쇄된다.

길이를 결정해야 한다. 문맥 교환 비용을 **상쇄**할 수 있을 만큼 길어야 하지만 그렇다고 응답 시간이 너무 길어지면 안 된다.

문맥 교환 비용에는 레지스터를 저장/복원하는 작업만 있는 것이 아니다. CPU 캐시, TLB, 분기 예측 등을 비롯하여 다른 하드웨어에도 프로그램과 관련된 다양한 작업 정보들이 저장되어 있다. 작업이 전환되면 이 정보들은 모두 갱신되어야 한다. 갱신 작업이 매우 큰 성능 비용을 유발한다 [MB91].

적당한 길이의 응답 시간이 유일한 평가 기준인 경우 타임 슬라이스를 가진 RR은 매우 훌륭한 스케줄러이다. 반환 시간 기준으로서는 어떤가? 앞의 예를 다시 한 번 살펴보자. A, B, C 각 작업은 5초간 실행되고, 동시에 도착하며, RR은 1초의 타임 슬라이스를 가진다. A는 13, B는 14, C는 15에 종료하고 평균 14의 반환 시간을 보인다는 것을 알 수 있다. 매우 안 좋다!

반환 시간이 유일한 측정 기준인 경우 RR이 확실히 최악의 정책이라는 것은 놀라운 일이 아니다. 직관적으로, 충분히 이해 가능하다. RR은 각 작업을 잠깐 실행하고 다음 작업으로 넘어가고 하면서 가능한 한 각 작업을 늘리는 것이 목표다. 반환 시간은 작업 완료 시간만을 고려하기 때문에, RR은 거의 최악이며, 많은 경우 단순한 FIFO보다도 성능이 좋지 않게 된다.

일반적으로 RR과 같은 공정한 정책, 즉 작은 시간 단위로 모든 프로세스에게 CPU를 분배하는 정책은 반환 시간과 같은 평가 기준에서는 성능이 나쁘다. 이건 당연하다. 불공정하게 한다면 하나의 작업을 끝까지 실행하고 종료할 수 있지만 나머지 작업들에 대한 응답 시간은 포기해야 한다. 반대로 공정성을 더 중히 여긴다면 응답 시간은 줄어 들지만 반환 시간은 나빠지게 된다. 이러한 유형의 절충은 시스템에서는 흔히 있는 일이다. 당신은 케익을 먹으면서 동시에 케익을 보관할 수는 없다<sup>2</sup>.

우리는 2종류의 스케줄러를 개발하였다. 첫째 유형은 (SJF, STCF) 반환 시간을 최적화하지만 응답 시간은 나쁘다. 두 번째 유형은 (RR) 응답 시간을 최적화하지만 반환 시간이 나쁘다. 아직 완화해야 하는 가정 두 개가 남아 있다. 가정 4 (작업은 입출력을 하지 않는다)와 가정 5 (각 작업의 실행 시간은 알려져 있다)이다. 이제 이 가정들을 정복해 보자.

2) 사람을 혼란시키는 속담. 원래는 “당신은 케익을 먹으면서 보존할 수 없다”여야 하기 때문이다. (분명하지 않은가, 아닌가?). 놀랍게도 이 속담에 관한 Wikipedia 페이지가 있다. 더욱 놀라운 것은 재미있는 읽을거리라는 것이다. [http://en.wikipedia.org/wiki/You\\_can't\\_have\\_your\\_cake\\_and\\_eat\\_it](http://en.wikipedia.org/wiki/You_can't_have_your_cake_and_eat_it)

**팁: 중첩은 높은 이용률을 가능하게 한다**

가능하면 시스템의 활용도를 극대화하기 위해 연산을 중첩되게 실행한다. 중첩은 디스크 입출력을 할 때나 원격 컴퓨터에 메시지를 보낼 때 등 다양한 영역에서 유용하다. 어느 경우라도 이 작업들을 시작한 후에 다른 작업으로 전환하는 것은 좋은 생각이다. 시스템의 전반적인 사용률과 효율성이 향상된다.

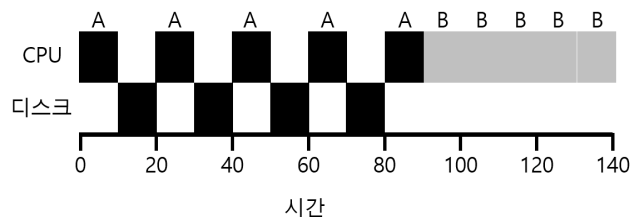
**10.8 입출력 연산의 고려**

우선 가정 4를 완화하자. 모든 프로그램은 입출력 작업을 수행한다. 아무런 입력도 받아들이지 않는 프로그램을 상상해 보자. 매번 같은 출력을 만들어 낼 것이다. 출력하지 않는 프로그램을 상상해 보자. 그건 속담에도 나올 법한 아무도 보는 사람이 없이 숲에서 잎이 떨어지는 것과 같다. 실행되었다는 사실이 아무 의미가 없다.

입출력 작업을 요청한 경우 스케줄러는 다음에 어떤 작업을 실행할지를 결정해야 한다. 현재 실행 중인 작업은 입출력이 완료될 때까지 CPU를 사용하지 않을 것이기 때문이다. 입출력 요청을 발생시킨 작업은 입출력 완료를 기다리며 대기 상태가 된다. 입출력이 하드 디스크 드라이브에 보내진 경우, 프로세스는 드라이브의 현재 입출력 워크로드에 따라 몇 초 또는 좀 더 긴 시간 동안 블록될 것이다. 스케줄러는 그 시간 동안 실행될 다른 작업을 스케줄 해야 한다.

마찬가지로 스케줄러는 입출력 완료 시에도 의사 결정을 해야 한다. 입출력이 완료되면 인터럽트가 발생하고 운영체제가 실행되어 입출력을 요청한 프로세스를 대기 상태에서 준비 상태로 다시 이동시킨다. 물론, 인터럽트가 발생했을 때 요청 프로세스를 즉시 실행시키기로 결정할 수도 있다. 운영체제는 각 작업을 어떻게 처리해야 하는가?

이 쟁점을 더 잘 이해하기 위해서 두 개의 작업 A와 B가 있다고 하자. 각 작업은 50 msec의 CPU 시간을 필요로 한다. 그러나 둘은 한 가지 큰 차이를 가진다. A는 10 msec 동안 실행된 후, 입출력 요청을 한다(여기서 입출력 시간은 10 msec라고 가정한다). 반면에 B는 입출력을 수행하지 않는다. 스케줄러는 A를 먼저 실행시키고 B를 다음에 실행시킨다(그림 10.8).

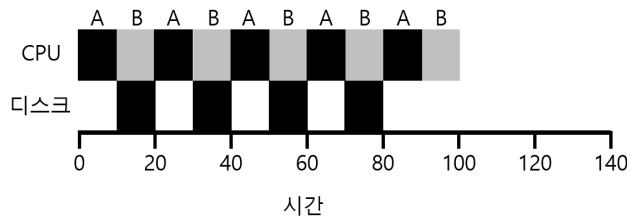


〈그림 10.8〉 자원의 비효율적인 활용

STCF 스케줄러를 구축하려고 한다고 가정하자. A가 5개의 10 msec 작업으로 분할되는 반면에 B는 하나의 50 msec의 CPU를 요청한다는 사실을 어떻게 활용할까?

분명히 입출력을 고려하지 않고 작업을 하나씩 실행시키는 것은 의미가 없다.

일반적인 접근 방식은 A의 각 10 msec 하위 작업을 독립적인 작업으로 다루는 것이다. 시스템이 시작할 때 10 msec 작업들과 50 msec를 스케줄하는 것이다. STCF의 경우 선택은 자명하다. 가장 짧은 작업을 선택하라. 이 경우에는 A가 된다. 그러면 A의 첫 번째 소 작업이 완료되면 B만 남게 되어 실행을 시작한다. A의 다음 작업이 제출되고 B를 선점하여 10 msec 동안 실행한다. 이렇게 하면 프로세스의 입출력이 끝나기를 기다리는 동안 CPU는 다른 프로세스에 의해 사용되어 연산의 중첩이 가능해진다. 시스템은 좀 더 잘 활용된다 (그림 10.9).



〈그림 10.9〉 중첩을 통해 효율적 자원 활용이 가능하다

입출력을 고려한 스케줄러 방법에 대해서 알아보았다. 각 CPU 버스트를 하나의 작업으로 간주함으로써 스케줄러는 대화형 프로세스가 더 자주, 즉 유리하게 실행되는 것을 보장한다. 이러한 대화형 작업이 입출력을 실행하는 동안 다른 CPU-집중 작업들이 실행되고 CPU의 이용률은 더 높아진다.

## 10.9 만병통치약은 없다 (No More Oracle)

입출력을 고려한 기본적인 접근 방식에 대해 논의하였으므로 마지막 가정에 대해 생각해 보자. 스케줄러가 각 작업의 실행 시간을 알고 있다는 가정이다. 전에 얘기했듯이, 이 가정은 아마 우리가 할 수 있는 최악의 가정이다. 사실 범용 운영체제에서(우리가 현재 고려 중인 것과 같은) 작업의 길이에 대해서 알 수 있는 길은 없다. 따라서 아무런 사전 지식 없이 SJF/STCF 처럼 행동하는 알고리즘을 구축할 수 있을까? 게다가, 응답 시간도 좋게 하기 위하여 RR 스케줄러의 경우에 보았던 아이디어를 어떻게 하면 포함시킬 수 있을까?

## 10.10 요약

우리는 스케줄링의 기본적인 개념과 두 가지 부류의 접근법을 살펴보았다. 첫 번째 부류는 남아 있는 작업 중 실행 시간이 제일 짧은 작업을 수행하고, 반환 시간을 최소화한다. 두 번째 부류는 모든 작업을 번갈아 실행시키고 응답 시간을 최소화한다. 안타깝게도 반환 시간과 응답 시간 중 하나를 최적화하면 나머지 하나는 좋지 않은 특성을 나타낸다. 이는 시스템에서 흔히 보이는 절충을 요구하는 상황이다. 전체적인 그림에 입출력을

어떻게 통합해야 하는지도 보았다. 그렇지만 미래를 예측할 수 없는 운영체제의 근본적인 문제는 해결할 수 없었다. 가까운 과거를 이용하여 미래를 예측하는 스케줄러를 구현하여 이 문제를 어떻게 해결하는지 곧 보게 될 것이다. 이 스케줄러는 **멀티 레벨 피드백 큐(multi-level feedback queue)**라고 불리며 다음 장의 주제이다.

## 참고 문헌

[Bla+79] “The Convoy Phenomenon”

M. Blasgen, J. Gray, M. Mitoma, and T. Price

*ACM Operating Systems Review*, 13:2, April 1979

아마도 운영체제뿐 아니라 데이터베이스에서 발생한 호위함 효과에 대한 첫 번째 참고 문헌.

[Cob54] “Priority Assignment in Waiting Line Problems”

A. Cobham

*Journal of Operations Research*, 2:70, pages 70-76, 1954

기계 수리 일정을 정하는 데 SJF 방식을 사용한 선구적인 논문.

[CK68] “Computer Scheduling Methods and their Countermeasures”

Edward G. Coffman and Leonard Kleinrock

*AFIPS '68 (Spring)*, April 1968

다양한 기본 스케줄링 알고리즘에 대한 우수한 초기 소개 및 분석.

[Jai91] “The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling”

R. Jain

*Interscience*, New York, April 1991

컴퓨터 시스템 측정에 관한 표준 교과서. 당신의 라이브러리를 빛낼 훌륭한 참고 문헌.

[JV56] “Machine Repair as a Priority Waiting-Line Problem”

Thomas E. Phipps Jr. and W.R. Van Voorhis

*Operations Research*, 4:1, pages 76-86, February 1956

기계 수리 스케줄링에 사용된 Cobham의 SJF를 일반화한 후속 연구. 또한 이러한 환경에서 STCF 방식의 유용성을 생각한 논문. 구체적으로 “수리 작업에는 특정 유형이 존재한다 ... 많은 해체와 마루를 너트와 볼트로 뒤덮음을 동반하기 때문에 한 번 시작하면 중단되지는 안 된다. 다른 경우, 하나 이상의 짧은 작업들이 존재할 때 긴 작업을 계속하는 것은 권하고 싶지 않을 것이다 (p.81)”

[Kle64] “Analysis of a Time-Shared Processor”

Leonard Kleinrock

*Naval Research Logistics Quarterly*, 11:1, pages 59-73, March 1964

라운드 로빈 스케줄링 알고리즘에 대한 첫 번째 참고 문헌. 시분할 시스템을 스케줄링 할 때 라운드 로빈 스케줄링 알고리즘 사용에 대한 분석을 제시한 논문 중 하나.

[Lav78] “The Manchester Mark I and Atlas: A Historical Perspective”

S.H. Lavington

*Communications of the ACM*, 21:1, January 1978

초창기 컴퓨터 개발과 Atlas의 개척적인 노력에 관한 역사.

[MB91] “The effect of context switches on cache performance”

Jeffrey C. Mogul and Anita Borg

*ASPLOS*, 1991

문맥 교환이 캐시의 성능에 미치는 영향에 관한 좋은 연구. 프로세서는 초당 수십억 명령어를 실행하지만 문맥 교환은 msec 시간 단위로 발생하기 때문에 현재 시스템에서는 크게 문제가 되지 않는다.

[Orw45] “Animal Farm”

George Orwell

*Secker and Warburg (London), 1945*

권력과 그의 변질에 관한 슬프지만 우울한 우화책. 어떤 이는 제2차 세계대전 전 스탈린 시대의 소련에 대한 비판이라고 말하기도 한다. 우리는 그저 돼지에 대한 비판이라고 말한다.

## 숙제

`scheduler.py` 프로그램은 응답 시간, 반환 시간, 총 대기 시간의 관점에서 여러 스케줄러가 어떻게 동작하는지를 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

## 문제

1. 길이가 200인 세 개의 작업을 SJF와 FIFO 스케줄링 방식으로 실행할 경우 응답 시간과 반환 시간을 계산하시오.
2. 같은 조건이지만 작업의 길이가 각각 100, 200 및 300일 경우에 대해 계산하시오.
3. 2번과 같은 조건으로 타임 슬라이스가 1인 RR 스케줄러에 대해서도 계산하시오.
4. SJF와 FIFO가 같은 반환 시간을 보이는 워크로드의 유형은 무엇인가?
5. SJF가 RR과 같은 응답 시간을 보이기 위한 워크로드와 타임 퀀텀의 길이는 무엇인가?
6. 작업의 길이가 증가하면 SJF의 응답 시간은 어떻게 되는가? 변화의 추이를 보이기 위해서 시뮬레이터를 사용할 수 있는가?
7. 타임 퀀텀의 길이가 증가하면 RR의 응답 시간은 어떻게 되는가?  $N$  개의 작업이 주어졌을 때, 최악의 응답 시간을 계산하는 식을 만들 수 있는가?

## Scheduling: Proportional Share

In this chapter, we'll examine a different type of scheduler known as a **proportional-share** scheduler, also sometimes referred to as a **fair-share** scheduler. Proportional-share is based around a simple concept: instead of optimizing for turnaround or response time, a scheduler might instead try to guarantee that each job obtain a certain percentage of CPU time.

An excellent early example of proportional-share scheduling is found in research by Waldspurger and Weihl [WW94], and is known as **lottery scheduling**; however, the idea is certainly older [KL88]. The basic idea is quite simple: every so often, hold a lottery to determine which process should get to run next; processes that should run more often should be given more chances to win the lottery. Easy, no? Now, onto the details! But not before our crux:

### CRUX: HOW TO SHARE THE CPU PROPORTIONALLY

How can we design a scheduler to share the CPU proportionally? What are the key mechanisms for doing so? How effective are they?

### 9.1 Basic Concept: Tickets Represent Your Share

Underlying lottery scheduling is one very basic concept: **tickets**, which are used to represent the share of a resource that a process (or user or whatever) should receive. The percent of tickets that a process has represents its share of the system resource in question.

Let's look at an example. Imagine two processes, A and B, and further that A has 75 tickets while B has only 25. Thus, what we would like is for A to receive 75% of the CPU and B the remaining 25%.

Lottery scheduling achieves this probabilistically (but not deterministically) by holding a lottery every so often (say, every time slice). Holding a lottery is straightforward: the scheduler must know how many total tickets there are (in our example, there are 100). The scheduler then picks

## 9. 스케줄링: 비례 배분

비례 배분 스케줄링 / 영문 원문 1쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이 장에서는 비례 배분 스케줄러(proportional-share scheduler)라는 또 다른 종류의 스케줄러를 살펴본다. 공정 배분 스케줄러(fair-share scheduler)라고도 한다. 비례 배분의 바탕이 되는 개념은 간단하다. 스케줄러는 반환 시간이나 응답 시간을 최적화하는 대신, 각 작업이 CPU 시간의 일정 비율을 얻도록 보장하려고 할 수 있다.

비례 배분 스케줄링의 훌륭한 초기 사례는 Waldspurger와 Weihl의 연구 [WW94]에서 볼 수 있으며, 추첨 스케줄링(lottery scheduling)이라고 알려져 있다. 다만 이 발상 자체는 그보다 오래되었다 [KL88]. 기본 아이디어는 아주 단순하다. 일정한 간격으로 추첨하여 다음에 실행할 프로세스를 정한다. 더 자주 실행되어야 하는 프로세스에는 추첨에서 당첨될 기회를 더 많이 준다. 간단하지 않은가? 이제 자세히 살펴보자! 그 전에 핵심 문제부터 제시하겠다.

### 핵심 문제: CPU를 비례해서 나누어 쓰려면?

CPU를 비례해서 나누어 쓰도록 스케줄러를 어떻게 설계할 수 있을까? 이를 위한 핵심 메커니즘은 무엇인가? 그 메커니즘은 얼마나 효과적인가?

#### 9.1 기본 개념: 티켓은 자신의 몫을 나타낸다

추첨 스케줄링의 바탕에는 아주 기본적인 개념 하나가 있다. 바로 티켓(ticket)이다. 티켓은 프로세스, 사용자 또는 다른 어떤 주체가 받아야 할 자원의 몫을 나타내는 데 사용한다. 어떤 프로세스가 가진 티켓의 비율은 해당 시스템 자원에서 그 프로세스가 차지하는 몫을 나타낸다.

예를 들어 보자. A와 B라는 두 프로세스가 있고, A는 티켓을 75장, B는 25장만 갖고 있다고 하자. 그러면 우리가 바라는 것은 A가 CPU의 75%를, B가 나머지 25%를 받는 것이다.

추첨 스케줄링은 일정한 간격, 예를 들어 매 타임 슬라이스마다 추첨하여 이를 확률적으로 달성한다. 결정론적으로 달성하는 것은 아니다. 추첨 방법은 간단하다. 스케줄러는 티켓의 총수를 알아야 한다. 이 예에서는 총 100장이다. 그다음 스케줄러는 당첨 티켓을 뽑는다. (다음 쪽에 계속)

## TIP: USE RANDOMNESS

One of the most beautiful aspects of lottery scheduling is its use of **randomness**. When you have to make a decision, using such a randomized approach is often a robust and simple way of doing so.

Random approaches have at least three advantages over more traditional decisions. First, random often avoids strange corner-case behaviors that a more traditional algorithm may have trouble handling. For example, consider the LRU replacement policy (studied in more detail in a future chapter on virtual memory); while often a good replacement algorithm, LRU attains worst-case performance for some cyclic-sequential workloads. Random, on the other hand, has no such worst case.

Second, random also is lightweight, requiring little state to track alternatives. In a traditional fair-share scheduling algorithm, tracking how much CPU each process has received requires per-process accounting, which must be updated after running each process. Doing so randomly necessitates only the most minimal of per-process state (e.g., the number of tickets each has).

Finally, random can be quite fast. As long as generating a random number is quick, making the decision is also, and thus random can be used in a number of places where speed is required. Of course, the faster the need, the more random tends towards pseudo-random.

a winning ticket, which is a number from 0 to 99<sup>1</sup>. Assuming A holds tickets 0 through 74 and B 75 through 99, the winning ticket simply determines whether A or B runs. The scheduler then loads the state of that winning process and runs it.

Here is an example output of a lottery scheduler's winning tickets:

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 12

Here is the resulting schedule:

A        A    A        A    A    A    A    A    A        A        A    A    A    A    A    A  
B                    B                    B                    B

As you can see from the example, the use of randomness in lottery scheduling leads to a probabilistic correctness in meeting the desired proportion, but no guarantee. In our example above, B only gets to run 4 out of 20 time slices (20%), instead of the desired 25% allocation. However, the longer these two jobs compete, the more likely they are to achieve the desired percentages.

<sup>1</sup>Computer Scientists always start counting at 0. It is so odd to non-computer-types that famous people have felt obliged to write about why we do it this way [D82].

## 무작위성과 추첨 스케줄링

비례 배분 스케줄링 / 영문 원문 2쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

### 팁: 무작위성을 활용하라

추첨 스케줄링의 가장 멋진 측면 중 하나는 무작위성을 사용한다는 점이다. 결정을 내려야 할 때 이러한 무작위 접근법은 흔히 견고하고 간단한 방법이 된다.

무작위 접근법은 전통적인 의사 결정 방식보다 적어도 세 가지 장점이 있다. 첫째, 무작위 방식은 전통적인 알고리즘이 처리하기 어려워하는 특이한 예외 상황을 피하는 경우가 많다. 예를 들어 LRU 교체 정책을 생각해 보자. 가상 메모리를 다루는 이후 장에서 더 자세히 배울 이 정책은 대체로 좋은 교체 알고리즘이지만, 일부 순환적 순차 워크로드에서는 최악의 성능을 보인다. 반면 무작위 방식에는 이러한 최악의 경우가 없다.

둘째, 무작위 방식은 가볍다. 대안들을 추적하는 데 필요한 상태 정보가 적기 때문이다. 전통적인 공정 배분 스케줄링 알고리즘에서는 각 프로세스가 CPU를 얼마나 받았는지 추적하려면 프로세스별 사용량 기록이 필요하고, 프로세스를 실행할 때마다 이를 갱신해야 한다. 무작위로 결정하면 각 프로세스가 가진 티켓 수처럼 최소한의 프로세스별 상태만 있으면 된다.

마지막으로 무작위 방식은 상당히 빠를 수 있다. 난수 생성이 빠르기만 하면 결정도 빠르므로 속도가 필요한 여러 곳에서 사용할 수 있다. 물론 더 빠른 속도를 요구할수록 무작위성은 의사 무작위성에 가까워지는 경향이 있다.

(앞쪽에서 계속) 당첨 티켓은 0부터 99 사이의 수이다[각주 1]. A가 0번부터 74번까지의 티켓을, B가 75번부터 99번까지의 티켓을 갖고 있다고 가정하면 당첨 티켓에 따라 A와 B 중 누가 실행할지가 결정된다. 그러면 스케줄러는 당첨된 프로세스의 상태를 불러와 실행한다.

추첨 스케줄러가 뽑은 당첨 티켓의 예는 다음과 같다.

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 12
```

이에 따른 실행 순서는 다음과 같다.

```
A B A A B A A A A A A B A B A A A A A
```

이 예에서 보듯 추첨 스케줄링은 무작위성을 사용하므로 원하는 비율을 확률적으로 충족하지만 보장하지는 않는다. 위 예에서 B는 원하는 25% 대신 20개의 타임 슬라이스 중 4개, 즉 20%에서만 실행된다. 그러나 두 작업이 경쟁하는 시간이 길어질수록 원하는 비율을 달성할 가능성은 더 커진다.

각주 1. 컴퓨터 과학자는 언제나 0부터 수를 센다. 컴퓨터 분야 밖의 사람들에게는 이것이 너무 이상해 보여서, 유명한 사람들조차 우리가 왜 이렇게 하는지 글을 써야겠다고 느꼈다 [D82].

## TIP: USE TICKETS TO REPRESENT SHARES

One of the most powerful (and basic) mechanisms in the design of lottery (and stride) scheduling is that of the **ticket**. The ticket is used to represent a process's share of the CPU in these examples, but can be applied much more broadly. For example, in more recent work on virtual memory management for hypervisors, Waldspurger shows how tickets can be used to represent a guest operating system's share of memory [W02]. Thus, if you are ever in need of a mechanism to represent a proportion of ownership, this concept just might be ... (wait for it) ... the ticket.

## 9.2 Ticket Mechanisms

Lottery scheduling also provides a number of mechanisms to manipulate tickets in different and sometimes useful ways. One way is with the concept of **ticket currency**. Currency allows a user with a set of tickets to allocate tickets among their own jobs in whatever currency they would like; the system then automatically converts said currency into the correct global value.

For example, assume users A and B have each been given 100 tickets. User A is running two jobs, A1 and A2, and gives them each 500 tickets (out of 1000 total) in A's currency. User B is running only 1 job and gives it 10 tickets (out of 10 total). The system converts A1's and A2's allocation from 500 each in A's currency to 50 each in the global currency; similarly, B1's 10 tickets is converted to 100 tickets. The lottery is then held over the global ticket currency (200 total) to determine which job runs.

```
User A -> 500 (A's currency) to A1 -> 50 (global currency)
        -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

Another useful mechanism is **ticket transfer**. With transfers, a process can temporarily hand off its tickets to another process. This ability is especially useful in a client/server setting, where a client process sends a message to a server asking it to do some work on the client's behalf. To speed up the work, the client can pass the tickets to the server and thus try to maximize the performance of the server while the server is handling the client's request. When finished, the server then transfers the tickets back to the client and all is as before.

Finally, **ticket inflation** can sometimes be a useful technique. With inflation, a process can temporarily raise or lower the number of tickets it owns. Of course, in a competitive scenario with processes that do not trust one another, this makes little sense; one greedy process could give itself a vast number of tickets and take over the machine. Rather, inflation can be applied in an environment where a group of processes trust one another; in such a case, if any one process knows it needs more CPU time, it can boost its ticket value as a way to reflect that need to the system, all without communicating with any other processes.

## 9.2 티켓 메커니즘

비례 배분 스케줄링 / 영문 원문 3쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

### 팁: 티켓으로 몫을 표현하라

추첨 스케줄링과 스트라이드 스케줄링 설계에서 가장 강력하면서도 기본적인 메커니즘 중 하나가 티켓이다. 여기의 예에서는 티켓으로 프로세스의 CPU 몫을 표현하지만, 훨씬 폭넓게 적용할 수 있다. 예를 들어 하이퍼바이저의 가상 메모리 관리에 관한 더 최근의 연구에서 Waldspurger는 게스트 운영체제의 메모리 몫을 티켓으로 표현하는 방법을 보여 준다 [W02]. 따라서 소유 비율을 표현할 메커니즘이 필요하다면, 이 개념이 바로... 잠깐만 기다리면... 알맞은 티켓이 될지도 모른다.

### 9.2 티켓 메커니즘

추첨 스케줄링은 티켓을 다양한 방식으로 조작하는 여러 메커니즘도 제공한다. 이러한 방식은 때로 유용하다. 그중 하나는 티켓 화폐(ticket currency)라는 개념이다. 화폐를 사용하면 티켓을 가진 사용자는 원하는 화폐 단위로 자기 작업들에 티켓을 배분할 수 있다. 그러면 시스템이 그 화폐를 알맞은 전역 값으로 자동 변환한다.

예를 들어 사용자 A와 B가 각각 티켓 100장을 받았다고 하자. 사용자 A는 작업 A1과 A2를 실행하며, A의 화폐 단위로 총 1,000장 중 500장씩을 각 작업에 준다. 사용자 B는 작업을 하나만 실행하며 총 10장 중 10장을 그 작업에 준다. 시스템은 A1과 A2에 배정된 A의 화폐 500장씩을 전역 화폐 50장씩으로 변환한다. 마찬가지로 B1의 티켓 10장은 100장으로 변환된다. 그러고 나서 전역 티켓 화폐 총 200장을 대상으로 추첨하여 실행할 작업을 결정한다.

사용자 A -> A1에 500장(A의 화폐) -> 50장(전역 화폐)  
 -> A2에 500장(A의 화폐) -> 50장(전역 화폐)  
 사용자 B -> B1에 10장(B의 화폐) -> 100장(전역 화폐)

또 다른 유용한 메커니즘은 티켓 양도(ticket transfer)이다. 양도를 통해 프로세스는 자신의 티켓을 다른 프로세스에 일시적으로 넘겨줄 수 있다. 이 기능은 클라이언트 프로세스가 서버에 메시지를 보내 자신을 대신해 어떤 작업을 해 달라고 요청하는 클라이언트/서버 환경에서 특히 유용하다. 작업 속도를 높이기 위해 클라이언트는 티켓을 서버에 넘겨주어, 서버가 자신의 요청을 처리하는 동안 서버 성능을 최대화하려고 할 수 있다. 작업이 끝나면 서버가 티켓을 클라이언트에 돌려주고 모든 것이 이전 상태로 돌아간다.

마지막으로 티켓 인플레이션(ticket inflation)도 때로 유용하다. 인플레이션을 이용하면 프로세스가 자신이 가진 티켓 수를 일시적으로 늘리거나 줄일 수 있다. 물론 서로 신뢰하지 않는 프로세스들이 경쟁하는 상황에서는 별 의미가 없다. 욕심 많은 프로세스 하나가 자신에게 막대한 수의 티켓을 주고 기계를 독차지할 수 있기 때문이다. 대신 인플레이션은 프로세스들이 서로 신뢰하는 환경에 적용할 수 있다. 그런 환경에서는 어떤 프로세스가 자신에게 더 많은 CPU 시간이 필요하다는 것을 알면 티켓 값을 늘려 그 필요를 시스템에 반영할 수 있다. 이 모든 일을 다른 프로세스와 통신하지 않고도 할 수 있다.

```

1 // counter: used to track if we've found the winner yet
2 int counter = 0;
3
4 // winner: call some random number generator to
5 //         get a value >= 0 and <= (totaltickets - 1)
6 int winner = getrandom(0, totaltickets);
7
8 // current: use this to walk through the list of jobs
9 node_t *current = head;
10 while (current) {
11     counter = counter + current->tickets;
12     if (counter > winner)
13         break; // found the winner
14     current = current->next;
15 }
16 // 'current' is the winner: schedule it...

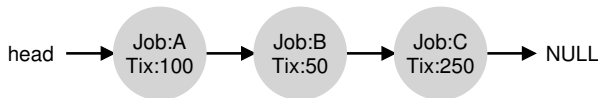
```

Figure 9.1: Lottery Scheduling Decision Code

### 9.3 Implementation

Probably the most amazing thing about lottery scheduling is the simplicity of its implementation. All you need is a good random number generator to pick the winning ticket, a data structure to track the processes of the system (e.g., a list), and the total number of tickets.

Let's assume we keep the processes in a list. Here is an example comprised of three processes, A, B, and C, each with some number of tickets.



To make a scheduling decision, we first have to pick a random number (the winner) from the total number of tickets (400)<sup>2</sup>. Let's say we pick the number 300. Then, we simply traverse the list, with a simple counter used to help us find the winner (Figure 9.1).

The code walks the process list, adding each ticket value to `counter` until the value exceeds `winner`. Once that is the case, the current list element is the winner. With our example of the winning ticket being 300, the following takes place. First, `counter` is incremented to 100 to account for A's tickets; because 100 is less than 300, the loop continues. Then `counter` would be updated to 150 (B's tickets), still less than 300 and thus again we continue. Finally, `counter` is updated to 400 (clearly greater than 300), and thus we break out of the loop with `current` pointing at C (the winner).

<sup>2</sup>Surprisingly, as pointed out by Björn Lindberg, this can be challenging to do correctly; for more details, see <http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range>.

## 9.3 구현

비례 배분 스케줄링 / 영문 원문 4쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

```
// counter: 당첨자를 찾았는지 추적하는 데 사용
int counter = 0;

// winner: 난수 생성기를 호출하여
// 0 이상 (totaltickets - 1) 이하의 값을 얻는다
int winner = getRandom(0, totaltickets);

// current: 작업 목록을 순회하는 데 사용
node_t *current = head;
while (current) {
    counter = counter + current->tickets;
    if (counter > winner)
        break; // 당첨자를 찾았다
    current = current->next;
}
// 'current'가 당첨자이다. 이를 스케줄링한다...
```

그림 9.1: 추첨 스케줄링 결정 코드

### 9.3 구현

추첨 스케줄링에서 아마 가장 놀라운 점은 구현이 단순하다는 것이다. 필요한 것은 당첨 티켓을 뽑을 좋은 난수 생성기, 시스템의 프로세스들을 추적할 자료구조(예: 리스트), 그리고 전체 티켓 수뿐이다.

프로세스들을 리스트에 저장한다고 가정하자. 다음은 각각 일정 수의 티켓을 가진 세 프로세스 A, B, C로 구성된 예이다.

```
head -> [작업: A, 티켓: 100]
      -> [작업: B, 티켓: 50]
      -> [작업: C, 티켓: 250] -> NULL
```

스케줄링을 결정하려면 먼저 총 400장의 티켓 중에서 난수, 즉 당첨 번호를 뽑아야 한다[각주 2]. 300을 뽑았다고 하자. 그리고 나서는 당첨자를 찾는 데 도움을 주는 간단한 카운터를 사용하면서 리스트를 순회하면 된다(그림 9.1).

코드는 프로세스 리스트를 순회하며 각 티켓 수를 counter에 더한다. counter가 winner를 초과하면 현재 리스트 원소가 당첨자이다. 당첨 티켓이 300인 이 예에서는 다음과 같이 진행된다. 먼저 A의 티켓을 반영해 counter를 100으로 늘린다. 100은 300보다 작으므로 반복을 계속한다. 다음으로 B의 티켓을 더해 counter를 150으로 갱신한다. 여전히 300보다 작으므로 계속 진행한다. 마지막으로 counter가 400으로 갱신되면 300보다 크므로 반복문을 빠져나온다. 이때 current는 당첨자인 C를 가리킨다.

각주 2. Björn Lindberg가 지적했듯이, 놀랍게도 이를 올바르게 수행하기는 어려울 수 있다. 자세한 내용: <http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range>

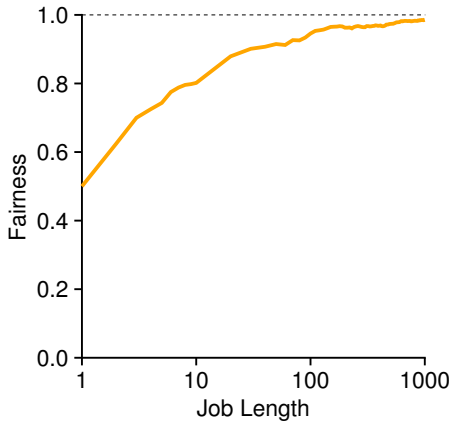


Figure 9.2: Lottery Fairness Study

To make this process most efficient, it might generally be best to organize the list in sorted order, from the highest number of tickets to the lowest. The ordering does not affect the correctness of the algorithm; however, it does ensure in general that the fewest number of list iterations are taken, especially if there are a few processes that possess most of the tickets.

## 9.4 An Example

To make the dynamics of lottery scheduling more understandable, we now perform a brief study of the completion time of two jobs competing against one another, each with the same number of tickets (100) and same run time ( $R$ , which we will vary).

In this scenario, we'd like for each job to finish at roughly the same time, but due to the randomness of lottery scheduling, sometimes one job finishes before the other. To quantify this difference, we define a simple **fairness metric**,  $F$  which is simply the time the first job completes divided by the time that the second job completes. For example, if  $R = 10$ , and the first job finishes at time 10 (and the second job at 20),  $F = \frac{10}{20} = 0.5$ . When both jobs finish at nearly the same time,  $F$  will be quite close to 1. In this scenario, that is our goal: a perfectly fair scheduler would achieve  $F = 1$ .

Figure 9.2 plots the average fairness as the length of the two jobs ( $R$ ) is varied from 1 to 1000 over thirty trials (results are generated via the simulator provided at the end of the chapter). As you can see from the graph, when the job length is not very long, average fairness can be quite low. Only as the jobs run for a significant number of time slices does the lottery scheduler approach the desired fair outcome.

## 9.4 예제와 공정성

비례 배분 스케줄링 / 영문 원문 5쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 9.2: 추첨 스케줄링의 공정성 연구. 그래프는 인접한 원문 페이지를 참조한다. 가로축 Job Length는 작업 길이이며 눈금은 1, 10, 100, 1000이다. 세로축 Fairness는 공정성이며 눈금은 0.0부터 1.0까지 0.2 간격이다.

이 과정을 가장 효율적으로 수행하려면 일반적으로 티켓 수가 많은 프로세스부터 적은 프로세스 순으로 리스트를 정렬하는 것이 좋다. 순서는 알고리즘의 정확성에 영향을 주지 않는다. 하지만 대체로 리스트 순회 횟수를 최소화하며, 특히 소수의 프로세스가 티켓 대부분을 갖고 있을 때 효과적이다.

### 9.4 예제

추첨 스케줄링의 동작을 더 잘 이해하기 위해, 서로 경쟁하는 두 작업의 완료 시간을 간단히 살펴보자. 두 작업은 티켓 수가 각각 100장으로 같고, 실행 시간도 R로 같다. R의 값은 바꾸어 가며 살펴본다.

이 상황에서 우리는 두 작업이 대략 같은 시각에 끝나기를 원한다. 하지만 추첨 스케줄링의 무작위성 때문에 때로 한 작업이 다른 작업보다 먼저 끝난다. 이 차이를 정량화하기 위해 간단한 공정성 지표 F를 정의한다. 먼저 끝난 작업의 완료 시각을 나중에 끝난 작업의 완료 시각으로 나눈 값이다. 예를 들어  $R = 10$ 이고 첫 번째 작업이 시각 10에, 두 번째 작업이 시각 20에 끝나면  $F = 10/20 = 0.5$ 이다. 두 작업이 거의 같은 시각에 끝나면 F는 1에 아주 가까워진다. 이 상황에서 그것이 우리의 목표이다. 완전히 공정한 스케줄러라면  $F = 1$ 을 달성할 것이다.

그림 9.2는 두 작업의 길이 R을 1부터 1,000까지 바꾸면서 각각 30회 실행한 평균 공정성을 나타낸다. 결과는 이 장 끝에 제공하는 시뮬레이터로 생성했다. 그래프에서 보듯 작업 길이가 그다지 길지 않으면 평균 공정성이 상당히 낮을 수 있다. 작업이 충분히 많은 타임 슬라이스 동안 실행되어야 추첨 스케줄러가 원하는 공정한 결과에 가까워진다.

## 9.5 How To Assign Tickets?

One problem we have not addressed with lottery scheduling is: how to assign tickets to jobs? This problem is a tough one, because of course how the system behaves is strongly dependent on how tickets are allocated. One approach is to assume that the users know best; in such a case, each user is handed some number of tickets, and a user can allocate tickets to any jobs they run as desired. However, this solution is a non-solution: it really doesn't tell you what to do. Thus, given a set of jobs, the "ticket-assignment problem" remains open.

## 9.6 Stride Scheduling

You might also be wondering: why use randomness at all? As we saw above, while randomness gets us a simple (and approximately correct) scheduler, it occasionally will not deliver the exact right proportions, especially over short time scales. For this reason, Waldspurger invented **stride scheduling**, a deterministic fair-share scheduler [W95].

Stride scheduling is also straightforward. Each job in the system has a stride, which is inverse in proportion to the number of tickets it has. In our example above, with jobs A, B, and C, with 100, 50, and 250 tickets, respectively, we can compute the stride of each by dividing some large number by the number of tickets each process has been assigned. For example, if we divide 10,000 by each of those ticket values, we obtain the following stride values for A, B, and C: 100, 200, and 40. We call this value the **stride** of each process; every time a process runs, we will increment a counter for it (called its **pass** value) by its stride to track its global progress.

The scheduler then uses the stride and pass to determine which process should run next. The basic idea is simple: at any given time, pick the process to run that has the lowest pass value so far; when you run a process, increment its pass counter by its stride. A pseudocode implementation is provided by Waldspurger [W95]:

```
curr = remove_min(queue); // pick client with min pass
schedule(curr);           // run for quantum
curr->pass += curr->stride; // update pass using stride
insert(queue, curr);       // return curr to queue
```

In our example, we start with three processes (A, B, and C), with stride values of 100, 200, and 40, and all with pass values initially at 0. Thus, at first, any of the processes might run, as their pass values are equally low. Assume we pick A (arbitrarily; any of the processes with equal low pass values can be chosen). A runs; when finished with the time slice, we update its pass value to 100. Then we run B, whose pass value is then set to 200. Finally, we run C, whose pass value is incremented to 40. At this point, the algorithm will pick the lowest pass value, which is C's, and run it, updating its pass to 80 (C's stride is 40, as you recall). Then C will

## 티켓 할당과 스트라이드 스케줄링

비례 배분 스케줄링 / 영문 원문 6쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

### 9.5 티켓을 어떻게 할당할까?

추첨 스케줄링에서 아직 다루지 않은 문제가 있다. 작업에 티켓을 어떻게 할당할 것인가? 시스템 동작은 티켓 할당 방식에 크게 좌우되므로 이는 어려운 문제이다. 한 가지 접근법은 사용자가 가장 잘 안다고 가정하는 것이다. 이 경우 각 사용자에게 일정 수의 티켓을 주고, 사용자는 자신이 실행하는 작업들에 원하는 대로 티켓을 배분할 수 있다. 하지만 이는 해결책이라고 할 수 없다. 실제로 무엇을 해야 하는지 알려 주지 않기 때문이다. 따라서 작업 집합이 주어졌을 때의 '티켓 할당 문제'는 여전히 해결되지 않은 문제로 남는다.

### 9.6 스트라이드 스케줄링

애초에 왜 무작위성을 사용해야 하는지 궁금할 수도 있다. 앞서 보았듯 무작위성을 이용하면 간단하고 대략 올바른 스케줄러를 만들 수 있지만, 특히 짧은 시간 범위에서는 정확한 비율을 제공하지 못할 때가 있다. 이 때문에 Waldspurger는 결정론적인 공정 배분 스케줄러인 스트라이드 스케줄링(stride scheduling)을 고안했다 [W95].

스트라이드 스케줄링도 간단하다. 시스템의 각 작업에는 자신이 가진 티켓 수에 반비례하는 스트라이드(stride, 보폭)가 있다. 앞의 예에서는 A, B, C가 각각 티켓 100장, 50장, 250장을 갖고 있다. 어떤 큰 수를 각 프로세스에 할당된 티켓 수로 나누어 스트라이드를 계산할 수 있다. 예를 들어 10,000을 각각의 티켓 수로 나누면 A, B, C의 스트라이드는 각각 100, 200, 40이다. 이 값을 각 프로세스의 스트라이드라고 한다. 프로세스가 실행될 때마다 패스 값(pass value)이라는 카운터를 해당 스트라이드만큼 늘려 전체적인 진행 정도를 추적한다.

스케줄러는 스트라이드와 패스를 이용해 다음에 실행할 프로세스를 정한다. 기본 아이디어는 단순하다. 어느 시점이든 현재까지 패스 값이 가장 작은 프로세스를 골라 실행한다. 프로세스를 실행하면 패스 카운터를 스트라이드만큼 늘린다. Waldspurger [W95]가 제시한 의사 코드는 다음과 같다.

```
curr = remove_min(queue); // 패스가 최소인 클라이언트를 선택
schedule(curr);           // 한 쿼텀 동안 실행
curr->pass += curr->stride; // 스트라이드로 패스를 갱신
insert(queue, curr);       // curr를 큐에 다시 넣음
```

이 예에서는 A, B, C 세 프로세스로 시작한다. 스트라이드는 각각 100, 200, 40이고 초기 패스 값은 모두 0이다. 따라서 처음에는 어느 프로세스나 실행될 수 있다. 패스 값이 모두 똑같이 작기 때문이다. 임의로 A를 골랐다고 하자. 같은 최소 패스 값을 가진 프로세스 중 어느 것이든 선택할 수 있다. A가 실행되고 타임 슬라이스가 끝나면 패스 값을 100으로 갱신한다. 이어서 B를 실행하여 패스 값을 200으로 설정한다. 마지막으로 C를 실행하여 패스 값을 40으로 늘린다. 이 시점에서 알고리즘은 패스 값이 가장 작은 C를 골라 실행하고, 패스를 80으로 갱신한다. C의 스트라이드는 40임을 기억하자. 그다음 C는 다시 실행된다. (다음 쪽에 계속)

| Pass(A)<br>(stride=100) | Pass(B)<br>(stride=200) | Pass(C)<br>(stride=40) | Who Runs? |
|-------------------------|-------------------------|------------------------|-----------|
| 0                       | 0                       | 0                      | A         |
| 100                     | 0                       | 0                      | B         |
| 100                     | 200                     | 0                      | C         |
| 100                     | 200                     | 40                     | C         |
| 100                     | 200                     | 80                     | C         |
| 100                     | 200                     | 120                    | A         |
| 200                     | 200                     | 120                    | C         |
| 200                     | 200                     | 160                    | C         |
| 200                     | 200                     | 200                    | ...       |

Figure 9.3: Stride Scheduling: A Trace

run again (still the lowest pass value), raising its pass to 120. A will run now, updating its pass to 200 (now equal to B’s). Then C will run twice more, updating its pass to 160 then 200. At this point, all pass values are equal again, and the process will repeat, ad infinitum. Figure 9.3 traces the behavior of the scheduler over time.

As we can see from the figure, C ran five times, A twice, and B just once, exactly in proportion to their ticket values of 250, 100, and 50. Lottery scheduling achieves the proportions probabilistically over time; stride scheduling gets them exactly right at the end of each scheduling cycle.

So you might be wondering: given the precision of stride scheduling, why use lottery scheduling at all? Well, lottery scheduling has one nice property that stride scheduling does not: no global state. Imagine a new job enters in the middle of our stride scheduling example above; what should its pass value be? Should it be set to 0? If so, it will monopolize the CPU. With lottery scheduling, there is no global state per process; we simply add a new process with whatever tickets it has, update the single global variable to track how many total tickets we have, and go from there. In this way, lottery makes it much easier to incorporate new processes in a sensible manner.

9.7 The Linux Completely Fair Scheduler (CFS)

Despite these earlier works in fair-share scheduling, one (older) Linux approach achieves similar goals in an alternate manner. The scheduler, entitled the **Completely Fair Scheduler** (or **CFS**) [J09], implements fair-share scheduling, but does so in a highly efficient and scalable manner.

To achieve its efficiency goals, CFS aims to spend very little time making scheduling decisions, through both its inherent design and its clever use of data structures well-suited to the task. Recent studies have shown that scheduler efficiency is surprisingly important; specifically, in a study of Google datacenters, Kanev et al. show that even after aggressive optimization, scheduling uses about 5% of overall datacenter CPU time [K+15]. Reducing that overhead as much as possible is thus a key goal in modern scheduler architecture.

# 스트라이드 실행 추적과 Linux CFS

비례 배분 스케줄링 / 영문 원문 7쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

| Pass(A) (stride=100) | Pass(B) (stride=200) | Pass(C) (stride=40) | 실행 프로세스 |
|----------------------|----------------------|---------------------|---------|
| 0                    | 0                    | 0                   | A       |
| 100                  | 0                    | 0                   | B       |
| 100                  | 200                  | 0                   | C       |
| 100                  | 200                  | 40                  | C       |
| 100                  | 200                  | 80                  | C       |
| 100                  | 200                  | 120                 | A       |
| 200                  | 200                  | 120                 | C       |
| 200                  | 200                  | 160                 | C       |
| 200                  | 200                  | 200                 | ...     |

그림 9.3: 스트라이드 스케줄링 실행 추적

(앞쪽에서 계속) C는 패스 값이 여전히 가장 작으므로 다시 실행되고, 패스가 120으로 증가한다. 이제 A가 실행되어 패스가 200으로 갱신된다. B와 같은 값이다. 이어서 C가 두 번 더 실행되어 패스가 160, 200으로 갱신된다. 이 시점에 모든 패스 값이 다시 같아지며, 이 과정이 끝없이 반복된다. 그림 9.3은 시간에 따른 스케줄러의 동작을 추적한 것이다.

그림에서 보듯 C는 다섯 번, A는 두 번, B는 한 번 실행되었다. 티켓 수인 250, 100, 50에 정확히 비례한다. 주춤 스케줄링은 시간이 지남에 따라 확률적으로 이 비율을 달성한다. 스트라이드 스케줄링은 매 스케줄링 주기가 끝날 때 정확히 이 비율을 달성한다.

이렇게 스트라이드 스케줄링이 정확하다면 왜 주춤 스케줄링을 사용하는지 궁금할 수 있다. 주춤 스케줄링에는 스트라이드 스케줄링에 없는 장점 하나가 있다. 전역 상태가 없다는 점이다. 위의 스트라이드 스케줄링 도중 새 작업이 들어온다고 생각해 보자. 이 작업의 패스 값은 얼마여야 할까? 0으로 설정해야 할까? 그렇게 하면 이 작업이 CPU를 독점할 것이다. 주춤 스케줄링에서는 프로세스별로 전체 진행 상황을 나타내는 상태가 없다. 새 프로세스를 그 프로세스의 티켓과 함께 추가하고, 전체 티켓 수를 추적하는 하나의 전역 변수만 갱신한 뒤 계속 진행하면 된다. 이처럼 주춤 스케줄링은 새 프로세스를 합리적으로 받아들이기가 훨씬 쉽다.

## 9.7 Linux 완전 공정 스케줄러(CFS)

공정 배분 스케줄링에 관한 앞선 연구들에도 불구하고, Linux의 한 가지 이전 방식은 다른 방법으로 비슷한 목표를 달성한다. 완전 공정 스케줄러(Completely Fair Scheduler, CFS) [J09]라는 이 스케줄러는 공정 배분 스케줄링을 매우 효율적이고 확장성 있는 방식으로 구현한다.

효율성 목표를 달성하기 위해 CFS는 설계 자체와 작업에 적합한 자료구조의 영리한 활용을 통해 스케줄링 결정에 아주 적은 시간만 쓰도록 한다. 최근 연구에 따르면 스케줄러 효율성은 놀라울 만큼 중요하다. 구체적으로 Kanev 등의 Google 데이터센터 연구는 적극적인 최적화 이후에도 스케줄링이 데이터센터 전체 CPU 시간의 약 5%를 사용한다는 것을 보여 준다 [K+15]. 따라서 이러한 오버헤드를 최대한 줄이는 것은 현대 스케줄러 구조의 핵심 목표이다.

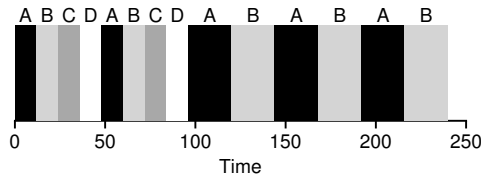


Figure 9.4: CFS Simple Example

## Basic Operation

Whereas most schedulers are based around the concept of a fixed time slice, CFS operates a bit differently. Its goal is simple: to fairly divide a CPU evenly among all competing processes. It does so through a simple counting-based technique known as **virtual runtime** (**vruntime**).

As each process runs, it accumulates **vruntime**. In the most basic case, each process's **vruntime** increases at the same rate, in proportion with physical (real) time. When a scheduling decision occurs, CFS will pick the process with the *lowest* **vruntime** to run next.

This raises a question: how does the scheduler know when to stop the currently running process, and run the next one? The tension here is clear: if CFS switches too often, fairness is increased, as CFS will ensure that each process receives its share of CPU even over miniscule time windows, but at the cost of performance (too much context switching); if CFS switches less often, performance is increased (reduced context switching), but at the cost of near-term fairness.

CFS manages this tension through various control parameters. The first is **sched\_latency**. CFS uses this value to determine how long one process should run before considering a switch (effectively determining its time slice but in a dynamic fashion). A typical **sched\_latency** value is 48 (milliseconds); CFS divides this value by the number ( $n$ ) of processes running on the CPU to determine the time slice for a process, and thus ensures that over this period of time, CFS will be completely fair.

For example, if there are  $n = 4$  processes running, CFS divides the value of **sched\_latency** by  $n$  to arrive at a per-process time slice of 12 ms. CFS then schedules the first job and runs it until it has used 12 ms of (virtual) runtime, and then checks to see if there is a job with lower **vruntime** to run instead. In this case, there is, and CFS would switch to one of the three other jobs, and so forth. Figure 9.4 shows an example where the four jobs (A, B, C, D) each run for two time slices in this fashion; two of them (C, D) then complete, leaving just two remaining, which then each run for 24 ms in round-robin fashion.

But what if there are “too many” processes running? Wouldn't that lead to too small of a time slice, and thus too many context switches? Good question! And the answer is yes.

To address this issue, CFS adds another parameter, **min\_granularity**, which is usually set to a value like 6 ms. CFS will never set the time slice

## CFS의 기본 동작

비례 배분 스케줄링 / 영문 원문 8쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 9.4: CFS의 간단한 예. 그림은 인접한 원문 페이지를 참조한다. 가로축 Time은 시간이며 눈금은 0, 50, 100, 150, 200, 250이다. 실행 순서는 A, B, C, D, A, B, C, D, A, B, A, B, A, B이다.

### 기본 동작

대부분의 스케줄러는 고정된 타임 슬라이스 개념을 기반으로 하지만 CFS는 조금 다르게 동작한다. 목표는 간단하다. 경쟁하는 모든 프로세스에 CPU를 공정하고 균등하게 나누어 주는 것이다. 이를 위해 가상 실행 시간 (virtual runtime, vruntime)이라는 단순한 계수 기법을 사용한다.

각 프로세스가 실행될 때 vruntime이 누적된다. 가장 기본적인 경우 각 프로세스의 vruntime은 물리적인 실제 시간에 비례하여 같은 속도로 증가한다. 스케줄링을 결정할 때 CFS는 vruntime이 가장 작은 프로세스를 다음 실행 대상으로 선택한다.

여기서 질문이 생긴다. 스케줄러는 현재 실행 중인 프로세스를 언제 멈추고 다음 프로세스를 실행할지 어떻게 알까? 상충 관계는 분명하다. CFS가 너무 자주 전환하면 아주 짧은 시간 구간에서도 각 프로세스가 자신의 CPU 몫을 받도록 보장하므로 공정성이 높아진다. 하지만 문맥 교환이 너무 많아져 성능을 희생한다. 반대로 CFS가 드물게 전환하면 문맥 교환이 줄어 성능이 높아지지만 단기적인 공정성을 희생한다.

CFS는 다양한 제어 매개변수로 이러한 상충 관계를 조절한다. 첫 번째는 sched\_latency이다. CFS는 이 값으로 한 프로세스를 얼마나 실행한 뒤 전환을 고려할지 정한다. 사실상 타임 슬라이스를 동적으로 정하는 것이다. sched\_latency의 전형적인 값은 48밀리초이다. CFS는 이 값을 CPU에서 실행되는 프로세스 수  $n$ 으로 나누어 프로세스의 타임 슬라이스를 정하고, 이 기간에 완전한 공정성을 보장한다.

예를 들어 실행되는 프로세스가  $n = 4$ 개이면, CFS는 sched\_latency를  $n$ 으로 나누어 프로세스당 12ms의 타임 슬라이스를 얻는다. 이어서 첫 작업을 스케줄링하고 가상 실행 시간을 12ms 사용할 때까지 실행한 뒤, 대신 실행할 vruntime이 더 작은 작업이 있는지 확인한다. 이 경우 그런 작업이 있으므로 나머지 세 작업 중 하나로 전환하고 같은 과정을 반복한다. 그림 9.4에서는 네 작업 A, B, C, D가 이러한 방식으로 각각 두 개의 타임 슬라이스 동안 실행된다. 그리고 나서 C와 D가 완료되어 두 작업만 남으며, 남은 작업은 라운드 로빈 방식으로 각 24ms씩 실행된다.

그런데 실행되는 프로세스가 '너무 많으면' 어떻게 될까? 타임 슬라이스가 너무 작아져 문맥 교환이 너무 많아지지 않을까? 좋은 질문이다! 답은 그렇다는 것이다.

이 문제를 해결하기 위해 CFS는 min\_granularity라는 매개변수를 하나 더 두며, 보통 6ms 정도로 설정한다. CFS는 프로세스의 타임 슬라이스를 이 값보다 작게 설정하지 않는다. (다음 쪽에 계속)

of a process to less than this value, ensuring that not too much time is spent in scheduling overhead.

For example, if there are ten processes running, our original calculation would divide `sched_latency` by ten to determine the time slice (result: 4.8 ms). However, because of `min_granularity`, CFS will set the time slice of each process to 6 ms instead. Although CFS won't (quite) be perfectly fair over the target scheduling latency (`sched_latency`) of 48 ms, it will be close, while still achieving high CPU efficiency.

Note that CFS utilizes a periodic timer interrupt, which means it can only make decisions at fixed time intervals. This interrupt goes off frequently (e.g., every 1 ms), giving CFS a chance to wake up and determine if the current job has reached the end of its run. If a job has a time slice that is not a perfect multiple of the timer interrupt interval, that is OK; CFS tracks `vruntime` precisely, which means that over the long haul, it will eventually approximate ideal sharing of the CPU.

### Weighting (Niceness)

CFS also enables controls over process priority, enabling users or administrators to give some processes a higher share of the CPU. It does this not with tickets, but through a classic UNIX mechanism known as the **nice** level of a process. The nice parameter can be set anywhere from -20 to +19 for a process, with a default of 0. Positive nice values imply *lower* priority and negative values imply *higher* priority; when you're too nice, you just don't get as much (scheduling) attention, alas.

CFS maps the nice value of each process to a weight, as shown here:

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */ 9548, 7620, 6100, 4904, 3906,
    /* -5 */ 3121, 2501, 1991, 1586, 1277,
    /* 0 */ 1024, 820, 655, 526, 423,
    /* 5 */ 335, 272, 215, 172, 137,
    /* 10 */ 110, 87, 70, 56, 45,
    /* 15 */ 36, 29, 23, 18, 15,
};
```

These weights allow us to compute the effective time slice of each process (as we did before), but now accounting for their priority differences. The formula used to do so is as follows, assuming  $n$  processes:

$$\text{time\_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched\_latency} \quad (9.1)$$

Let's do an example to see how this works. Assume there are two jobs, A and B. A, because it's our most precious job, is given a higher priority

## CFS의 최소 실행 단위와 가중치

비례 배분 스케줄링 / 영문 원문 9쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

(앞쪽에서 계속) 이렇게 하여 스케줄링 오버헤드에 너무 많은 시간을 쓰지 않도록 보장한다.

예를 들어 실행되는 프로세스가 10개라면 원래 계산은 sched\_latency를 10으로 나누어 타임 슬라이스를 정한다. 결과는 4.8ms이다. 그러나 min\_granularity 때문에 CFS는 각 프로세스의 타임 슬라이스를 대신 6ms로 설정한다. 목표 스케줄링 지연인 sched\_latency 48ms 내에서 완전히 공정하지는 않겠지만, 높은 CPU 효율을 유지하면서 이에 가까운 공정성을 달성한다.

CFS는 주기적인 타이머 인터럽트를 사용하므로 고정된 시간 간격에서만 결정을 내릴 수 있다는 점에 유의하자. 이 인터럽트는 예를 들어 1ms마다 자주 발생하여, CFS가 깨어나 현재 작업의 실행 시간이 끝났는지 판단할 기회를 준다. 작업의 타임 슬라이스가 타이머 인터럽트 간격의 정확한 정수배가 아니어도 괜찮다. CFS는 vruntime을 정확히 추적하므로 장기적으로는 이상적인 CPU 배분에 가까워진다.

### 가중치 부여(nice 값)

CFS는 프로세스 우선순위로 제어할 수 있게 하므로 사용자나 관리자가 일부 프로세스에 CPU의 더 큰 몫을 줄 수 있다. 티켓 대신 프로세스의 nice 수준이라는 고전적인 UNIX 메커니즘을 사용한다. 프로세스의 nice 매개 변수는 -20부터 +19 사이로 설정할 수 있으며 기본값은 0이다. 양수 nice 값은 낮은 우선순위를, 음수는 높은 우선순위를 뜻한다. 너무 친절하게 굴면(nice), 안타깝게도 스케줄링의 관심을 그만큼 덜 받게 된다.

CFS는 각 프로세스의 nice 값을 다음과 같이 가중치에 대응시킨다.

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */ 9548, 7620, 6100, 4904, 3906,
    /* -5 */ 3121, 2501, 1991, 1586, 1277,
    /* 0 */ 1024, 820, 655, 526, 423,
    /* 5 */ 335, 272, 215, 172, 137,
    /* 10 */ 110, 87, 70, 56, 45,
    /* 15 */ 36, 29, 23, 18, 15,
};
```

이 가중치로 앞서처럼 각 프로세스의 실제 타임 슬라이스를 계산할 수 있지만, 이제는 우선순위 차이를 반영한다. 프로세스가  $n$ 개라고 할 때 사용하는 공식은 다음과 같다.

$$\text{time\_slice\_k} = (\text{weight\_k} / \sum_{i=0..n-1} \text{weight\_i}) * \text{sched\_latency} \quad (9.1)$$

예를 들어 어떻게 동작하는지 살펴보자. 작업 A와 B가 있다고 하자. A는 우리의 가장 소중한 작업이므로 더 높은 우선순위를 준다. (다음 쪽에 계속)

by assigning it a nice value of -5; B, because we hates it<sup>3</sup>, just has the default priority (nice value equal to 0). This means  $\text{weight}_A$

(from the table) is 3121, whereas  $\text{weight}_B$

is 1024. If you then compute the time slice of each job, you'll find that A's time slice is about  $\frac{3}{4}$  of `sched_latency` (hence, 36 ms), and B's about  $\frac{1}{4}$  (hence, 12 ms).

In addition to generalizing the time slice calculation, the way CFS calculates `vruntime` must also be adapted. Here is the new formula, which takes the actual run time that process  $i$  has accrued ( $\text{runtime}_i$ )

and scales it inversely by the weight of the process, by dividing the default weight of 1024 ( $\text{weight}_0$ )

by its weight,  $\text{weight}_i$ .

In our running example, A's `vruntime` will accumulate at one-third the rate of B's.

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i \quad (9.2)$$

One smart aspect of the construction of the table of weights above is that the table preserves CPU proportionality ratios when the difference in nice values is constant. For example, if process A instead had a nice value of 5 (not -5), and process B had a nice value of 10 (not 0), CFS would schedule them in exactly the same manner as before. Run through the math yourself to see why.

## Using Red-Black Trees

One major focus of CFS is efficiency, as stated above. For a scheduler, there are many facets of efficiency, but one of them is as simple as this: when the scheduler has to find the next job to run, it should do so as quickly as possible. Simple data structures like lists don't scale: modern systems sometimes are comprised of 1000s of processes, and thus searching through a long-list every so many milliseconds is wasteful.

CFS addresses this by keeping processes in a **red-black tree** [B72]. A red-black tree is one of many types of balanced trees; in contrast to a simple binary tree (which can degenerate to list-like performance under worst-case insertion patterns), balanced trees do a little extra work to maintain low depths, and thus ensure that operations are logarithmic (and not linear) in time.

CFS does not keep *all* processes in this structure; rather, only running (or runnable) processes are kept therein. If a process goes to sleep (say, waiting on an I/O to complete, or for a network packet to arrive), it is removed from the tree and kept track of elsewhere.

<sup>3</sup>Yes, yes, we are using bad grammar here on purpose, please don't send in a bug fix. Why? Well, just a most mild of references to the Lord of the Rings, and our favorite anti-hero Gollum, nothing to get too excited about.

## 가중 가상 실행 시간과 레드-블랙 트리

비례 배분 스케줄링 / 영문 원문 10쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

(앞쪽에서 계속) A에는 nice 값 -5를 부여한다. B는 우리가 싫어하므로[각주 3] 기본 우선순위인 nice 값 0을 그대로 둔다. 따라서 위 표에서  $weight\_A$ 는 3121이고  $weight\_B$ 는 1024이다. 각 작업의 타임 슬라이스를 계산하면 A는  $sched\_latency$ 의 약 3/4, 즉 36ms를 받고 B는 약 1/4, 즉 12ms를 받는다.

타임 슬라이스 계산을 일반화하는 것 외에도 CFS가  $vruntime$ 을 계산하는 방식을 바꾸어야 한다. 아래의 새 공식은 프로세스  $i$ 가 누적인 실제 실행 시간  $runtime\_i$ 에, 기본 가중치 1024인  $weight\_0$ 을 해당 프로세스의 가중치  $weight\_i$ 로 나눈 값을 곱한다. 즉 프로세스 가중치에 반비례하도록 실제 실행 시간을 조정한다. 지금의 예에서 A의  $vruntime$ 은 B의 약 1/3 속도로 누적된다.

$$vruntime\_i = vruntime\_i + (weight\_0 / weight\_i) * runtime\_i \quad (9.2)$$

위 가중치 표의 영리한 점 하나는 nice 값의 차이가 일정하면 CPU 배분 비율도 유지하도록 구성되었다는 것이다. 예를 들어 프로세스 A의 nice 값이 -5 대신 5이고 B의 nice 값이 0 대신 10이라도 CFS는 이전과 똑같은 방식으로 스케줄링한다. 직접 계산하여 그 이유를 확인해 보자.

### 레드-블랙 트리 사용

앞에서 말했듯 CFS의 주요 관심사 중 하나는 효율성이다. 스케줄러의 효율성에는 여러 측면이 있지만, 그중 하나는 다음처럼 단순하다. 다음에 실행할 작업을 찾아야 할 때 가능한 한 빠르게 찾아야 한다는 것이다. 리스트 같은 단순한 자료구조는 규모가 커지면 잘 대응하지 못한다. 현대 시스템에는 때로 수천 개의 프로세스가 있으므로 몇 밀리초마다 긴 리스트를 검색하는 것은 낭비이다.

CFS는 프로세스를 레드-블랙 트리(red-black tree)에 저장하여 이 문제를 해결한다 [B72]. 레드-블랙 트리는 여러 종류의 균형 트리 중 하나이다. 단순한 이진 트리는 최악의 삽입 순서에서 리스트 수준의 성능으로 퇴화할 수 있다. 반면 균형 트리는 약간의 추가 작업으로 트리 깊이를 낮게 유지하므로 연산에 선형 시간이 아닌 로그 시간이 걸리도록 보장한다.

CFS가 모든 프로세스를 이 자료구조에 저장하는 것은 아니다. 실행 중이거나 실행 가능한 프로세스만 저장한다. 프로세스가 I/O 완료나 네트워크 패킷 도착을 기다리느라 잠들면 트리에서 제거되어 다른 곳에서 관리된다.

각주 3. 그렇다. 원문의 'we hates it'은 일부러 문법을 틀리게 쓴 것이므로 수정 요청을 보내지 말기 바란다. 이유는 반지의 제왕과 우리가 좋아하는 반응용 골룸을 아주 살짝 인용한 것일 뿐이다. 너무 호들갑 떨 일은 아니다.

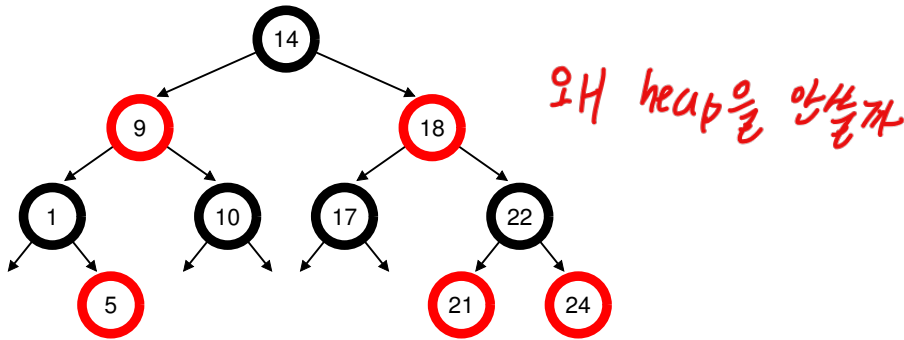


Figure 9.5: CFS Red-Black Tree

Let's look at an example to make this more clear. Assume there are ten jobs, and that they have the following values of `vruntime`: 1, 5, 9, 10, 14, 18, 17, 21, 22, and 24. If we kept these jobs in an ordered list, finding the next job to run would be simple: just remove the first element. However, when placing that job back into the list (in order), we would have to scan the list, looking for the right spot to insert it, an  $O(n)$  operation. Any search is also quite inefficient, also taking linear time on average.

Keeping the same values in a red-black tree makes most operations more efficient, as depicted in Figure 9.5. Processes are ordered in the tree by `vruntime`, and most operations (such as insertion and deletion) are logarithmic in time, i.e.,  $O(\log n)$ . When  $n$  is in the thousands, logarithmic is noticeably more efficient than linear.

### Dealing With I/O And Sleeping Processes

One problem with picking the lowest `vruntime` to run next arises with jobs that have gone to sleep for a long period of time. Imagine two processes, A and B, one of which (A) runs continuously, and the other (B) which has gone to sleep for a long period of time (say, 10 seconds). When B wakes up, its `vruntime` will be 10 seconds behind A's, and thus (if we're not careful), B will now monopolize the CPU for the next 10 seconds while it catches up, effectively starving A.

CFS handles this case by altering the `vruntime` of a job when it wakes up. Specifically, CFS sets the `vruntime` of that job to the minimum value found in the tree (remember, the tree only contains running jobs) [B+18]. CFS thus avoids starvation but with a cost: jobs that sleep for short periods of time frequently do not ever get their fair share [AC97].

### Other CFS Fun

CFS has many other features, too many to discuss at this point in the book. It includes numerous heuristics to improve cache performance, has strategies for handling multiple CPUs effectively (as discussed later in the book), can schedule across large groups of processes (instead of treating

## CFS의 트리와 잠든 프로세스 처리

비례 배분 스케줄링 / 영문 원문 11쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 9.5: CFS의 레드-블랙 트리. 트리의 모양과 노드 배치는 인접한 원문 페이지를 참조한다. 노드에 표시된 `vruntime` 값은 1, 5, 9, 10, 14, 18, 17, 22, 21, 24이다.

예를 들어 좀 더 명확히 살펴보자. 작업이 10개이고 `vruntime` 값이 각각 1, 5, 9, 10, 14, 18, 17, 21, 22, 24라고 하자. 이 작업들을 정렬된 리스트에 저장하면 다음에 실행할 작업을 찾기는 쉽다. 첫 번째 원소를 꺼내기만 하면 된다. 그러나 그 작업을 정렬 순서에 맞게 리스트에 다시 넣을 때는 알맞은 삽입 위치를 찾으려 리스트를 훑어야 하며, 이는  $O(n)$  연산이다. 검색도 상당히 비효율적이어서 평균적으로 선형 시간이 걸린다.

그림 9.5처럼 같은 값들을 레드-블랙 트리에 저장하면 대부분의 연산이 더 효율적이다. 프로세스는 `vruntime` 을 기준으로 트리에 정렬되고, 삽입과 삭제 같은 대부분의 연산은  $O(\log n)$ , 즉 로그 시간이 걸린다.  $n$ 이 수천 단위이면 로그 시간은 선형 시간보다 눈에 띄게 효율적이다.

### I/O와 잠든 프로세스 처리

`vruntime`이 가장 작은 작업을 다음 실행 대상으로 선택할 때의 한 가지 문제는 오랫동안 잠들어 있던 작업에서 발생한다. 프로세스 A와 B가 있는데, A는 계속 실행되고 B는 예를 들어 10초 동안 오래 잠들어 있다고 하자. B가 깨어나면 `vruntime`이 A보다 10초 뒤쳐져 있다. 따라서 주의하지 않으면 B가 이를 따라잡는 다음 10초 동안 CPU를 독점하여 사실상 A를 기아 상태에 빠뜨린다.

CFS는 작업이 깨어날 때 그 작업의 `vruntime`을 바꾸어 이 상황을 처리한다. 구체적으로는 작업의 `vruntime`을 트리에서 찾은 최솟값으로 설정한다. 트리에는 실행 중인 작업들만 들어 있다는 점을 기억하자 [B+18]. 이렇게 CFS는 기아를 방지하지만 대가가 따른다. 짧게 잠드는 일을 자주 반복하는 작업은 결국 자신의 공정한 몫을 받지 못한다 [AC97].

### CFS의 다른 흥미로운 기능

CFS에는 이 책의 이 지점에서 모두 다루기에는 너무 많은 기능이 있다. 캐시 성능을 개선하는 다양한 휴리스틱, 책의 뒷부분에서 다룰 여러 CPU를 효율적으로 처리하는 전략을 포함한다. 또한 각 프로세스를 독립된 개체로 취급하는 대신 큰 프로세스 그룹들을 대상으로 스케줄링할 수 있다. (다음 쪽에 계속)

**TIP: USE EFFICIENT DATA STRUCTURES WHEN APPROPRIATE**

In many cases, a list will do. In many cases, it will not. Knowing which data structure to use when is a hallmark of good engineering. In the case discussed herein, simple lists found in earlier schedulers simply do not work well on modern systems, particular in the heavily loaded servers found in datacenters. Such systems contain thousands of active processes; searching through a long list to find the next job to run on each core every few milliseconds would waste precious CPU cycles. A better structure was needed, and CFS provided one by adding an excellent implementation of a red-black tree. More generally, when picking a data structure for a system you are building, carefully consider its access patterns and its frequency of usage; by understanding these, you will be able to implement the right structure for the task at hand.

each process as an independent entity), and many other interesting features. Read recent research, starting with Bouron [B+18], to learn more.

## 9.8 Summary

We have introduced the concept of proportional-share scheduling and briefly discussed three approaches: lottery scheduling, stride scheduling, and CFS. Lottery uses randomness in a clever way to achieve proportional share; stride does so deterministically. CFS, the only “real” scheduler discussed in this chapter, is a bit like weighted round-robin with dynamic time slices, but built to scale and perform well under load; for many years, it was the most widely used fair-share scheduler.

No scheduler is a panacea, and fair-share schedulers have their fair share of problems. One issue is that such approaches do not particularly mesh well with I/O [AC97]; as mentioned above, jobs that perform I/O occasionally may not get their fair share of CPU. Another issue is that they leave open the hard problem of ticket or priority assignment, i.e., how do you know how many tickets your browser should be allocated, or to what nice value to set your text editor? Other general-purpose schedulers (such as the MLFQ we discussed previously) handle these issues automatically and thus may be more easily deployed.

The good news is that there are many domains in which these problems are not the dominant concern, and proportional-share schedulers are used to great effect. For example, in a **virtualized** data center (or **cloud**), where you might like to assign one-quarter of your CPU cycles to the Windows VM and the rest to your base Linux installation, proportional sharing can be simple and effective. The idea can also be extended to other resources; see Waldspurger [W02] for further details.

Finally, the world of Linux is always changing. As of Linux 6.6, the Earliest Eligible Virtual Deadline First (EEVDF) scheduler (another fair-sharing approach) became the default. Go read more about EEVDF [Z23, SA95] for more information. Just when you thought you were done!

## 효율적인 자료구조와 비례 배분 요약

비례 배분 스케줄링 / 영문 원문 12쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

### 팁: 적절한 경우 효율적인 자료구조를 사용하라

많은 경우에는 리스트면 충분하다. 또 많은 경우에는 그렇지 않다. 언제 어떤 자료구조를 사용할지 아는 것은 좋은 공학의 특징이다. 여기서 다룬 사례에서는 초기 스케줄러의 단순한 리스트가 현대 시스템, 특히 데이터센터의 부하가 높은 서버에서는 잘 작동하지 않는다. 이러한 시스템에는 수천 개의 활성 프로세스가 있다. 각 코어에서 다음으로 실행할 작업을 찾기 위해 몇 밀리초마다 긴 리스트를 검색하면 귀중한 CPU 사이클을 낭비한다. 더 나은 자료구조가 필요했고 CFS는 뛰어난 레드-블랙 트리 구현을 추가하여 이를 제공했다. 더 일반적으로 말하면, 만드는 시스템의 자료구조를 선택할 때는 접근 패턴과 사용 빈도를 주의 깊게 고려해야 한다. 이를 이해하면 현재 작업에 알맞은 자료구조를 구현할 수 있다.

(앞쪽에서 계속) 이 외에도 흥미로운 기능이 많다. 더 알아보려면 Bouron 등의 연구 [B+18]를 시작으로 최근 연구를 읽어 보자.

### 9.8 요약

비례 배분 스케줄링의 개념을 소개하고 추첨 스케줄링, 스트라이드 스케줄링, CFS라는 세 가지 접근법을 간단히 살펴보았다. 추첨 스케줄링은 무작위성을 영리하게 이용하여 비례 배분을 달성하고, 스트라이드 스케줄링은 이를 결정론적으로 달성한다. 이 장에서 다룬 유일한 '실제' 스케줄러인 CFS는 동적 타임 슬라이스를 사용하는 가중 라운드 로빈과 다소 비슷하지만, 부하가 있는 환경에서 규모를 확장하고 좋은 성능을 내도록 만들어졌다. 오랫동안 가장 널리 사용된 공정 배분 스케줄러였다.

어떤 스케줄러도 만능은 아니며, 공정 배분 스케줄러에도 나름의 문제가 있다. 한 가지는 이러한 접근법이 I/O와 특히 잘 맞지는 않는다는 점이다 [AC97]. 앞서 말했듯 가끔 I/O를 수행하는 작업은 CPU의 공정한 몫을 받지 못할 수 있다. 또 다른 문제는 티켓이나 우선순위를 할당하는 어려운 문제를 해결하지 않은 채 남겨 둔다는 것이다. 브라우저에 티켓을 몇 장 할당할지, 텍스트 편집기의 nice 값을 얼마로 설정할지 어떻게 알 수 있을까? 앞서 살펴본 MLFQ 같은 다른 범용 스케줄러는 이러한 문제를 자동으로 처리하므로 더 쉽게 적용할 수 있다.

다행히도 이러한 문제가 주요 관심사가 아닌 분야가 많으며, 그런 곳에서 비례 배분 스케줄러는 큰 효과를 낸다. 예를 들어 가상화된 데이터센터나 클라우드에서 CPU 사이클의 1/4을 Windows 가상 머신에, 나머지를 기반 Linux 환경에 할당하고 싶다면 비례 배분은 간단하고 효과적일 수 있다. 이 아이디어는 다른 자원으로도 확장할 수 있다. 자세한 내용은 Waldspurger [W02]를 참고하자.

마지막으로 Linux 세계는 항상 변한다. Linux 6.6부터는 또 다른 공정 배분 방식인 EEVDF(Earliest Eligible Virtual Deadline First) 스케줄러가 기본값이 되었다. 더 자세히 알고 싶으면 EEVDF 자료 [Z23, SA95]를 읽어 보자. 이제 다 끝났다고 생각했는데 말이다!

## References

- [AC97] “Extending Proportional-Share Scheduling to a Network of Workstations” by Andrea C. Arpaci-Dusseau and David E. Culler. PDPTA’97, June 1997. *A paper by one of the authors on how to extend proportional-share scheduling to work better in a clustered environment.*
- [B+18] “The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS” by J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, J. Sopena. USENIX ATC ’18, July 2018, Boston, Massachusetts. *A recent, detailed work comparing Linux CFS and the FreeBSD schedulers. An excellent overview of each scheduler is also provided. The result of the comparison: inconclusive. In some cases CFS was better, and in others, ULE (the BSD scheduler), was. Sometimes in life there are no easy answers.*
- [B72] “Symmetric binary B-Trees: Data Structure And Maintenance Algorithms” by Rudolf Bayer. Acta Informatica, Volume 1, Number 4, December 1972. *A cool balanced tree introduced before you were born (most likely). One of many balanced trees out there; study your algorithms book for more alternatives!*
- [D82] “Why Numbering Should Start At Zero” by Edsger Dijkstra, August 1982. Available: <http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF>. *A short note from E. Dijkstra, one of the pioneers of computer science. We’ll be hearing much more on this guy in the section on Concurrency. In the meanwhile, enjoy this note, which includes this motivating quote: “One of my colleagues — not a computing scientist — accused a number of younger computing scientists of ‘pedantry’ because they started numbering at zero.” The note explains why doing so is logical.*
- [K+15] “Profiling A Warehouse-scale Computer” by S. Kanev, P. Ranganathan, J. P. Darago, K. Hazelwood, T. Moseley, G. Wei, D. Brooks. ISCA ’15, June, 2015, Portland, Oregon. *A fascinating study of where the cycles go in modern data centers, which are increasingly where most of computing happens. Almost 20% of CPU time is spent in the operating system, 5% in the scheduler alone!*
- [J09] “Inside The Linux 2.6 Completely Fair Scheduler” by M. Tim Jones. December 15, 2009. <https://pages.cs.wisc.edu/~7Eremzi/OSTEP/Citations/inside-cfs.pdf>. *This is a simple overview of CFS from its earlier days. CFS was created by Ingo Molnar in a short burst of creativity which led to a 100K kernel patch developed in 62 hours.*
- [KL88] “A Fair Share Scheduler” by J. Kay and P. Lauder. CACM, Volume 31 Issue 1, January 1988. *An early reference to a fair-share scheduler.*
- [SA95] “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation” by Ion Stoica and Hussein M. Abdel-Wahab. Technical report, CS Department, Old Dominion University, TR-95-22. *The technical report that introduced EEVDF. Ion Stoica has since gone on to having great impact in systems, including with Apache Spark, the founding of Databricks, and much more.*
- [WW94] “Lottery Scheduling: Flexible Proportional-Share Resource Management” by Carl A. Waldspurger and William E. Weihl. OSDI ’94, November 1994. *The landmark paper on lottery scheduling that got the systems community re-energized about scheduling, fair sharing, and the power of simple randomized algorithms.*
- [W95] “Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management” by Carl A. Waldspurger. Ph.D. Thesis, MIT, 1995. *The award-winning thesis of Waldspurger’s that outlines lottery and stride scheduling. If you’re thinking of writing a Ph.D. dissertation at some point, you should always have a good example around, to give you something to strive for: this is such a good one.*
- [W02] “Memory Resource Management in VMware ESX Server” by Carl A. Waldspurger. OSDI ’02, Boston, Massachusetts. *The paper to read about memory management in VMMs (a.k.a., hypervisors). In addition to being relatively easy to read, the paper contains numerous cool ideas about this new type of VMM-level memory management.*
- [Z23] “The EEVDF Scheduler” by Peter Zijlstra. Please go to this URL for details on EEVDF: <https://docs.kernel.org/scheduler/sched-eevdf.html>. *Linux is always a ‘changing’!*

## 참고문헌

비례 배분 스케줄링 / 영문 원문 13쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

[AC97] "Extending Proportional-Share Scheduling to a Network of Workstations" by Andrea C. Arpaci-Dusseau and David E. Culler. PDPTA'97, June 1997. 이 책의 저자 중 한 명이 쓴 논문으로, 비례 배분 스케줄링을 확장해 클러스터 환경에서 더 잘 동작하도록 하는 방법을 다룬다.

[B+18] "The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS" by J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, J. Sopena. USENIX ATC '18, July 2018, Boston, Massachusetts. Linux CFS와 FreeBSD 스케줄러를 자세히 비교한 최근 연구이다. 각 스케줄러에 대한 훌륭한 개요도 제공한다. 비교 결과는 명확한 결론이 없다는 것이다. 어떤 경우에는 CFS가 더 좋았고 다른 경우에는 BSD 스케줄러인 ULE가 더 좋았다. 인생에는 때로 쉬운 답이 없다.

[B72] "Symmetric binary B-Trees: Data Structure And Maintenance Algorithms" by Rudolf Bayer. Acta Informatica, Volume 1, Number 4, December 1972. 아마도 독자가 태어나기 전에 소개되었을 멋진 균형 트리이다. 존재하는 많은 균형 트리 중 하나이므로 다른 대안을 더 알고 싶으면 알고리즘 책을 공부하자!

[D82] "Why Numbering Should Start At Zero" by Edsger Dijkstra, August 1982. Available: <http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF>. 컴퓨터 과학의 선구자 중 한 명인 E. Dijkstra의 짧은 글이다. 동시성 부분에서 이 사람의 이야기를 더 많이 듣게 될 것이다. 그동안 다음과 같은 동기 유발 문구가 들어 있는 글을 즐겨 보자. '컴퓨터 과학자가 아닌 내 동료 한 명은 몇몇 젊은 컴퓨터 과학자들이 0부터 번호를 매긴다는 이유로 그들이 현학적이라고 비난했다.' 이 글은 왜 그렇게 하는 것이 논리적인지 설명한다.

[K+15] "Profiling A Warehouse-scale Computer" by S. Kanev, P. Ranganathan, J. P. Darago, K. Hazelwood, T. Moseley, G. Wei, D. Brooks. ISCA '15, June, 2015, Portland, Oregon. 점점 더 많은 컴퓨팅이 이루어지는 현대 데이터센터에서 CPU 사이클이 어디에 쓰이는지에 관한 흥미로운 연구이다. CPU 시간의 거의 20%가 운영체제에서 사용되고 스케줄러만 해도 5%를 사용한다!

[J09] "Inside The Linux 2.6 Completely Fair Scheduler" by M. Tim Jones. December 15, 2009. <https://pages.cs.wisc.edu/%7Eremzi/OSTEP/Citations/inside-cfs.pdf>. 초기 CFS의 간단한 개요이다. CFS는 Ingo Molnar가 짧은 창작의 집중 기간에 만든 것으로, 62시간 만에 100K 크기의 커널 패치로 개발되었다.

[KL88] "A Fair Share Scheduler" by J. Kay and P. Lauder. CACM, Volume 31 Issue 1, January 1988. 공정 배분 스케줄링에 관한 초기 참고문헌이다.

[SA95] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" by Ion Stoica and Hussein M. Abdel-Wahab. Technical report, CS Department, Old Dominion University, TR-95-22. EEVDF를 소개한 기술 보고서이다. Ion Stoica는 이후 Apache Spark, Databricks 창립 등을 통해 시스템 분야에 큰 영향을 주었다.

[WW94] "Lottery Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger and William E. Weihl. OSDI '94, November 1994. 시스템 연구 공동체가 스케줄링, 공정 배분, 단순한 무작위 알고리즘의 힘에 다시 관심을 갖게 한 주춤 스케줄링의 기념비적인 논문이다.

[W95] "Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger. Ph.D. Thesis, MIT, 1995. 주춤 및 스트라이드 스케줄링을 설명하는 Waldspurger의 수상 경력 있는 학위논문이다. 언젠가 박사학위 논문을 쓸 생각이라면 목표로 삼을 좋은 본보기가 될 것이다. 이 논문이 바로 그런 훌륭한 예이다.

[W02] "Memory Resource Management in VMware ESX Server" by Carl A. Waldspurger. OSDI '02, Boston, Massachusetts. VMM, 즉 하이퍼바이저의 메모리 관리를 알아보려면 읽어야 할 논문이다. 비교적 읽기 쉬우면서도 이러한 새로운 VMM 수준 메모리 관리에 관한 멋진 아이디어를 많이 담고 있다.

[Z23] "The EEVDF Scheduler" by Peter Zijlstra. EEVDF의 자세한 내용은 다음 주소를 참고하자: <https://docs.kernel.org/scheduler/sched-eevdf.html>. Linux는 언제나 변하고 있다!

## Homework (Simulation)

This program, `lottery.py`, allows you to see how a lottery scheduler works. See the README for details.

## Questions

1. Compute the solutions for simulations with 3 jobs and random seeds of 1, 2, and 3.
2. Now run with two specific jobs: each of length 10, but one (job 0) with 1 ticket and the other (job 1) with 100 (e.g., `-l 10:1, 10:100`). What happens when the number of tickets is so imbalanced? Will job 0 ever run before job 1 completes? How often? In general, what does such a ticket imbalance do to the behavior of lottery scheduling?
3. When running with two jobs of length 100 and equal ticket allocations of 100 (`-l 100:100, 100:100`), how unfair is the scheduler? Run with some different random seeds to determine the (probabilistic) answer; let unfairness be determined by how much earlier one job finishes than the other.
4. How does your answer to the previous question change as the quantum size (`-q`) gets larger?
5. Can you make a version of the graph that is found in the chapter? What else would be worth exploring? How would the graph look with a stride scheduler?

## 숙제: 시뮬레이션

비례 배분 스케줄링 / 영문 원문 14쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

lottery.py 프로그램으로 주춤 스케줄러가 어떻게 동작하는지 살펴볼 수 있다. 자세한 내용은 README를 참고하자.

### 문제

- 작업이 3개이고 난수 시드가 각각 1, 2, 3인 시뮬레이션의 답을 계산하라.
- 이제 두 작업을 구체적으로 지정하여 실행하라. 두 작업의 길이는 각각 10이지만, 작업 0의 티켓은 1장이고 작업 1의 티켓은 100장이다. 예를 들어 -i 10:1,10:100 을 사용한다. 티켓 수가 이렇게 불균형하면 어떤 일이 일어나는가? 작업 1이 완료되기 전에 작업 0이 실행되기는 하는가? 얼마나 자주 실행되는가? 일반적으로 이러한 티켓 불균형은 주춤 스케줄링의 동작에 어떤 영향을 주는가?
- 길이가 각각 100이고 티켓도 각각 100장으로 똑같이 할당된 두 작업을 실행할 때(-i 100:100,100:100), 스케줄러는 얼마나 불공정한가? 여러 난수 시드로 실행하여 확률적인 답을 구하라. 한 작업이 다른 작업보다 얼마나 일찍 끝나는지로 불공정성을 판단하라.
- 퀀텀 크기(-q)가 커질수록 앞 문제의 답은 어떻게 달라지는가?
- 이 장에 나온 것과 같은 그래프를 만들 수 있는가? 그 밖에 무엇을 탐구해 볼 만한가? 스트라이드 스케줄러라면 그래프가 어떻게 나타날까?

## Multiprocessor Scheduling (Advanced)

This chapter will introduce the basics of **multiprocessor scheduling**. As this topic is relatively advanced, it may be best to cover it *after* you have studied the topic of concurrency in some detail (i.e., the second major “easy piece” of the book).

After years of existence only in the high-end of the computing spectrum, **multiprocessor** systems are increasingly commonplace, and have found their way into desktop machines, laptops, and even mobile devices. The rise of the **multicore** processor, in which multiple CPU cores are packed onto a single chip, is the source of this proliferation; these chips have become popular as computer architects have had a difficult time making a single CPU much faster without using (way) too much power. And thus we all now have a few CPUs available to us, which is a good thing, right?

Of course, there are many difficulties that arise with the arrival of more than a single CPU. A primary one is that a typical application (i.e., some C program you wrote) only uses a single CPU; adding more CPUs does not make that single application run faster. To remedy this problem, you’ll have to rewrite your application to run in **parallel**, perhaps using **threads** (as discussed in great detail in the second piece of this book). Multi-threaded applications can spread work across multiple CPUs and thus run faster when given more CPU resources.

### ASIDE: ADVANCED CHAPTERS

Advanced chapters require material from a broad swath of the book to truly understand, while logically fitting into a section that is earlier than said set of prerequisite materials. For example, this chapter on multiprocessor scheduling makes much more sense if you’ve first read the middle piece on concurrency; however, it logically fits into the part of the book on virtualization (generally) and CPU scheduling (specifically). Thus, it is recommended such chapters be covered out of order; in this case, after the second piece of the book.

## 제10장 멀티프로세서 스케줄링 (고급)

멀티프로세서 스케줄링 / 영문 원문 1쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이 장에서는 멀티프로세서 스케줄링의 기초를 소개한다. 비교적 고급 주제이므로, 병행성에 관해 어느 정도 자세히 공부한 뒤에, 즉 이 책의 두 번째 주요 '쉬운 부분'을 공부한 뒤에 다루는 것이 좋다.

여러 해 동안 고성능 컴퓨팅 분야에만 존재하던 멀티프로세서 시스템이 점점 흔해지고 있으며, 이제 데스크톱과 노트북은 물론 모바일 기기에도 사용되고 있다. 여러 CPU 코어를 하나의 칩에 집적한 멀티코어 프로세서의 등장이라는 이러한 확산을 이끌었다. 컴퓨터 설계자들이 전력을 지나치게 많이 사용하지 않고 단일 CPU의 속도를 크게 높이는 데 어려움을 겪으면서, 이런 칩이 널리 쓰이게 되었다. 따라서 이제 우리 모두 몇 개의 CPU를 사용할 수 있게 되었다. 좋은 일 아닌가?

물론 CPU가 하나보다 많아지면 여러 어려움이 생긴다. 가장 중요한 문제 중 하나는 일반적인 응용 프로그램, 예를 들어 여러분이 작성한 C 프로그램이 CPU를 하나만 사용한다는 것이다. CPU를 더 추가해도 그 응용 프로그램 하나가 더 빨리 실행되지는 않는다. 이 문제를 해결하려면 응용 프로그램을 병렬로 실행되도록 다시 작성해야 한다. 이를 위해 스레드를 사용할 수 있으며, 스레드는 이 책의 두 번째 부분에서 아주 자세히 다룬다. 멀티스레드 응용 프로그램은 작업을 여러 CPU에 분산할 수 있으므로 더 많은 CPU 자원을 제공받으면 더 빨리 실행될 수 있다.

### 보충 설명: 고급 장

고급 장을 제대로 이해하려면 이 책의 여러 부분에 걸친 내용이 필요하지만, 논리적인 구성상 그러한 선수 지식보다 앞선 부분에 들어간다. 예를 들어 멀티프로세서 스케줄링을 다루는 이 장은 책의 중간 부분인 병행성을 먼저 읽으면 훨씬 이해하기 쉽다. 그러나 논리적으로는 책의 가상화 부분, 더 구체적으로는 CPU 스케줄링 부분에 속한다. 따라서 이런 장은 책의 순서와 다르게 공부하는 것을 권장한다. 이 장의 경우 책의 두 번째 부분을 마친 뒤에 읽는 것이 좋다.

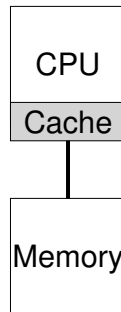


Figure 10.1: **Single CPU With Cache**

Beyond applications, a new problem that arises for the operating system is (not surprisingly!) that of **multiprocessor scheduling**. Thus far we've discussed a number of principles behind single-processor scheduling; how can we extend those ideas to work on multiple CPUs? What new problems must we overcome? And thus, our problem:

#### CRUX: HOW TO SCHEDULE JOBS ON MULTIPLE CPUS

How should the OS schedule jobs on multiple CPUs? What new problems arise? Do the same old techniques work, or are new ideas required?

## 10.1 Background: Multiprocessor Architecture

To understand the new issues surrounding multiprocessor scheduling, we have to understand a new and fundamental difference between single-CPU hardware and multi-CPU hardware. This difference centers around the use of hardware **caches** (e.g., Figure 10.1), and exactly how data is shared across multiple processors. We now discuss this issue further, at a high level. Details are available elsewhere [CSG99], in particular in an upper-level or perhaps graduate computer architecture course.

In a system with a single CPU, there are a hierarchy of **hardware caches** that in general help the processor run programs faster. Caches are small, fast memories that (in general) hold copies of *popular* data that is found in the main memory of the system. Main memory, in contrast, holds *all* of the data, but access to this larger memory is slower. By keeping frequently accessed data in a cache, the system can make the large, slow memory appear to be a fast one.

## 문제의 핵심과 멀티프로세서 구조

멀티프로세서 스케줄링 / 영문 원문 2쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 10.1: 캐시가 있는 단일 CPU. 그림 표기: Memory = 메모리, CPU = CPU, Cache = 캐시. 그림은 인접한 영문 원본 페이지를 참조.

응용 프로그램 외에도 운영체제에는 새로운 문제가 생긴다. 예상했겠지만, 바로 멀티프로세서 스케줄링이다. 지금까지 단일 프로세서 스케줄링의 여러 원칙을 논의했다. 이 아이디어들을 여러 CPU에서도 작동하도록 어떻게 확장할 수 있을까? 어떤 새로운 문제를 극복해야 할까? 따라서 우리가 풀어야 할 문제는 다음과 같다.

### 핵심 질문: 여러 CPU에서 작업을 어떻게 스케줄링할 것인가

운영체제는 여러 CPU에서 작업을 어떻게 스케줄링해야 할까? 어떤 새로운 문제가 생길까? 기존 기법들을 그대로 사용할 수 있을까, 아니면 새로운 아이디어가 필요할까?

#### 10.1 배경: 멀티프로세서 구조

멀티프로세서 스케줄링과 관련된 새로운 문제를 이해하려면 단일 CPU 하드웨어와 다중 CPU 하드웨어 사이의 새롭고 근본적인 차이를 이해해야 한다. 이 차이는 하드웨어 캐시의 사용, 예를 들어 그림 10.1과 같은 캐시의 사용, 그리고 여러 프로세서가 데이터를 공유하는 정확한 방식에 있다. 이제 이 문제를 개략적으로 더 살펴보자. 자세한 내용은 다른 자료 [CSG99]에서, 특히 학부 고학년 또는 대학원 컴퓨터 구조 수업에서 배울 수 있다.

단일 CPU 시스템에는 일반적으로 프로세서가 프로그램을 더 빨리 실행하도록 돕는 계층적인 하드웨어 캐시가 있다. 캐시는 시스템의 주 메모리에 있는 자주 사용되는 데이터의 복사본을 담은 작고 빠른 메모리이다. 이에 비해 주 메모리는 모든 데이터를 담지만, 용량이 큰 만큼 접근 속도가 느리다. 자주 접근하는 데이터를 캐시에 보관하면 시스템은 크고 느린 메모리가 빠른 메모리처럼 보이게 할 수 있다.

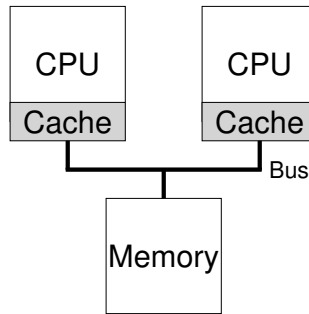


Figure 10.2: Two CPUs With Caches Sharing Memory

As an example, consider a program that issues an explicit load instruction to fetch a value from memory, and a simple system with only a single CPU; the CPU has a small cache (say 64 KB) and a large main memory. The first time a program issues this load, the data resides in main memory, and thus takes a long time to fetch (perhaps in the tens of nanoseconds, or even hundreds). The processor, anticipating that the data may be reused, puts a copy of the loaded data into the CPU cache. If the program later fetches this same data item again, the CPU first checks for it in the cache; if it finds it there, the data is fetched much more quickly (say, just a few nanoseconds), and thus the program runs faster.

Caches are thus based on the notion of **locality**, of which there are two kinds: **temporal locality** and **spatial locality**. The idea behind temporal locality is that when a piece of data is accessed, it is likely to be accessed again in the near future; imagine variables or even instructions themselves being accessed over and over again in a loop. The idea behind spatial locality is that if a program accesses a data item at address  $x$ , it is likely to access data items near  $x$  as well; here, think of a program streaming through an array, or instructions being executed one after the other. Because locality of these types exist in many programs, hardware systems can make good guesses about which data to put in a cache and thus work well.

Now for the tricky part: what happens when you have multiple processors in a single system, with a single shared main memory, as we see in Figure 10.2?

As it turns out, caching with multiple CPUs is much more complicated. Imagine, for example, that a program running on CPU 1 reads a data item (with value  $D$ ) at address  $A$ ; because the data is not in the cache on CPU 1, the system fetches it from main memory, and gets the

## 캐시, 지역성, 공유 메모리

멀티프로세서 스케줄링 / 영문 원문 3쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그림 10.2: 캐시를 갖추고 메모리를 공유하는 두 CPU. 그림 표기: Memory = 메모리, CPU = CPU, Cache = 캐시, Bus = 버스. 그림은 인접한 영문 원본 페이지를 참조.

예를 들어 메모리에서 값을 가져오기 위해 명시적인 load 명령을 실행하는 프로그램과, CPU가 하나뿐인 단순한 시스템을 생각해 보자. 이 CPU에는 작은 캐시, 예컨대 64KB 캐시와 큰 주 메모리가 있다. 프로그램이 이 load 명령을 처음 실행할 때 데이터는 주 메모리에 있으므로 이를 가져오는 데 오래 걸린다. 수십 나노초, 심하면 수백 나노초가 걸릴 수 있다. 프로세서는 이 데이터가 다시 사용될 것으로 예상하여 가져온 데이터의 복사본을 CPU 캐시에 넣는다. 프로그램이 나중에 같은 데이터를 다시 가져오면 CPU는 먼저 캐시에 그 데이터가 있는지 확인한다. 캐시에 있으면 데이터를 훨씬 빠르게, 예컨대 몇 나노초 만에 가져올 수 있으므로 프로그램이 더 빨리 실행된다.

따라서 캐시는 지역성이라는 개념에 바탕을 두며, 지역성에는 시간 지역성과 공간 지역성이라는 두 종류가 있다. 시간 지역성은 어떤 데이터에 접근하면 가까운 미래에 다시 접근할 가능성이 높다는 생각이다. 반복문 안에서 변수나 명령어 자체에 계속해서 접근하는 경우를 떠올려 보자. 공간 지역성은 프로그램이 주소 x의 데이터에 접근하면 x 근처의 데이터에도 접근할 가능성이 높다는 생각이다. 배열을 순서대로 읽는 프로그램이나 명령어를 하나씩 순서대로 실행하는 경우를 생각하면 된다. 많은 프로그램에서 이런 두 종류의 지역성이 나타나므로, 하드웨어 시스템은 어떤 데이터를 캐시에 넣을지 잘 추측할 수 있고 그 결과 효과적으로 작동한다.

이제 까다로운 부분이다. 그림 10.2처럼 하나의 시스템에 여러 프로세서가 있고 이들이 하나의 주 메모리를 공유하면 어떤 일이 일어날까?

여러 CPU가 있을 때의 캐싱은 훨씬 복잡하다. 예를 들어 CPU 1에서 실행되는 프로그램이 주소 A의 데이터, 즉 값 D를 읽는다고 하자. CPU 1의 캐시에 그 데이터가 없으므로 시스템은 주 메모리에서 이를 가져와...

value  $D$ . The program then modifies the value at address  $A$ , just updating its cache with the new value  $D'$ ; writing the data through all the way to main memory is slow, so the system will (usually) do that later. Then assume the OS decides to stop running the program and move it to CPU 2. The program then re-reads the value at address  $A$ ; there is no such data in CPU 2's cache, and thus the system fetches the value from main memory, and gets the old value  $D$  instead of the correct value  $D'$ . Oops!

This general problem is called the problem of **cache coherence**, and there is a vast research literature that describes many different subtleties involved with solving the problem [SHW11]. Here, we will skip all of the nuance and make some major points; take a computer architecture class (or three) to learn more.

The basic solution is provided by the hardware: by monitoring memory accesses, hardware can ensure that basically the “right thing” happens and that the view of a single shared memory is preserved. One way to do this on a bus-based system (as described above) is to use an old technique known as **bus snooping** [G83]; each cache pays attention to memory updates by observing the bus that connects them to main memory. When a CPU then sees an update for a data item it holds in its cache, it will notice the change and either **invalidate** its copy (i.e., remove it from its own cache) or **update** it (i.e., put the new value into its cache too). Write-back caches, as hinted at above, make this more complicated (because the write to main memory isn't visible until later), but you can imagine how the basic scheme might work.

## 10.2 Don't Forget Synchronization

Given that the caches do all of this work to provide coherence, do programs (or the OS itself) have to worry about anything when they access shared data? The answer, unfortunately, is yes, and is documented in great detail in the second piece of this book on the topic of concurrency. While we won't get into the details here, we'll sketch/review some of the basic ideas here (assuming you're familiar with concurrency).

When accessing (and in particular, updating) shared data items or structures across CPUs, mutual exclusion primitives (such as locks) should likely be used to guarantee correctness (other approaches, such as building **lock-free** data structures, are complex and only used on occasion; see the chapter on deadlock in the piece on concurrency for details). For example, assume we have a shared queue being accessed on multiple CPUs concurrently. Without locks, adding or removing elements from the queue concurrently will not work as expected, even with the underlying coherence protocols; one needs locks to atomically update the data structure to its new state.

To make this more concrete, imagine this code sequence, which is used to remove an element from a shared linked list, as we see in Figure 10.3. Imagine if threads on two CPUs enter this routine at the same time. If

## 캐시 일관성과 동기화

멀티프로세서 스케줄링 / 영문 원문 4쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

값 D를 얻는다. 그런 다음 프로그램은 주소 A의 값을 수정하는데, 캐시에만 새로운 값 D'를 반영한다. 데이터를 주 메모리까지 기록하는 일은 느리므로 시스템은 보통 나중에 수행한다. 이어서 운영체제가 프로그램 실행을 멈추고 이를 CPU 2로 옮기기로 했다고 하자. 프로그램이 주소 A의 값을 다시 읽으면 CPU 2의 캐시에는 해당 데이터가 없다. 따라서 시스템은 주 메모리에서 값을 가져오지만, 올바른 값 D' 대신 예전 값 D를 얻게 된다. 이런!

이 일반적인 문제를 캐시 일관성 문제라고 부른다. 이 문제를 해결할 때 고려해야 하는 여러 미묘한 사항을 설명하는 방대한 연구 문헌이 있다 [SHW11]. 여기서는 세부적인 사항을 생략하고 주요 내용만 짚는다. 더 배우고 싶다면 컴퓨터 구조 수업을 한 과목, 아니 세 과목쯤 들어 보자.

기본적인 해결책은 하드웨어가 제공한다. 하드웨어는 메모리 접근을 감시하여 기본적으로 '올바른 동작'이 일어나고 하나의 공유 메모리가 있다는 관점이 유지되도록 보장할 수 있다. 앞에서 설명한 버스 기반 시스템에서는 버스 스누핑이라는 오래된 기법을 사용할 수 있다 [G83]. 각 캐시는 자신을 주 메모리에 연결하는 버스를 관찰하여 메모리 갱신을 감시한다. 어떤 CPU가 자신의 캐시에 보관한 데이터의 갱신을 발견하면 그 변화를 알아차리고 자신의 복사본을 무효화하거나, 즉 자신의 캐시에서 제거하거나, 갱신한다. 갱신한다는 것은 새로운 값을 자신의 캐시에도 넣는다는 뜻이다. 앞서 암시했듯이 쓰기 지연 캐시는 주 메모리에 대한 쓰기가 나중에까지 보이지 않으므로 이 과정을 더 복잡하게 만든다. 그래도 기본적인 방식이 어떻게 작동할지는 짐작할 수 있을 것이다.

### 10.2 동기화를 잊지 말자

캐시가 일관성을 제공하기 위해 이렇게 많은 일을 한다면, 프로그램이나 운영체제 자체는 공유 데이터에 접근할 때 신경 쓸 일이 있을까? 안타깝지만 그렇다. 이 내용은 병행성을 다루는 이 책의 두 번째 부분에 아주 자세히 설명되어 있다. 여기서는 세부 사항에 들어가지 않고, 독자가 병행성을 알고 있다고 가정하여 몇 가지 기본 개념을 간단히 설명하거나 복습한다.

여러 CPU에서 공유 데이터 항목이나 자료구조에 접근할 때, 특히 이를 갱신할 때는 정확성을 보장하기 위해 락과 같은 상호 배제 기본 연산을 사용하는 것이 일반적이다. 락 없는 자료구조를 만드는 등의 다른 접근법도 있지만 복잡하며 제한적인 경우에만 사용된다. 자세한 내용은 병행성 부분의 교착 상태 장을 참조하자. 예를 들어 여러 CPU가 하나의 공유 큐에 동시에 접근한다고 하자. 락이 없으면 기본적인 일관성 프로토콜이 있더라도 큐에 원소를 동시에 추가하거나 제거하는 작업은 예상대로 동작하지 않는다. 자료구조를 새로운 상태로 원자적으로 갱신하려면 락이 필요하다.

더 구체적으로 살펴보기 위해 그림 10.3처럼 공유 연결 리스트에서 원소를 제거하는 코드를 생각해 보자. 두 CPU의 스레드가 동시에 이 루틴에 진입한다고 상상해 보자.

```

1  typedef struct __Node_t {
2      int          value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;          // remember old head
8      int value  = head->value;    // ... and its value
9      head      = head->next;      // advance to next
10     free(tmp);                   // free old head
11     return value;                // return value @head
12 }

```

Figure 10.3: Simple List Delete Code

Thread 1 executes the first line, it will have the current value of `head` stored in its `tmp` variable; if Thread 2 then executes the first line as well, it also will have the same value of `head` stored in its own private `tmp` variable (`tmp` is allocated on the stack, and thus each thread will have its own private storage for it). Thus, instead of each thread removing an element from the head of the list, each thread will try to remove the same head element, leading to all sorts of problems (such as an attempted double free of the head element at Line 10, as well as potentially returning the same data value twice).

The solution, of course, is to make such routines correct via **locking**. In this case, allocating a simple mutex (e.g., `pthread_mutex_t m;`) and then adding a `lock(&m)` at the beginning of the routine and an `unlock(&m)` at the end will solve the problem, ensuring that the code will execute as desired. Unfortunately, as we will see, such an approach is not without problems, in particular with regards to performance. Specifically, as the number of CPUs grows, access to a synchronized shared data structure becomes quite slow.

### 10.3 One Final Issue: Cache Affinity

One final issue arises in building a multiprocessor cache scheduler, known as **cache affinity** [TTG95]. This notion is simple: a process, when run on a particular CPU, builds up a fair bit of state in the caches (and TLBs) of the CPU. The next time the process runs, it is often advantageous to run it on the same CPU, as it will run faster if some of its state is already present in the caches on that CPU. If, instead, one runs a process on a different CPU each time, the performance of the process will be worse, as it will have to reload the state each time it runs (note it will run correctly on a different CPU thanks to the cache coherence protocols of the hardware). Thus, a multiprocessor scheduler should consider cache affinity when making its scheduling decisions, perhaps preferring to keep a process on the same CPU if at all possible.

## 공유 리스트와 캐시 친화성

멀티프로세서 스케줄링 / 영문 원문 5쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

```

1 typedef struct __Node_t {
2     int value;
3     struct __Node_t *next;
4 } Node_t;
5
6 int List_Pop() {
7     Node_t *tmp = head;    // 기존 head를 기억한다
8     int value = head->value; // 그 값도 기억한다
9     head = head->next;    // 다음 원소로 이동한다
10    free(tmp);            // 기존 head를 해제한다
11    return value;          // head에 있던 값을 반환한다
12 }
```

그림 10.3: 간단한 리스트 삭제 코드.

스레드 1이 첫 번째 문장을 실행하면 현재 head 값이 자신의 tmp 변수에 저장된다. 이어서 스레드 2도 첫 번째 문장을 실행하면 자신의 전용 tmp 변수에 같은 head 값이 저장된다. tmp는 스택에 할당되므로 각 스레드는 이를 위한 별도의 저장 공간을 갖는다. 따라서 각 스레드가 리스트의 머리에서 원소를 하나씩 제거하는 대신, 두 스레드 모두 같은 머리 원소를 제거하려고 한다. 그 결과 10행에서 머리 원소를 두 번 해제하려 하거나 같은 데이터 값을 두 번 반환할 수 있는 등 여러 문제가 생긴다.

물론 해결책은 락을 사용하여 이러한 루틴이 올바르게 동작하도록 만드는 것이다. 여기서는 단순한 뮤텍스, 예를 들어 pthread\_mutex\_t m;을 할당하고 루틴의 시작 부분에 lock(&m)을, 끝부분에 unlock(&m)을 추가하면 문제가 해결되어 코드가 의도한 대로 실행된다. 안타깝게도 앞으로 보겠지만 이 접근법에도 문제가 있으며, 특히 성능과 관련된 문제가 있다. 구체적으로 CPU 수가 늘어나면 동기화된 공유 자료구조에 접근하는 속도가 상당히 느려진다.

### 10.3 마지막 문제: 캐시 친화성

멀티프로세서용 캐시를 고려하는 스케줄러를 만들 때는 캐시 친화성이라는 마지막 문제가 생긴다 [TTG95]. 개념은 간단하다. 어떤 프로세스가 특정 CPU에서 실행되면 그 CPU의 캐시와 TLB에 상당한 상태가 쌓인다. 다음에 이 프로세스가 실행될 때도 같은 CPU에서 실행하는 것이 유리한 경우가 많다. 그 CPU의 캐시에 상태의 일부가 이미 남아 있으면 더 빨리 실행되기 때문이다. 반대로 매번 다른 CPU에서 프로세스를 실행하면 실행할 때마다 상태를 다시 불러와야 하므로 성능이 떨어진다. 하드웨어의 캐시 일관성 프로토콜 덕분에 다른 CPU에서도 올바르게 실행되기는 한다. 따라서 멀티프로세서 스케줄러는 스케줄링을 결정할 때 캐시 친화성을 고려해야 하며, 가능하다면 프로세스를 같은 CPU에 유지하는 쪽을 선호할 수 있다.

10.4 Single-Queue Scheduling

With this background in place, we now discuss how to build a scheduler for a multiprocessor system. The most basic approach is to simply reuse the basic framework for single processor scheduling, by putting all jobs that need to be scheduled into a single queue; we call this **single-queue multiprocessor scheduling** or **SQMS** for short. This approach has the advantage of simplicity; it does not require much work to take an existing policy that picks the best job to run next and adapt it to work on more than one CPU (where it might pick the best two jobs to run, if there are two CPUs, for example).

However, SQMS has obvious shortcomings. The first problem is a lack of **scalability**. To ensure the scheduler works correctly on multiple CPUs, the developers will have inserted some form of **locking** into the code, as described above. Locks ensure that when SQMS code accesses the single queue (say, to find the next job to run), the proper outcome arises.

Locks, unfortunately, can greatly reduce performance, particularly as the number of CPUs in the systems grows [A90]. As contention for such a single lock increases, the system spends more and more time in lock overhead and less time doing the work the system should be doing (note: it would be great to include a real measurement of this in here someday).

The second main problem with SQMS is cache affinity. For example, let us assume we have five jobs to run (*A, B, C, D, E*) and four processors. Our scheduling queue thus looks like this:



Over time, assuming each job runs for a time slice and then another job is chosen, here is a possible job schedule across CPUs:

|       |   |   |   |   |   |                  |
|-------|---|---|---|---|---|------------------|
| CPU 0 | A | E | D | C | B | ... (repeat) ... |
| CPU 1 | B | A | E | D | C | ... (repeat) ... |
| CPU 2 | C | B | A | E | D | ... (repeat) ... |
| CPU 3 | D | C | B | A | E | ... (repeat) ... |

Because each CPU simply picks the next job to run from the globally-shared queue, each job ends up bouncing around from CPU to CPU, thus doing exactly the opposite of what would make sense from the standpoint of cache affinity.

To handle this problem, most SQMS schedulers include some kind of affinity mechanism to try to make it more likely that process will continue

## 10.4 단일 큐 스케줄링

멀티프로세서 스케줄링 / 영문 원문 6쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이러한 배경을 바탕으로 이제 멀티프로세서 시스템용 스케줄러를 어떻게 만들지 살펴보자. 가장 기본적인 접근법은 스케줄링해야 할 모든 작업을 하나의 큐에 넣어 단일 프로세서 스케줄링의 기본 틀을 그대로 재사용하는 것이다. 이를 단일 큐 멀티프로세서 스케줄링, 줄여서 SQMS라고 한다. 이 접근법의 장점은 단순함이다. 다음에 실행할 가장 적절한 작업을 선택하는 기존 정책을 가져와 여러 CPU에서 작동하도록 바꾸는 데는 많은 작업이 필요하지 않다. 예를 들어 CPU가 두 개라면 실행할 가장 적절한 작업 두 개를 고르면 된다.

하지만 SQMS에는 뚜렷한 단점이 있다. 첫 번째 문제는 확장성 부족이다. 스케줄러가 여러 CPU에서 올바르게 작동하도록 개발자는 앞서 설명한 것처럼 코드에 어떤 형태로든 락을 넣어야 한다. 락은 SQMS 코드가 다음에 실행할 작업을 찾는 등의 목적으로 단일 큐에 접근할 때 올바른 결과를 보장한다.

안타깝게도 락은 성능을 크게 떨어뜨릴 수 있으며, 특히 시스템의 CPU 수가 늘어날수록 그렇다 [A90]. 하나의 락을 둘러싼 경합이 증가하면 시스템은 락에 드는 부가 비용에 점점 더 많은 시간을 쓰고, 실제로 해야 할 작업에는 더 적은 시간을 쓰게 된다. 참고로, 언젠가 이 부분에 실제 측정 결과를 넣으면 좋겠다.

SQMS의 두 번째 주요 문제는 캐시 친화성이다. 예를 들어 실행할 작업이 다섯 개, A, B, C, D, E이고 프로세서는 네 개라고 하자. 스케줄링 큐는 다음과 같다.

큐 → A → B → C → D → E → NULL

각 작업이 한 타임 슬라이스 동안 실행된 다음 다른 작업이 선택된다고 가정하면, 시간이 흐름에 따라 CPU별로 다음과 같은 스케줄이 만들어질 수 있다.

| CPU   | 시간 순서에 따른 작업                 |
|-------|------------------------------|
| CPU 0 | A → E → D → C → B → ... (반복) |
| CPU 1 | B → A → E → D → C → ... (반복) |
| CPU 2 | C → B → A → E → D → ... (반복) |
| CPU 3 | D → C → B → A → E → ... (반복) |

각 CPU가 전역 공유 큐에서 단순히 다음 작업을 선택하므로 각 작업은 CPU 사이를 계속 옮겨 다니게 된다. 이는 캐시 친화성의 관점에서 바람직하지 않은 동작과 정반대이다.

이 문제를 해결하기 위해 대부분의 SQMS 스케줄러는 가능하면 프로세스가 같은 CPU에서 계속 실행될 가능성을 높이려는 어떤 형태의 친화성 기법을 포함한다.

to run on the same CPU if possible. Specifically, one might provide affinity for some jobs, but move others around to balance load. For example, imagine the same five jobs scheduled as follows:

|       |   |   |   |   |   |                  |
|-------|---|---|---|---|---|------------------|
| CPU 0 | A | E | A | A | A | ... (repeat) ... |
| CPU 1 | B | B | E | B | B | ... (repeat) ... |
| CPU 2 | C | C | C | E | C | ... (repeat) ... |
| CPU 3 | D | D | D | D | E | ... (repeat) ... |

In this arrangement, jobs *A* through *D* are not moved across processors, with only job *E* **migrating** from CPU to CPU, thus preserving affinity for most. You could then decide to migrate a different job the next time through, thus achieving some kind of affinity fairness as well. Implementing such a scheme, however, can be complex.

Thus, we can see the SQMS approach has its strengths and weaknesses. It is straightforward to implement given an existing single-CPU scheduler, which by definition has only a single queue. However, it does not scale well (due to synchronization overheads), and it does not readily preserve cache affinity.

10.5 Multi-Queue Scheduling

Because of the problems caused in single-queue schedulers, some systems opt for multiple queues, e.g., one per CPU. We call this approach **multi-queue multiprocessor scheduling** (or **MQMS**).

In MQMS, our basic scheduling framework consists of multiple scheduling queues. Each queue will likely follow a particular scheduling discipline, such as round robin, though of course any algorithm can be used. When a job enters the system, it is placed on exactly one scheduling queue, according to some heuristic (e.g., random, or picking one with fewer jobs than others). Then it is scheduled essentially independently, thus avoiding the problems of information sharing and synchronization found in the single-queue approach.

For example, assume we have a system where there are just two CPUs (labeled CPU 0 and CPU 1), and some number of jobs enter the system: *A*, *B*, *C*, and *D* for example. Given that each CPU has a scheduling queue now, the OS has to decide into which queue to place each job. It might do something like this:



# 친화성 유지와 다중 큐 스케줄링

멀티프로세서 스케줄링 / 영문 원문 7쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

구체적으로 일부 작업에는 친화성을 제공하면서, 부하의 균형을 맞추기 위해 다른 작업들은 이동시킬 수 있다. 예를 들어 같은 다섯 작업을 다음처럼 스케줄링한다고 하자.

| CPU   | 시간 순서에 따른 작업                 |
|-------|------------------------------|
| CPU 0 | A → E → A → A → A → ... (반복) |
| CPU 1 | B → B → E → B → B → ... (반복) |
| CPU 2 | C → C → C → E → C → ... (반복) |
| CPU 3 | D → D → D → D → E → ... (반복) |

이 배치에서는 A부터 D까지의 작업은 프로세서 사이를 이동하지 않고 E만 CPU 사이를 이동하므로 대부분 작업의 친화성이 유지된다. 다음번에는 다른 작업을 이동시키기로 하여 친화성 측면에서도 어느 정도 공정성을 달성할 수 있다. 그러나 이러한 방식을 구현하는 것은 복잡할 수 있다.

따라서 SQMS 접근법에는 장단점이 있다. 정의상 큐를 하나만 갖는 기존 단일 CPU 스케줄러가 있다면 구현하기 쉽다. 그러나 동기화에 따른 부가 비용 때문에 확장성이 좋지 않고, 캐시 친화성을 쉽게 유지하지도 못한다.

## 10.5 다중 큐 스케줄링

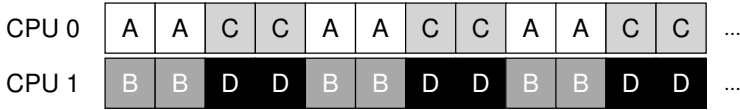
단일 큐 스케줄러에서 발생하는 문제 때문에 어떤 시스템은 여러 큐를 선택한다. 예를 들어 CPU마다 큐를 하나씩 둔다. 이를 다중 큐 멀티프로세서 스케줄링, 줄여서 MQMS라고 한다.

MQMS의 기본 스케줄링 틀은 여러 스케줄링 큐로 구성된다. 각 큐는 보통 라운드 로빈 같은 특정 스케줄링 방식을 따르지만, 물론 어떤 알고리즘이든 사용할 수 있다. 작업이 시스템에 들어오면 임의로 고르거나 다른 큐보다 작업 수가 적은 큐를 고르는 등의 경험적 방법에 따라 정확히 하나의 스케줄링 큐에 배치된다. 이후에는 본질적으로 독립적으로 스케줄링되므로 단일 큐 접근법에서 발생하는 정보 공유와 동기화 문제를 피할 수 있다.

예를 들어 CPU 0과 CPU 1이라는 두 CPU만 있는 시스템에 A, B, C, D 같은 몇 개의 작업이 들어온다고 하자. 이제 각 CPU가 스케줄링 큐를 가지므로 운영체제는 각 작업을 어느 큐에 넣을지 결정해야 한다. 다음과 같이 할 수 있다.

| 큐  | 작업    |
|----|-------|
| Q0 | A → C |
| Q1 | B → D |

Depending on the queue scheduling policy, each CPU now has two jobs to choose from when deciding what should run. For example, with **round robin**, the system might produce a schedule that looks like this:

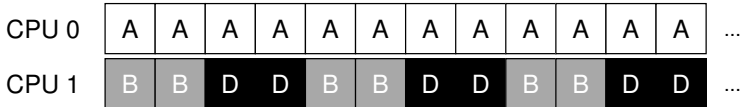


MQMS has a distinct advantage over SQMS in that it should be inherently more scalable. As the number of CPUs grows, so too does the number of queues, and thus lock and cache contention should not become a central problem. In addition, MQMS intrinsically provides cache affinity; jobs stay on the same CPU and thus reap the advantage of reusing cached contents therein.

But, if you've been paying attention, you might see that we have a new problem, which is fundamental in the multi-queue based approach: **load imbalance**. Let's assume we have the same set up as above (four jobs, two CPUs), but then one of the jobs (say *C*) finishes. We now have the following scheduling queues:



If we then run our round-robin policy on each queue of the system, we will see this resulting schedule:



As you can see from this diagram, *A* gets twice as much CPU as *B* and *D*, which is not the desired outcome. Even worse, let's imagine that both *A* and *C* finish, leaving just jobs *B* and *D* in the system. The two scheduling queues, and resulting timeline, will look like this:



How terrible – CPU 0 is idle! (*insert dramatic and sinister music here*) And thus our CPU usage timeline looks quite sad.

# 다중 큐의 장점과 부하 불균형

멀티프로세서 스케줄링 / 영문 원문 8쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이제 각 CPU는 큐의 스케줄링 정책에 따라 두 작업 중 하나를 선택하여 실행한다. 예를 들어 라운드 로빈을 사용하면 시스템은 다음과 같은 스케줄을 만들 수 있다.

| CPU   | 시간 순서에 따른 작업                |
|-------|-----------------------------|
| CPU 0 | A A C C A A C C A A C C ... |
| CPU 1 | B B D D B B D D B B D D ... |

MQMS는 본질적으로 확장성이 더 높다는 점에서 SQMS보다 뚜렷한 장점이 있다. CPU 수가 늘어나면 큐 수도 늘어나므로 락과 캐시 경합이 핵심적인 문제가 되지 않을 것이다. 또한 MQMS는 본래 캐시 친화성을 제공한 다. 작업이 같은 CPU에 머무르므로 그 캐시에 있는 내용을 재사용하는 이점을 얻는다.

하지만 주의 깊게 읽었다면 다중 큐 접근법에 근본적인 새로운 문제가 생긴다는 사실을 눈치챘을 것이다. 바로 부하 불균형이다. 위와 똑같이 작업 네 개와 CPU 두 개가 있지만, 그중 하나의 작업, 예를 들어 C가 끝났다고 하자. 그러면 스케줄링 큐는 다음과 같다.

| 큐  | 작업    |
|----|-------|
| Q0 | A     |
| Q1 | B → D |

시스템의 각 큐에 라운드 로빈 정책을 적용하면 다음과 같은 스케줄이 나온다.

| CPU   | 시간 순서에 따른 작업                |
|-------|-----------------------------|
| CPU 0 | A A A A A A A A A A A ...   |
| CPU 1 | B B D D B B D D B B D D ... |

이 그림에서 보듯 A는 B나 D보다 CPU 시간을 두 배나 많이 받는다. 이는 원하는 결과가 아니다. 더 나쁜 경우로, A와 C가 모두 끝나고 B와 D만 시스템에 남았다고 하자. 두 스케줄링 큐와 그에 따른 시간표는 다음과 같아진다.

| 큐  | 작업    |
|----|-------|
| Q0 | 비어 있음 |
| Q1 | B → D |

| CPU   | 시간 순서에 따른 작업                |
|-------|-----------------------------|
| CPU 0 | 유휴 상태                       |
| CPU 1 | B B D D B B D D B B D D ... |

끔찍하다. CPU 0이 놀고 있다! 여기에 극적이고 불길한 음악을 깔아 보자. 이렇게 CPU 사용 시간표는 매우 안타까운 모습이 된다.

So what should a poor multi-queue multiprocessor scheduler do? How can we overcome the insidious problem of load imbalance and defeat the evil forces of ... the Decepticons<sup>1</sup>? How do we stop asking questions that are hardly relevant to this otherwise wonderful book?

CRUX: HOW TO DEAL WITH LOAD IMBALANCE

How should a multi-queue multiprocessor scheduler handle load imbalance, so as to better achieve its desired scheduling goals?

The obvious answer to this query is to move jobs around, a technique which we (once again) refer to as **migration**. By migrating a job from one CPU to another, true load balance can be achieved.

Let's look at a couple of examples to add some clarity. Once again, we have a situation where one CPU is idle and the other has some jobs.



In this case, the desired migration is easy to understand: the OS should simply move one of *B* or *D* to CPU 0. The result of this single job migration is evenly balanced load and everyone is happy.

A more tricky case arises in our earlier example, where *A* was left alone on CPU 0 and *B* and *D* were alternating on CPU 1:



In this case, a single migration does not solve the problem. What would you do in this case? The answer, alas, is continuous migration of one or more jobs. One possible solution is to keep switching jobs, as we see in the following timeline. In the figure, first *A* is alone on CPU 0, and *B* and *D* alternate on CPU 1. After a few time slices, *B* is moved to compete with *A* on CPU 0, while *D* enjoys a few time slices alone on CPU 1. And thus load is balanced:

|       |   |   |   |   |   |   |   |   |   |   |   |   |     |
|-------|---|---|---|---|---|---|---|---|---|---|---|---|-----|
| CPU 0 | A | A | A | A | B | A | B | A | B | B | B | B | ... |
| CPU 1 | B | D | B | D | D | D | D | D | A | D | A | D | ... |

Of course, many other possible migration patterns exist. But now for the tricky part: how should the system decide to enact such a migration?

<sup>1</sup>Little known fact is that the home planet of Cybertron was destroyed by bad CPU scheduling decisions. And now let that be the first and last reference to Transformers in this book, for which we sincerely apologize.

# 부하 불균형과 작업 이동

멀티프로세서 스케줄링 / 영문 원문 9쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

그렇다면 불쌍한 다중 큐 멀티프로세서 스케줄러는 어떻게 해야 할까? 어떻게 하면 교묘한 부하 불균형 문제를 극복하고 사악한 세력인... 디셉티콘을 물리칠 수 있을까? [각주 1] 다른 부분은 이렇게 훌륭한 이 책에서, 별로 관련도 없는 질문을 그만하려면 어떻게 해야 할까?

## 핵심 질문: 부하 불균형에 어떻게 대처할 것인가

다중 큐 멀티프로세서 스케줄러는 원하는 스케줄링 목표를 더 잘 달성하기 위해 부하 불균형을 어떻게 처리해야 할까?

이 질문의 분명한 답은 작업을 이리저리 옮기는 것이다. 우리는 이 기법을 다시 한 번 이주, 즉 마이그레이션이라고 부른다. 작업을 한 CPU에서 다른 CPU로 옮기면 실질적인 부하 균형을 이룰 수 있다.

몇 가지 예로 더 분명히 살펴보자. 다시 한 CPU는 유휴 상태이고 다른 CPU에는 몇 개의 작업이 있는 상황이다.

| 큐  | 작업    |
|----|-------|
| Q0 | 비어 있음 |
| Q1 | B → D |

이 경우 바람직한 이동은 이해하기 쉽다. 운영체제가 B나 D 중 하나를 CPU 0으로 옮기기만 하면 된다. 이 작업 하나를 옮긴 결과 부하가 고르게 균형을 이루고 모두가 행복해진다.

앞선 예처럼 CPU 0에는 A만 남고 CPU 1에서는 B와 D가 번갈아 실행되는 경우에는 더 까다로운 문제가 생긴다.

| 큐  | 작업    |
|----|-------|
| Q0 | A     |
| Q1 | B → D |

이 경우 한 번의 이동만으로는 문제가 해결되지 않는다. 여러분이라면 어떻게 하겠는가? 안타깝지만 답은 하나 이상의 작업을 계속 이동시키는 것이다. 가능한 해결책 하나는 다음 시간표처럼 작업의 위치를 계속 바꾸는 것이다. 그림에서는 처음에 A가 CPU 0을 혼자 사용하고, B와 D는 CPU 1에서 번갈아 실행된다. 몇 타임 슬라이스가 지난 뒤 B를 CPU 0으로 옮겨 A와 경쟁하게 하고, D는 CPU 1에서 몇 타임 슬라이스를 혼자 누리게 한다. 이렇게 부하의 균형이 맞춰진다.

| CPU   | 시간 순서에 따른 작업              |
|-------|---------------------------|
| CPU 0 | A A A B A B A B B B B ... |
| CPU 1 | B D B D D D D A D A D ... |

물론 이 외에도 가능한 이동 패턴은 많다. 하지만 이제 어려운 부분이다. 시스템은 이러한 이동을 언제 수행할지 어떻게 결정해야 할까?

각주 1: 잘 알려지지 않은 사실이지만, 고향 행성 사이버트론은 잘못된 CPU 스케줄링 결정 때문에 파괴되었다. 이 책에서 트랜스포머를 언급하는 것은 이것이 처음이자 마지막이 되도록 하자. 진심으로 사과드린다.

One basic approach is to use a technique known as **work stealing** [FLR98]. With a work-stealing approach, a (source) queue that is low on jobs will occasionally peek at another (target) queue, to see how full it is. If the target queue is (notably) more full than the source queue, the source will “steal” one or more jobs from the target to help balance load.

Of course, there is a natural tension in such an approach. If you look around at other queues too often, you will suffer from high overhead and have trouble scaling, which was the entire purpose of implementing the multiple queue scheduling in the first place! If, on the other hand, you don’t look at other queues very often, you are in danger of suffering from severe load imbalances. Finding the right threshold remains, as is common in system policy design, a black art.

## 10.6 Linux Multiprocessor Schedulers

Interestingly, in the Linux community, no common solution has emerged to building a multiprocessor scheduler. Over time, three different schedulers arose: the O(1) scheduler, the Completely Fair Scheduler (CFS), and the BF Scheduler (BFS)<sup>2</sup>. See Meehan’s dissertation for an excellent overview of the strengths and weaknesses of said schedulers [M11]; here we just summarize a few of the basics.

Both O(1) and CFS use multiple queues, whereas BFS uses a single queue, showing that both approaches can be successful. Of course, there are many other details which separate these schedulers. For example, the O(1) scheduler is a priority-based scheduler (similar to the MLFQ discussed before), changing a process’s priority over time and then scheduling those with highest priority in order to meet various scheduling objectives; interactivity is a particular focus. CFS, in contrast, is a deterministic proportional-share approach (more like Stride scheduling, as discussed earlier). BFS, the only single-queue approach among the three, is also proportional-share, but based on a more complicated scheme known as Earliest Eligible Virtual Deadline First (EEVDF) [SA96]. Read more about these modern algorithms on your own; you should be able to understand how they work now!

## 10.7 Summary

We have seen various approaches to multiprocessor scheduling. The single-queue approach (SQMS) is rather straightforward to build and balances load well but inherently has difficulty with scaling to many processors and cache affinity. The multiple-queue approach (MQMS) scales better and handles cache affinity well, but has trouble with load imbalance and is more complicated. Whichever approach you take, there is no simple answer: building a general purpose scheduler remains a daunting task, as small code changes can lead to large behavioral differences. Only undertake such an exercise if you know exactly what you are doing, or, at least, are getting paid a large amount of money to do so.

---

<sup>2</sup>Look up what BF stands for on your own; be forewarned, it is not for the faint of heart.

## 작업 훔치기, Linux 스케줄러, 정리

멀티프로세서 스케줄링 / 영문 원문 10쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

기본적인 접근법 하나는 작업 훔치기라는 기법을 사용하는 것이다 [FLR98]. 작업 훔치기에서는 작업이 부족한 출발 큐가 가끔 다른 대상 큐를 들여다보며 얼마나 차 있는지 확인한다. 대상 큐가 출발 큐보다 눈에 띄게 더 많이 차 있으면 출발 큐는 대상 큐에서 하나 이상의 작업을 '훔쳐' 부하 균형을 맞춘다.

물론 이 접근법에는 자연스러운 상충 관계가 있다. 다른 큐를 너무 자주 살펴보면 부가 비용이 커지고 확장성이 나빠진다. 애초에 다중 큐 스케줄링을 구현한 목적이 바로 확장성이었는데 말이다! 반대로 다른 큐를 충분히 자주 확인하지 않으면 심각한 부하 불균형을 겪을 위험이 있다. 적절한 임계값을 찾는 일은 시스템 정책 설계에서 흔히 그렇듯 여전히 경험과 요령이 필요한 기술이다.

### 10.6 Linux 멀티프로세서 스케줄러

흥미롭게도 Linux 커뮤니티에서는 멀티프로세서 스케줄러를 만드는 데 공통된 해결책 하나가 자리 잡지 못했다. 시간이 흐르면서  $O(1)$  스케줄러, 완전 공정 스케줄러(CFS), BF 스케줄러(BFS)라는 세 가지 서로 다른 스케줄러가 등장했다. [각주 2] 이 스케줄러들의 장단점을 훌륭하게 정리한 자료는 Meehan의 학위 논문 [M11]을 참조하자. 여기서는 몇 가지 기본 사항만 요약한다.

$O(1)$ 과 CFS는 모두 다중 큐를 사용하고 BFS는 단일 큐를 사용한다. 이는 두 접근법 모두 성공할 수 있음을 보여 준다. 물론 이 스케줄러들은 다른 세부 사항에서도 여러 차이가 있다. 예를 들어  $O(1)$  스케줄러는 앞서 다룬 MLFQ와 비슷한 우선순위 기반 스케줄러이다. 시간에 따라 프로세스의 우선순위를 바꾸고 가장 높은 우선순위를 가진 프로세스부터 스케줄링하여 여러 스케줄링 목표를 달성하려 하며, 특히 상호작용성을 중시한다. 반면 CFS는 앞서 설명한 스트라이드 스케줄링에 더 가까운 결정론적 비례 배분 방식이다. 세 가지 중 유일한 단일 큐 방식인 BFS도 비례 배분 방식이지만, Earliest Eligible Virtual Deadline First(EEVDF)라는 더 복잡한 방식에 기반한다 [SA96]. 이 현대적인 알고리즘들은 스스로 더 읽어 보자. 이제 어떻게 작동하는지 이해할 수 있을 것이다!

### 10.7 요약

멀티프로세서 스케줄링의 여러 접근법을 살펴보았다. 단일 큐 접근법(SQMS)은 비교적 만들기 쉽고 부하 균형도 잘 맞추지만, 본질적으로 많은 프로세서로 확장하거나 캐시 친화성을 유지하는 데 어려움이 있다. 다중 큐 접근법(MQMS)은 확장성이 더 좋고 캐시 친화성도 잘 처리하지만, 부하 불균형 문제가 있으며 더 복잡하다. 어떤 접근법을 택하든 간단한 정답은 없다. 작은 코드 변경이 동작의 큰 차이로 이어질 수 있으므로 범용 스케줄러를 만드는 일은 여전히 만만치 않다. 자신이 무엇을 하는지 정확히 알거나, 적어도 그 일을 하는 대가로 많은 돈을 받는 경우에만 이런 작업에 도전하자.

각주 2: BF가 무엇의 약자인지는 직접 찾아보자. 미리 경고하지만, 마음이 약한 사람에게는 권하지 않는다.

## References

- [A90] “The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors” by Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. *A classic paper on how different locking alternatives do and don't scale. By Tom Anderson, very well known researcher in both systems and networking. And author of a very fine OS textbook, we must say.*
- [B+10] “An Analysis of Linux Scalability to Many Cores Abstract” by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A terrific modern paper on the difficulties of scaling Linux to many cores.*
- [CSG99] “Parallel Computer Architecture: A Hardware/Software Approach” by David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. *A treasure filled with details about parallel machines and algorithms. As Mark Hill humorously observes on the jacket, the book contains more information than most research papers.*
- [FLR98] “The Implementation of the Cilk-5 Multithreaded Language” by Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. *Cilk is a lightweight language and runtime for writing parallel programs, and an excellent example of the work-stealing paradigm.*
- [G83] “Using Cache Memory To Reduce Processor-Memory Traffic” by James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. *The pioneering paper on how to use bus snooping, i.e., paying attention to requests you see on the bus, to build a cache coherence protocol. Goodman's research over many years at Wisconsin is full of cleverness, this being but one example.*
- [M11] “Towards Transparent CPU Scheduling” by Joseph T. Meehan. Doctoral Dissertation at University of Wisconsin—Madison, 2011. *A dissertation that covers a lot of the details of how modern Linux multiprocessor scheduling works. Pretty awesome! But, as co-advisors of Joe's, we may be a bit biased here.*
- [SHW11] “A Primer on Memory Consistency and Cache Coherence” by Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. *A definitive overview of memory consistency and multiprocessor caching. Required reading for anyone who likes to know way too much about a given topic.*
- [SA96] “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation” by Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. *A tech report on this cool scheduling idea, from Ion Stoica, now a professor at U.C. Berkeley and world expert in networking, distributed systems, and many other things.*
- [TTG95] “Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors” by Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. *This is not the first paper on the topic, but it has citations to earlier work, and is a more readable and practical paper than some of the earlier queuing-based analysis papers.*

## 참고문헌

멀티프로세서 스케줄링 / 영문 원문 11쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

[A90] "The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors" - Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. 여러 락 구현 대안의 확장성이 어떻게 좋거나 나쁜지를 다루는 고전적인 논문이다. 시스템과 네트워킹 분야에서 매우 유명한 연구자 Tom Anderson의 논문이다. 그는 아주 훌륭한 운영체제 교재의 저자이기도 하다고 말해 두어야겠다.

[B+10] "An Analysis of Linux Scalability to Many Cores Abstract" - Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. Linux를 많은 코어로 확장할 때의 어려움을 다루는 훌륭한 현대적 논문이다.

[CSG99] "Parallel Computer Architecture: A Hardware/Software Approach" - David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. 병렬 컴퓨터와 알고리즘에 관한 세부 내용이 가득한 보물 같은 책이다. Mark Hill이 책 표지에서 재미있게 평했듯, 이 책에는 대부분의 연구 논문보다 더 많은 정보가 담겨 있다.

[FLR98] "The Implementation of the Cilk-5 Multithreaded Language" - Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. Cilk는 병렬 프로그램 작성을 위한 경량 언어이자 런타임이며, 작업 훔치기 패러다임의 훌륭한 예이다.

[G83] "Using Cache Memory To Reduce Processor-Memory Traffic" - James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. 버스에서 관찰되는 요청에 주의를 기울이는 버스 스누핑을 사용하여 캐시 일관성 프로토콜을 만드는 방법을 제시한 선구적인 논문이다. Goodman이 Wisconsin에서 오랫동안 수행한 연구에는 기발한 아이디어가 가득 하며, 이 논문은 그중 한 사례일 뿐이다.

[M11] "Towards Transparent CPU Scheduling" - Joseph T. Meehan. Doctoral Dissertation at University of Wisconsin-Madison, 2011. 현대 Linux의 멀티프로세서 스케줄링이 작동하는 방식의 많은 세부 내용을 다룬 학위 논문이다. 상당히 훌륭하다! 하지만 우리가 Joe의 공동 지도교수였으니 조금 편향되어 있을 수도 있다.

[SHW11] "A Primer on Memory Consistency and Cache Coherence" - Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. 메모리 일관성과 멀티프로세서 캐싱을 정리한 결정적인 개론서이다. 어떤 주제에 관해 지나칠 정도로 많이 알고 싶어 하는 사람이라면 반드시 읽어야 한다.

[SA96] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" - Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. 이 멋진 스케줄링 아이디어에 관한 기술 보고서이다. 저자인 Ion Stoica는 현재 U.C. Berkeley 교수이며 네트워킹, 분산 시스템 등 여러 분야의 세계적인 전문가이다.

[TTG95] "Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors" - Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. 이 주제를 처음 다룬 논문은 아니지만 이전 연구를 인용하고 있으며, 초기의 일부 큐잉 기반 분석 논문보다 읽기 쉽고 실용적이다.

## Homework (Simulation)

In this homework, we'll use `multi.py` to simulate a multi-processor CPU scheduler, and learn about some of its details. Read the related README for more information about the simulator and its options.

### Questions

1. To start things off, let's learn how to use the simulator to study how to build an effective multi-processor scheduler. The first simulation will run just one job, which has a run-time of 30, and a working-set size of 200. Run this job (called job 'a' here) on one simulated CPU as follows: `./multi.py -n 1 -L a:30:200`. How long will it take to complete? Turn on the `-c` flag to see a final answer, and the `-t` flag to see a tick-by-tick trace of the job and how it is scheduled.
2. Now increase the cache size so as to make the job's working set (size=200) fit into the cache (which, by default, is size=100); for example, run `./multi.py -n 1 -L a:30:200 -M 300`. Can you predict how fast the job will run once it fits in cache? (hint: remember the key parameter of the `warm_rate`, which is set by the `-r` flag) Check your answer by running with the solve flag (`-c`) enabled.
3. One cool thing about `multi.py` is that you can see more detail about what is going on with different tracing flags. Run the same simulation as above, but this time with `time_left` tracing enabled (`-T`). This flag shows both the job that was scheduled on a CPU at each time step, as well as how much run-time that job has left after each tick has run. What do you notice about how that second column decreases?
4. Now add one more bit of tracing, to show the status of each CPU cache for each job, with the `-C` flag. For each job, each cache will either show a blank space (if the cache is cold for that job) or a 'w' (if the cache is warm for that job). At what point does the cache become warm for job 'a' in this simple example? What happens as you change the `warmup_time` parameter (`-w`) to lower or higher values than the default?
5. At this point, you should have a good idea of how the simulator works for a single job running on a single CPU. But hey, isn't this a multi-processor CPU scheduling chapter? Oh yeah! So let's start working with multiple jobs. Specifically, let's run the following three jobs on a two-CPU system (i.e., type `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`) Can you predict how long this will take, given a round-robin centralized scheduler? Use `-c` to see if you were right, and then dive down into details with `-t`

## 과제: 시뮬레이션 (1-5)

멀티프로세서 스케줄링 / 영문 원문 12쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

이 과제에서는 multi.py로 멀티프로세서 CPU 스케줄러를 시뮬레이션하고 몇 가지 세부 사항을 배운다. 시뮬레이터와 옵션에 관한 자세한 내용은 관련 README를 읽어 보자.

### 질문

1. 먼저 효과적인 멀티프로세서 스케줄러를 만드는 방법을 알아보기 위해 시뮬레이터 사용법을 익혀 보자. 첫 시뮬레이션에서는 실행 시간이 30이고 작업 집합 크기가 200인 작업 하나만 실행한다. 다음과 같이 시뮬레이션된 CPU 하나에서 이 작업, 여기서는 'a'라는 작업을 실행하자: `./multi.py -n 1 -L a:30:200`. 완료하는 데 얼마나 걸릴까? `-c` 플래그를 켜서 최종 답을 확인하고, `-t` 플래그를 켜서 작업과 그 스케줄링 방식을 매 틱마다 추적해 보자.
2. 이제 작업 집합, 즉 크기 200의 작업 집합이 캐시에 들어가도록 캐시 크기를 늘려 보자. 기본 캐시 크기는 100이다. 예를 들어 `./multi.py -n 1 -L a:30:200 -M 300`을 실행하자. 작업 집합이 캐시에 들어가면 작업이 얼마나 빨리 실행될지 예측할 수 있는가? 힌트: `-r` 플래그로 설정하는 핵심 매개변수 `warm_rate`를 기억하자. 해당 표시 플래그 `-c`를 켜고 실행하여 답을 확인하자.
3. multi.py의 좋은 점 하나는 여러 추적 플래그로 내부에서 무슨 일이 일어나는지 더 자세히 볼 수 있다는 것이다. 위와 같은 시뮬레이션을 실행하되 이번에는 `time_left` 추적을 활성화하자(`-T`). 이 플래그는 매 시간 단계에서 CPU에 스케줄링된 작업과, 각 틱의 실행이 끝난 뒤 그 작업에 남은 실행 시간을 모두 보여 준다. 두 번째 열의 값이 감소하는 방식에서 무엇을 발견했는가?
4. 이제 `-C` 플래그로 추적 정보를 하나 더 추가하여 각 작업에 대한 각 CPU 캐시의 상태를 표시하자. 각 작업에 대해 캐시는 해당 작업의 캐시가 차가운 상태이면 공백을, 워밍업된 상태이면 'w'를 표시한다. 이 단순한 예에서 작업 'a'의 캐시는 어느 시점에 워밍업되는가? `warmup_time` 매개변수(`-w`)를 기본값보다 낮거나 높게 바꾸면 어떤 일이 일어나는가?
5. 이제 CPU 하나에서 작업 하나를 실행할 때 시뮬레이터가 어떻게 동작하는지 잘 이해했을 것이다. 그런데 이 장은 멀티프로세서 CPU 스케줄링을 다루는 장 아니었던가? 그렇다! 그러면 이제 여러 작업을 다뤄 보자. 구체적으로 CPU 두 개가 있는 시스템에서 다음 세 작업을 실행하자. 즉, `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`을 입력하자. 라운드 로빈 중앙 집중식 스케줄러를 사용할 때 얼마나 오래 걸릴지 예측할 수 있는가? `-c`로 예측이 맞았는지 확인한 다음, `-t`로 세부 동작을 살펴보자.

to see a step-by-step and then `-C` to see whether caches got warmed effectively for these jobs. What do you notice?

6. Now we'll apply some explicit controls to study **cache affinity**, as described in the chapter. To do this, you'll need the `-A` flag. This flag can be used to limit which CPUs the scheduler can place a particular job upon. In this case, let's use it to place jobs 'b' and 'c' on CPU 1, while restricting 'a' to CPU 0. This magic is accomplished by typing this `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`; don't forget to turn on various tracing options to see what is really happening! Can you predict how fast this version will run? Why does it do better? Will other combinations of 'a', 'b', and 'c' onto the two processors run faster or slower?
7. One interesting aspect of caching multiprocessors is the opportunity for better-than-expected speed up of jobs when using multiple CPUs (and their caches) as compared to running jobs on a single processor. Specifically, when you run on  $N$  CPUs, sometimes you can speed up by more than a factor of  $N$ , a situation entitled **super-linear speedup**. To experiment with this, use the job description here (`-L a:100:100,b:100:100,c:100:100`) with a small cache (`-M 50`) to create three jobs. Run this on systems with 1, 2, and 3 CPUs (`-n 1`, `-n 2`, `-n 3`). Now, do the same, but with a larger per-CPU cache of size 100. What do you notice about performance as the number of CPUs scales? Use `-c` to confirm your guesses, and other tracing flags to dive even deeper.
8. One other aspect of the simulator worth studying is the per-CPU scheduling option, the `-p` flag. Run with two CPUs again, and this three job configuration (`-L a:100:100,b:100:50,c:100:50`). How does this option do, as opposed to the hand-controlled affinity limits you put in place above? How does performance change as you alter the 'peek interval' (`-P`) to lower or higher values? How does this per-CPU approach work as the number of CPUs scales?
9. Finally, feel free to just generate random workloads and see if you can predict their performance on different numbers of processors, cache sizes, and scheduling options. If you do this, you'll soon be a **multi-processor scheduling master**, which is a pretty awesome thing to be. Good luck!

## 과제: 시뮬레이션 (5-9)

멀티프로세서 스케줄링 / 영문 원문 13쪽에 대응합니다. 그림과 필기는 바로 앞의 원문 페이지에서 함께 확인하세요.

5. (계속) -t로 단계별 동작을 확인한 다음, -C로 이 작업들에 대해 캐시가 효과적으로 워밍업되었는지 확인하자. 무엇을 발견했는가?
6. 이제 이 장에서 설명한 캐시 친화성을 살펴보기 위해 명시적인 제어를 적용해 보자. 이를 위해 -A 플래그가 필요하다. 이 플래그를 사용하면 스케줄러가 특정 작업을 배치할 수 있는 CPU를 제한할 수 있다. 여기서는 작업 'b'와 'c'를 CPU 1에 배치하고 'a'는 CPU 0으로 제한하자. ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1을 입력하면 된다. 실제로 무슨 일이 일어나는지 확인할 수 있도록 여러 추적 옵션을 켜는 것도 잊지 말자! 이 방식이 얼마나 빨리 실행될지 예측할 수 있는가? 왜 더 좋은 성능을 보이는가? 두 프로세서에 'a', 'b', 'c'를 다르게 조합하여 배치하면 더 빨라질까, 더 느려질까?
7. 캐시를 사용하는 멀티프로세서의 흥미로운 특징 하나는 단일 프로세서에서 작업을 실행할 때와 비교하여 여러 CPU와 그 캐시를 사용하면 예상보다 더 큰 속도 향상을 얻을 수도 있다는 것이다. 구체적으로 CPU N개에서 실행하면 때로는 N배를 넘는 속도 향상을 얻을 수 있다. 이를 초선형 속도 향상이라고 한다. 이를 실험하려면 여기의 작업 명세(-L a:100:100,b:100:100,c:100:100)를 작은 캐시(-M 50)와 함께 사용하여 세 작업을 만들자. CPU가 각각 1개, 2개, 3개인 시스템에서 실행하자(-n 1, -n 2, -n 3). 이제 CPU별 캐시 크기를 100으로 늘리고 똑같이 실행하자. CPU 수가 늘어날 때 성능에 관해 무엇을 발견했는가? -c로 예측을 확인하고 다른 추적 플래그로 더 깊이 살펴보자.
8. 살펴볼 만한 또 다른 시뮬레이터 기능은 CPU별 스케줄링 옵션인 -p 플래그이다. CPU 두 개에서 이 세 작업 구성(-L a:100:100,b:100:50,c:100:50)을 다시 실행하자. 위에서 직접 제어한 친화성 제한과 비교하면 이 옵션의 성능은 어떠한가? '들여다보기 간격'인 peek interval(-P)을 더 낮거나 높은 값으로 바꾸면 성능이 어떻게 달라지는가? CPU 수가 늘어날 때 이러한 CPU별 접근법은 어떻게 작동하는가?
9. 마지막으로 임의의 작업 부하를 자유롭게 생성하고, 프로세서 수와 캐시 크기 및 스케줄링 옵션이 다를 때 그 성능을 예측할 수 있는지 살펴보자. 이렇게 하다 보면 곧 멀티프로세서 스케줄링의 달인이 될 것이다. 꽤 멋진 일이다. 행운을 빈다!